JAVA BACKEND

INTERVIEW HANDBOOK

Product-Based & Service-Based Companies — Complete Reference

Prepared for: Sathiyavani

3 years experience · Java Backend Development

Confidential C

Table of Contents

Table of Contents 2
PART 1 10
Core Java (Interview Level) 10
1.1 OOP Fundamentals 10
The Four Pillars 10
Polymorphism: Compile-time vs Runtime 10
Why composition over inheritance? 10
1.2 Java Memory & Object Basics 11
Stack vs Heap 11
String Pool & String Immutability 11
== vs equals() vs hashCode() 11
Object Class Methods Every Class Inherits 12
1.3 Access Modifiers & Keywords 12
final / static / this / super — precisely 12
1.4 Interfaces & Abstract Classes 12
1.5 Exception Handling 13
try-with-resources 13
finally, and the return-value trap 13
1.6 Generics 14
Type Erasure 14
1.7 Core Java Interview Q&A Bank 15
1.8 Cheat Sheet — Core Java (last-minute revision) 16
PART 2 17
Collections Internals 17
2.1 The Collections Hierarchy 17
2.2 List Implementations 17
ArrayList growth 17
2.3 HashMap Internals (the single most-asked collections topic) 17
Step by step: put(key, value) 18
Time complexity summary 18
HashMap vs Hashtable vs ConcurrentHashMap vs LinkedHashMap vs TreeMap 18
2.4 Set Implementations 19
2.5 Comparable vs Comparator 19
2.6 Iterator, ListIterator & Fail-Fast vs Fail-Safe 19
2.7 Queue, Deque & PriorityQueue 20

Confidential C

2.8 Immutable Collections & Utility Classes 20
2.9 Collections Interview Q&A Bank 21
2.10 Cheat Sheet — Collections (last-minute revision) 22
PART 3 23
Multithreading & Concurrency Deep Dive 23
3.1 Process vs Thread, and the Thread Lifecycle 23
Thread Lifecycle 23
Creating threads: 3 ways 23
3.2 synchronized, Intrinsic Locks & the Monitor 24
3.3 volatile & the Java Memory Model 24
happens-before, briefly 25
3.4 Explicit Locks: ReentrantLock, ReadWriteLock, StampedLock 25
3.5 Atomic Classes & CAS 25
3.6 Executor Framework & Thread Pools 27
ThreadPoolExecutor — the parameters interviewers actually test 27
Execution order of a submitted task 27
Future vs CompletableFuture 27
3.7 ConcurrentHashMap Locking — Revisited in Depth 28
3.8 Producer-Consumer Pattern 28
3.9 Deadlock 28
3.10 Thread-Safety Patterns 28
Thread-safe Singleton (asked very often) 29
3.11 Multithreading Interview Q&A Bank 30
3.12 Cheat Sheet — Multithreading (last-minute revision) 32
PART 4 33
Java 8+ Features 33
4.1 Lambda Expressions — Starting From Why 33
4.2 Functional Interfaces 33
Method References — a shorthand for a shorthand 34
4.3 Streams — What Problem They Actually Solve 35
Intermediate vs Terminal Operations, and Laziness 35
Collectors — turning a stream back into something useful 35
reduce() — the fundamental building block 35
Parallel Streams — powerful, but not free 36
4.4 Optional<T> — Making 'Might Be Absent' Explicit 36
4.5 default and static Methods in Interfaces 36
4.6 java.time — The Modern Date/Time API 37
4.7 Java 8+ Interview Q&A Bank 38

Confidential C

4.8 Cheat Sheet — Java 8+ (last-minute revision) 39
PART 5 40
JVM Internals 40
5.1 JVM Architecture — the Big Picture 40
5.2 Class Loading — How .class Files Get Into Memory 40
Phase 1: Loading 40
Phase 2: Linking (3 sub-steps) 40
Phase 3: Initialization 40
Classloader Hierarchy & Delegation 41
5.3 Runtime Memory Areas 42
The Heap in Detail — Generational GC 42
Object lifecycle through the heap 42
5.4 Garbage Collection Algorithms 42
Minor GC vs Major GC vs Full GC — precise definitions 43
5.5 Interpreter vs JIT Compiler — Why Java Isn't 'Slow' 43
5.6 JVM Interview Q&A Bank 45
5.7 Cheat Sheet — JVM Internals (last-minute revision) 46
PART 6 47
Spring Framework 47
6.1 Inversion of Control & Dependency Injection — From First Principles 47
The 3 ways to inject a dependency 47
6.2 The IoC Container: BeanFactory vs ApplicationContext 48
6.3 Declaring Beans: Component Scanning & @Configuration 48
6.4 Bean Lifecycle 49
6.5 Bean Scopes 49
6.6 Circular Dependencies 49
6.7 Aspect-Oriented Programming (AOP) 51
How AOP actually works: proxies 51
6.8 Spring MVC Request Flow: DispatcherServlet 52
6.9 Spring Framework Interview Q&A Bank 53
6.10 Cheat Sheet — Spring Framework (last-minute revision) 55
PART 7 56
Spring Boot 56
7.1 What Spring Boot Actually Adds Over Plain Spring 56
7.2 @SpringBootApplication — What It Actually Is 56
7.3 Auto-Configuration Internals 56
How Boot finds these classes: AutoConfiguration.imports 57
7.4 Starters 57

Confidential C

7.5 Embedded Servers 58
7.6 Configuration: application.properties/yml, @Value, @ConfigurationProperties 58
7.7 Profiles 58
7.8 Actuator 59
7.9 Centralized Exception Handling: @ControllerAdvice 59
7.10 Bean Validation: @Valid and @Validated 59
7.11 What SpringApplication.run() Actually Does 60
7.12 Testing Slices 60
7.13 Spring Boot Interview Q&A Bank 61
7.14 Cheat Sheet — Spring Boot (last-minute revision) 62
PART 8 63
Hibernate & JPA 63
8.1 JPA vs Hibernate — Spec vs Implementation 63
8.2 Entity Mapping Basics 63
8.3 Entity Relationships 64
Fetch Types: LAZY vs EAGER 64
Cascade Types 64
8.4 The Persistence Context — Hibernate's First-Level Cache 65
Entity Lifecycle States 65
8.5 The N+1 Select Problem — the Single Most-Asked Hibernate Topic 66
The fixes 66
8.6 JPQL vs Native Queries vs Criteria API 66
8.7 @Transactional — Propagation & Isolation 66
8.8 Optimistic vs Pessimistic Locking 67
8.9 The equals()/hashCode() Trap in Entities 68
8.10 Hibernate/JPA Interview Q&A Bank 69
8.11 Cheat Sheet — Hibernate & JPA (last-minute revision) 71
PART 9 72
Spring Security + JWT 72
9.1 Authentication vs Authorization 72
9.2 The Security Filter Chain 72
9.3 Configuring Security (Modern Lambda DSL) 72
9.4 How a Login Request Actually Flows Through Spring Security 73
9.5 JWT (JSON Web Tokens) — Stateless Authentication 74
Verification — what happens on every incoming request 74
Implementing the filter 74
Access Tokens vs Refresh Tokens 74
9.6 Method-Level Security 75

Confidential C

9.7 CORS and CSRF — Frequently Confused 75
9.8 Common Vulnerabilities — Interview Awareness Checklist 75
9.9 Spring Security + JWT Interview Q&A Bank 77
9.10 Cheat Sheet — Spring Security + JWT (last-minute revision) 78
PART 10 79
Microservices 79
10.1 Monolith vs Microservices — the Actual Trade-off 79
10.2 Service Discovery 79
10.3 API Gateway 80
10.4 Inter-Service Communication: Sync vs Async 80
10.5 Circuit Breaker Pattern (Resilience4j) 81
10.6 Centralized Configuration (Spring Cloud Config) 81
10.7 Distributed Transactions: the Saga Pattern 81
10.8 Idempotency & Distributed Tracing 82
10.9 CAP Theorem (a Standard Follow-Up Question) 82
10.10 API Versioning 83
10.11 Microservices Interview Q&A Bank 84
10.12 Cheat Sheet — Microservices (last-minute revision) 85
PART 11 86
SQL 86
11.1 Joins 86
11.2 Normalization 86
11.3 Primary Key, Foreign Key, Unique Key 87
11.4 ACID Properties 87
11.5 Indexes — the Single Most Practically Important SQL Topic 89
11.6 GROUP BY, HAVING vs WHERE, Aggregate Functions 89
11.7 Window Functions 89
11.8 Subqueries vs CTEs 90
11.9 Reading an EXPLAIN Plan (the Practical Skill) 90
11.10 Classic SQL Coding Questions 92
Find the Nth highest salary 92
Find duplicate rows 92
Department-wise top earner (a very common one) 92
Delete duplicate rows, keeping only one copy 92
11.11 SQL Interview Q&A Bank 93
11.12 Cheat Sheet — SQL (last-minute revision) 94
PART 12 95
Design Patterns 95

Confidential C

12.1 SOLID Principles — the Foundation Patterns Are Built On 95
12.2 Creational Patterns — Controlling Object Creation 95
Singleton 95
Factory Method 96
Builder 96
Abstract Factory 96
Prototype 96
12.3 Structural Patterns — Composing Objects 97
Adapter 97
Decorator 97
Facade 97
Proxy 97
Composite 97
12.4 Behavioral Patterns — Communication Between Objects 98
Strategy 98
Observer 98
Template Method 98
Chain of Responsibility 98
Command 98
12.5 Design Patterns Interview Q&A Bank 100
12.6 Cheat Sheet — Design Patterns & SOLID (last-minute revision) 101
PART 13 102
System Design Basics 102
13.1 Vertical vs Horizontal Scaling 102
13.2 Load Balancing 102
13.3 Caching Strategies 103
Cache Invalidation — 'the two hard problems in CS' 103
13.4 Database Scaling: Replication & Sharding 103
13.5 CDN (Content Delivery Network) 104
13.6 Rate Limiting Algorithms 104
13.7 Consistent Hashing (Brief) 104
13.8 Worked Example: Design a URL Shortener 105
Step 1: Clarify Requirements 105
Step 2: API Design 105
Step 3: Data Model 105
Step 4: The Interesting Part — Generating the Short Code 105
Step 5: High-Level Architecture 105
Step 6: Scaling Further 105

Confidential C

13.9 System Design Interview Q&A Bank 107
13.10 Cheat Sheet — System Design Basics (last-minute revision) 108
PART 14 109
100+ Coding Questions (DSA) 109
14.1 Two Pointers & Sliding Window 109
Two Sum (sorted array) — Two Pointers 109
Longest Substring Without Repeating Characters — Sliding Window 109
14.2 Binary Search 109
14.3 Linked Lists 110
Reverse a Linked List (iterative) 110
Detect a Cycle — Floyd's Tortoise and Hare 110
14.4 Stacks & Queues 111
Valid Parentheses 111
14.5 Trees & BSTs 111
Maximum Depth of a Binary Tree 111
Level Order Traversal (BFS) 111
14.6 Graphs: BFS & DFS 112
14.7 Recursion & Backtracking 112
14.8 Dynamic Programming 112
Coin Change (minimum coins to make an amount) 112
14.9 Greedy Algorithms 113
14.10 Big-O Cheat Sheet 114
14.11 Full Practice Checklist — 100+ Problems by Pattern 115
Arrays & Strings 115
Searching & Sorting 115
Linked Lists 115
Stacks & Queues 115
Trees & BSTs 115
Graphs 115
Recursion & Backtracking 115
Dynamic Programming 115
Greedy 115
Heaps / Priority Queues 115
Hashing / Design 116
Matrix 116
14.12 DSA Interview Approach Q&A Bank 117
14.13 Cheat Sheet — DSA Patterns (last-minute revision) 118
PART 15 119

Confidential C

One-Day Revision Guide 119
15.1 Suggested One-Day Schedule 119
15.2 The 60-Second Version of Every Part 119
Core Java 119
Collections 119
Multithreading 119
Java 8+ 119
JVM Internals 120
Spring Framework 120
Spring Boot 120
Hibernate & JPA 120
Spring Security + JWT 120
Microservices 121
SQL 121
Design Patterns & SOLID 121
System Design Basics 121
DSA 121
15.3 What to Expect: Typical Round Structure 121
15.4 Resume & Project Deep-Dive Prep 122
15.5 Day-Of Checklist 122

Confidential C

PART 1

Core Java (Interview Level)

This part covers everything interviewers actually probe on Core Java: OOP, object/memory basics, access control, interfaces vs abstract classes, exceptions, and generics — each with the "why", not just the "what".

1.1 OOP Fundamentals

The Four Pillars

Pillar	What interviewers want to hear
Encapsulation	Hiding internal state behind private fields + public getters/setters; protects invariants, not just "data hiding"
Abstraction	Exposing WHAT an object does, hiding HOW — via interfaces/abstract classes
Inheritance	Code reuse + "is-a" relationship; overused in practice, prefer composition when unsure
Polymorphism	One interface, many implementations — compile-time (overloading) vs runtime (overriding)

Polymorphism: Compile-time vs Runtime

```
class Calculator {
    int add(int a, int b) { return a + b; }        // overload 1
    double add(double a, double b) { return a+b; } // overload 2 (compile-time / static
binding)
}

class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Bark"); } // runtime / dynamic binding
}

Animal a = new Dog();
a.sound();   // "Bark" — decided at RUNTIME via the vtable, not the reference type
```

Interview trap

Overloading is resolved by the COMPILER using the reference type + argument types (static binding).

Overriding is resolved by the JVM using the ACTUAL object type at runtime (dynamic binding via method table).

If asked "what does a.sound() print if Animal has no sound()", the answer hinges on this distinction — fields are NEVER polymorphic in Java, only methods are.

Why composition over inheritance?

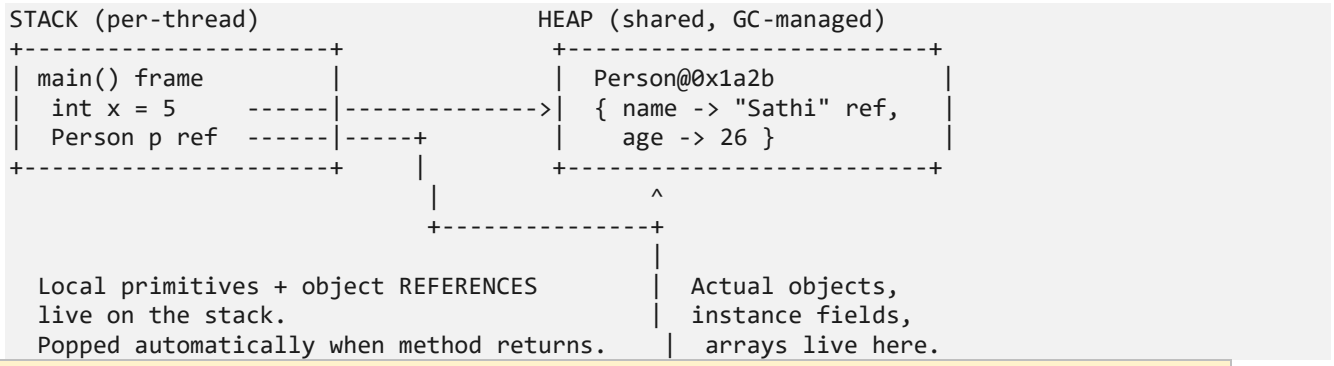
- Inheritance breaks encapsulation — a subclass depends on the parent's implementation details, not just its contract (the "fragile base class" problem).
- Composition ("has-a") lets you swap behavior at runtime; inheritance locks it at compile time.

Confidential C

- Real answer interviewers want: "Favor composition, use inheritance only for genuine is-a relationships with stable contracts."

1.2 Java Memory & Object Basics

Stack vs Heap



Key line to say in interviews

"Primitives and references live on the stack; the objects those references point to live on the heap. Only the heap is garbage-collected — the stack is cleaned up automatically when a method frame is popped."

String Pool & String Immutability

```
String a = "hello";        // goes to String Constant Pool (SCP) inside heap
String b = "hello";        // reuses SAME reference from SCP
String c = new String("hello"); // forces a NEW object on heap (outside pool)

a == b        // true  (same SCP reference)
a == c        // false (different heap objects)
a.equals(c)   // true  (content is equal)
```

- Strings are immutable so they can be safely shared (pool), used as HashMap keys (hashcode cached), and are thread-safe by default.
- StringBuilder is mutable + not synchronized (fast, single-threaded use). StringBuffer is mutable + synchronized (thread-safe, slower).

== vs equals() vs hashCode()

Method	Meaning
==	Reference comparison for objects; value comparison for primitives
.equals()	Logical/content equality — default is == unless overridden (e.g., String, Integer, records)
.hashCode()	Integer used to bucket objects in hash-based collections; MUST be overridden alongside equals()

The contract you must recite

1) If a.equals(b) is true, then a.hashCode() == b.hashCode() MUST be true.

2) The reverse is NOT required — equal hashCodes don't imply equal objects (collision).

3) Break this contract and HashMap/HashSet silently lose or duplicate entries — classic production bug when someone overrides equals() but forgets hashCode().

Confidential C

Object Class Methods Every Class Inherits

- toString(), equals(), hashCode(), getClass(), clone(), wait()/notify()/notifyAll(), finalize() (deprecated since Java 9).

1.3 Access Modifiers & Keywords

Modifier	Visibility
private	Same class only
default (no modifier)	Same package only
protected	Same package + subclasses (even in other packages)
public	Everywhere

final / static / this / super — precisely

- final variable: value can't be reassigned after init (but for objects, the reference is final, not the internal state — a final List can still have items added).
- final method: cannot be overridden by subclasses.
- final class: cannot be extended at all (e.g. String, Integer).
- static: belongs to the CLASS, not the instance — one copy shared across all objects, lives in the Metaspace (Java 8+).
- this: reference to the current instance; used to disambiguate fields from parameters, or to chain constructors (this(...)).
- super: reference to the parent class; calls parent constructor (super(...)) or parent method (super.method()).

Common gotcha

static methods CANNOT be overridden — they can only be 'hidden' (method hiding). Calling a static method through a reference resolves at compile time based on the REFERENCE type, unlike instance method overriding which resolves at runtime based on the OBJECT type.

1.4 Interfaces & Abstract Classes

Interface	Abstract Class
Pure contract — "can-do" relationship	Partial implementation — "is-a" relationship
Multiple inheritance allowed (implements A, B, C)	Single inheritance only (extends one class)
No instance state (only constants, implicitly public static final)	Can hold instance fields and constructors
default/static/private methods since Java 8/9	Can mix abstract + concrete methods since always
Use when unrelated classes share behavior	Use when classes share a common base + some state

```
interface Payable {
    double calculateTax();          // implicitly public abstract
    default void printReceipt() {   // Java 8+: default method, has a body
        System.out.println("Tax: " + calculateTax());
    }
}

abstract class Employee {
```


Confidential C

```
    protected String name;                  // state allowed
    Employee(String name) { this.name = name; } // constructor allowed
    abstract double getSalary();            // must be implemented by subclass
    void printName() { System.out.println(name); } // concrete method, inherited as-is
}
```

Interview one-liner

"I use an interface to define a capability multiple unrelated classes can promise (Comparable, Runnable). I use an abstract class when I need to share state or partial implementation across closely related classes."

1.5 Exception Handling

```
                    Throwable
                   /         \
              Error           Exception
          (OutOfMemory,      /          \
           StackOverflow)   /            \
                RuntimeException      Checked Exceptions
                (Unchecked)           (IOException, SQLException)
                NullPointerException
                ArrayIndexOOB
                ClassCastException
                IllegalArgumentException
```

- Checked exceptions: must be declared (throws) or caught — compiler enforces it. Represent recoverable conditions (file not found, network down).
- Unchecked (RuntimeException): compiler doesn't force handling. Represent programming bugs (null deref, bad cast, illegal argument).
- Error: JVM-level problems (OutOfMemoryError, StackOverflowError) — not meant to be caught/recovered from.

try-with-resources

```
try (BufferedReader br = new BufferedReader(new FileReader("f.txt"))) {
    return br.readLine();
} catch (IOException e) {
    log.error("read failed", e);
}
// br.close() is called AUTOMATICALLY, even if an exception is thrown —
// resource must implement AutoCloseable. Eliminates finally-block leaks.
```

finally, and the return-value trap

```
static int test() {
    try {
        return 1;
    } finally {
        return 2;   // BAD PRACTICE: this SWALLOWS the try's return AND any exception
    }
}
// test() returns 2. finally always runs; a return/throw inside finally
// overrides whatever the try block was about to do.
```

Custom exceptions — production pattern

Extend RuntimeException for business rule violations you want to bubble up to a @ControllerAdvice / global handler without polluting every method signature with 'throws'.

Always chain the original cause: throw new OrderNotFoundException("id="+id, originalEx); — never swallow the stack trace.

Confidential C

1.6 Generics

Generics give compile-time type safety and remove the need for manual casting. They exist only at compile time — erased at runtime (type erasure).

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}

// Bounded type — T must be a Number or subclass
class NumericBox<T extends Number> {
    T value;
    double doubled() { return value.doubleValue() * 2; }
}

// Wildcards
void printAll(List<? extends Number> list) { }  // read-only producer (PECS: Producer Extends)
void addNumbers(List<? super Integer> list) { }  // write-only consumer (PECS: Consumer Super)
```

PECS rule (this gets asked a lot)

"Producer Extends, Consumer Super."

Use <? extends T> when you only READ from the structure.

Use <? super T> when you only WRITE into the structure.

Example: Collections.copy(List<? super T> dest, List<? extends T> src)

Type Erasure

- At runtime, List<String> and List<Integer> are both just List — the JVM has no idea what T was.
- Consequence: you cannot do new T(), new T[], or T.class inside a generic class; you cannot overload methods that differ only by generic type.

Confidential C

1.7 Core Java Interview Q&A Bank

Rated by how often they actually come up: [Very High] [High] [Medium]

Q: Why is Java called platform-independent, and how exactly does the JVM achieve it? [Very High]

Java source compiles to bytecode (.class), not native machine code. The JVM (platform-specific) interprets/JIT-compiles that same bytecode on any OS — "write once, run anywhere". The JDK/JVM is platform-dependent; the bytecode is not.

Q: What's the difference between JDK, JRE, and JVM? [Very High]

JVM: runs bytecode (the execution engine). JRE: JVM + core libraries needed to RUN Java apps. JDK: JRE + compiler (javac), debugger, dev tools — needed to BUILD Java apps.

Q: Why are Strings immutable in Java? [Very High]

Security (can't be altered after passed to a method, e.g. file paths, network URLs), safe HashMap keys (hashcode computed once and cached), thread-safety without synchronization, and enables the String pool for memory efficiency.

Q: Can you override a private or static method? [High]

No. Private methods aren't inherited at all, so there's nothing to override. Static methods can be 'hidden' by a same-signature static method in the subclass, but that's resolved at compile time by reference type — it is NOT polymorphism.

Q: What happens if you don't override hashCode() but override equals()? [Very High]

You break the equals/hashCode contract. Two 'equal' objects can land in different hash buckets, so HashSet.contains() or HashMap.get() can return false/null for an object that IS logically present. This is a very common real bug.

Q: Difference between abstract class and interface — when do you actually pick one over the other in a real codebase? [High]

Pick abstract class when subclasses share state/behavior and are tightly related (template method pattern). Pick interface when you're defining a capability unrelated classes can opt into (Comparable, Serializable) — also required if the class already extends something else, since Java allows only single inheritance of classes.

Q: What's the difference between checked and unchecked exceptions, and how do you decide which to throw from your own code? [High]

Checked = caller can reasonably be expected to recover (retry, fallback) — compiler forces handling. Unchecked = programming errors / unrecoverable business rule violations. Most modern codebases (Spring included) lean toward unchecked exceptions to avoid throws-clause pollution, using a global @ExceptionHandler instead.

Q: What is method overloading resolution ambiguity, and give an example. [Medium]

When multiple overloaded methods are equally applicable via autoboxing/widening, compiler throws an ambiguity error. E.g. print(int) and print(Integer) both exist and you call print(null) — compiler can't decide, fails to compile.

Q: Why can't we create a generic array like new T[10]? [Medium]

Due to type erasure, T isn't known at runtime, and array creation needs a concrete component type for runtime type-checking (arrays are covariant and check element types on insertion via ArrayStoreException). Java disallows it to prevent heap pollution.

Q: What is the diamond problem, and how does Java avoid it? [Medium]

Multiple inheritance of STATE via classes causes ambiguity if two parents define the same field/method differently. Java disallows multiple class inheritance entirely. For interfaces with conflicting default methods, Java forces the implementing class to explicitly override and resolve the conflict — it never silently picks one.

Confidential C

1.8 Cheat Sheet — Core Java (last-minute revision)

- OOP: Encapsulation (hide state) | Abstraction (hide how) | Inheritance (reuse, is-a) | Polymorphism (overload=compile-time, override=runtime)
- Stack = primitives + references, auto-cleaned. Heap = objects, GC-managed.
- == compares references; equals() compares content; hashCode() buckets for hash collections — override both together, always.
- private < default < protected < public (increasing visibility)
- final var = no reassignment | final method = no override | final class = no extend | static = class-level, one copy
- Interface = contract, multiple implements allowed, no state. Abstract class = partial impl, single extends, can hold state.
- Throwable → Error (JVM-fatal) / Exception → RuntimeException (unchecked) + checked (IOException etc.)
- try-with-resources auto-closes AutoCloseable resources — prefer over manual finally blocks
- Generics: type erasure at runtime; PECS — Producer Extends, Consumer Super

Confidential C

PART 2

Collections Internals

Collections questions separate candidates who memorized method names from candidates who understand what's happening underneath. This part goes bucket-by-bucket into HashMap, tree structures, and the iterator contract.

2.1 The Collections Hierarchy

```
                    Iterable
                        |
                    Collection
        /               |               \
     List              Set              Queue
ArrayList           HashSet           PriorityQueue
LinkedList          LinkedHashSet     ArrayDeque
Vector/Stack        TreeSet (SortedSet, NavigableSet)

                      Map  (NOT a Collection — separate root)
                   /    |    \
              HashMap  TreeMap  LinkedHashMap
              (SortedMap, NavigableMap for TreeMap)
```

Interview trap

Map does NOT extend Collection. It's a completely separate interface (key-value pairs, not a group of single elements). Saying "Map is a Collection" is an instant red flag to interviewers.

2.2 List Implementations

ArrayList	LinkedList
Backed by a resizable Object[] array	Backed by a doubly-linked list of nodes
get(i): O(1) — direct index access	get(i): O(n) — must walk from head/tail
add at end: O(1) amortized; middle insert: O(n) shift	add/remove at ends: O(1); middle insert: O(n) traversal but O(1) splice
Less memory overhead per element	Extra memory per node (prev/next pointers)
Better for random access & iteration	Better for frequent insert/delete at ends (also implements Deque)

ArrayList growth

- Default initial capacity: 10. When full, grows to 1.5x the old capacity (newCapacity = oldCapacity + oldCapacity>>1), then copies elements via Arrays.copyOf — an O(n) operation, amortized to O(1) per add over time.
- Vector is the legacy synchronized twin of ArrayList — grows 2x, every method is synchronized (slow); avoid in new code, use Collections.synchronizedList() or CopyOnWriteArrayList instead.

2.3 HashMap Internals (the single most-asked collections topic)

HashMap is backed by an array of buckets — Node<K,V>[] table. Each bucket holds either a singly-linked list of entries (default) or, once a bucket gets crowded, a Red-Black Tree.

Confidential C

```
table[]                Bucket contents
  [0] -> null
  [1] -> Node(k1,v1) -> Node(k5,v5) -> Node(k9,v9)   <- collision chain (linked list)
  [2] -> null
  [3] -> Node(k3,v3)
   ...
  [n] -> null

index = (n - 1) & hash(key)      // n = table.length, always a power of 2
hash(key) = (h = key.hashCode()) ^ (h >>> 16)   // spreads high bits into low bits
```

Step by step: put(key, value)

1. Compute hash(key) — XOR the hashCode with its own upper 16 bits shifted down, to reduce collisions from poor hashCode() implementations.
2. index = (table.length - 1) & hash — works only because table.length is always a power of 2, making this equivalent to a fast modulo.
3. If bucket is empty, insert directly. If not, walk the chain: if a key matches (== or .equals()), overwrite the value. Otherwise append to the end (Java 8+: appended at tail, not head, unlike Java 7).
4. If the chain length at that bucket reaches TREEIFY_THRESHOLD (8) AND table capacity >= 64, the chain is converted into a Red-Black Tree for that bucket (O(log n) lookup instead of O(n)).
5. If size > capacity * loadFactor (default 0.75), the table resizes: capacity doubles, and every entry is rehashed into the new table.

Why load factor 0.75?

It's the sweet spot between space and time: too low (e.g. 0.5) wastes memory with mostly-empty buckets; too high (e.g. 0.9) causes long collision chains and slower lookups. 0.75 is the JDK's tuned default — you can pass a custom one in the constructor if asked to justify it.

Why treeify at 8, and untreeify at 6?

Treeifying has overhead (tree nodes are heavier than list nodes), so it only pays off once a bucket is badly collided. The JDK authors chose 8 based on Poisson-distribution math — with a good hash function, the probability of 8+ collisions in one bucket is astronomically small, so this mainly guards against malicious/degenerate hashCode() implementations (hash-flooding DoS).

6 (not 8) is used for untreeify to avoid rapid thrashing back and forth if a bucket size hovers right at the threshold.

Time complexity summary

Operation	Average / Worst case
get / put / remove	O(1) average; O(log n) worst case (post-treeify) — O(n) pre-Java-8 worst case
containsKey	O(1) average
iteration	O(capacity + size) — this is why an oversized initial capacity hurts iteration speed

HashMap vs Hashtable vs ConcurrentHashMap vs LinkedHashMap vs TreeMap

Type	Key behavior
HashMap	No ordering guarantee, allows one null key + multiple null values, NOT thread-safe

Confidential C

Type	Key behavior
Hashtable	Legacy, synchronized on every method (coarse lock), no null keys/values allowed — avoid in new code
ConcurrentHashMap	Thread-safe via bucket-level locking (Java 8+: CAS + synchronized on first node per bin, not a single global lock) — no null keys/values
LinkedHashMap	HashMap + a doubly-linked list threading through entries to preserve insertion (or access) order — base for LRU cache implementations
TreeMap	Red-Black tree — keys always sorted (natural order or a Comparator); O(log n) for get/put, not O(1)

Production example: LRU cache in 5 lines

```
class LRUCache<K,V> extends LinkedHashMap<K,V> {
  private final int capacity;
  LRUCache(int cap){ super(16, 0.75f, true); this.capacity = cap; } // accessOrder=true
  protected boolean removeEldestEntry(Map.Entry<K,V> eldest){ return size() > capacity; }
}
```

2.4 Set Implementations

- HashSet: backed internally by a HashMap where every value is a dummy constant (PRESENT) — the elements you add become the map's keys. Same O(1) average, same collision behavior.
- LinkedHashSet: HashSet + insertion-order iteration (backed by LinkedHashMap internally).
- TreeSet: backed by a TreeMap — sorted, O(log n), supports NavigableSet methods (higher(), lower(), ceiling(), floor()).

2.5 Comparable vs Comparator

Comparable	Comparator
compareTo(T o) — defined INSIDE the class itself	compare(T a, T b) — defined OUTSIDE, as a separate class/lambda
Only one natural ordering per class	Unlimited orderings — pass a different Comparator per use-case
java.lang package	java.util package

```
class Employee implements Comparable<Employee> {
    int salary;
    public int compareTo(Employee other) { return Integer.compare(this.salary, other.salary);
}
}

// Comparator — chained, reusable, no need to touch the Employee class
Comparator<Employee> byNameThenSalary = Comparator
        .comparing((Employee e) -> e.name)
        .thenComparing(e -> e.salary, Comparator.reverseOrder());
employees.sort(byNameThenSalary);
```

2.6 Iterator, ListIterator & Fail-Fast vs Fail-Safe

Confidential C

- Iterator: hasNext(), next(), remove() — forward-only, works on any Collection.
- ListIterator: adds hasPrevious(), previous(), set(), add() — bidirectional, only for List.

```
// WRONG — throws ConcurrentModificationException
for (String s : list) { if (s.equals("x")) list.remove(s); }

// RIGHT — remove via the iterator itself
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().equals("x")) it.remove(); }
```

Fail-fast vs Fail-safe — the actual mechanism

Fail-fast (ArrayList, HashMap, HashSet): iterator holds a modCount snapshot taken at creation. Any structural change outside the iterator bumps the real modCount, and the next next()/remove() call detects the mismatch and throws ConcurrentModificationException immediately. It's a best-effort detection, not a guarantee.

Fail-safe (CopyOnWriteArrayList, ConcurrentHashMap): the iterator works on a snapshot/segment, so it never throws CME, but it may not reflect the very latest concurrent writes made after iteration started.

2.7 Queue, Deque & PriorityQueue

- ArrayDeque: resizable array-backed double-ended queue — faster than LinkedList for stack/queue use, no capacity restriction, disallows nulls. Preferred over the legacy Stack class.
- PriorityQueue: backed by a binary heap (array-based), NOT sorted overall — only guarantees the head is the smallest (or per a custom Comparator). poll() is O(log n), peek() is O(1).

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();              // natural order, min at head
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> byFirst = new PriorityQueue<>((a,b) -> a[0]-b[0]); // custom comparator, common in interview problems
```

2.8 Immutable Collections & Utility Classes

- List.of(...), Map.of(...), Set.of(...) (Java 9+): truly immutable — any mutation throws UnsupportedOperationException. Disallow null elements.
- Collections.unmodifiableList(list): a read-only VIEW — the underlying list can still change if you hold a reference to the original, and that change will show through the view.
- Arrays.asList(arr): fixed-size list backed by the array — set() works, but add()/remove() throw UnsupportedOperationException.
- Collections.synchronizedMap/-List: wraps with a single global lock on every method — simpler but far more contended than ConcurrentHashMap under load.

Confidential C

2.9 Collections Interview Q&A Bank

Q: Walk me through what happens internally when you call hashMap.put(key, value) and there's a collision. [Very High]

Compute hash(key) via h^(h>>>16), map to a bucket index via (n-1)&hash. If the bucket already has entries, walk the chain comparing hashCode then equals(); if no match is found, append a new node at the tail. If the chain now has 8+ nodes and the table has 64+ buckets, that bucket is converted to a Red-Black tree.

Q: Why does HashMap use power-of-2 table sizes? [High]

So that (n-1) & hash is a valid substitute for hash % n — bitwise AND is much faster than modulo, and only works correctly as an even distribution when n is a power of 2 (since n-1 is then all 1-bits, e.g. 15 = 0b1111).

Q: What's the initial capacity and resize trigger of a default HashMap? [High]

Default capacity 16, load factor 0.75, so it resizes (doubles) once size exceeds 12. On resize, EVERY entry is rehashed into the new, larger table — an O(n) operation, which is why setting an appropriate initial capacity matters for hot paths with a known size.

Q: Difference between HashMap and ConcurrentHashMap under concurrent writes? [Very High]

Plain HashMap is not thread-safe — concurrent structural modification can corrupt the internal linked list into a cycle, causing an infinite loop on get() in older JDKs (a real production incident class). ConcurrentHashMap (Java 8+) uses CAS operations for empty-bucket inserts and synchronizes only on the first node of a bin for updates — far less contention than a single global lock, and reads are largely lock-free.

Q: Why does ArrayList throw ConcurrentModificationException, and how do you safely remove elements while iterating? [Very High]

The iterator captures modCount at creation and checks it on every next()/remove() call — a mismatch means the list changed outside the iterator. Fix: use it.remove() via the Iterator itself, use a ListIterator, iterate a copy, use removeIf(), or use CopyOnWriteArrayList if concurrent safety is genuinely needed.

Q: When would you choose LinkedList over ArrayList in a real system? [Medium]

Rarely for pure lists — ArrayDeque usually wins even for queue/stack use cases due to better cache locality. LinkedList is justified when you need frequent O(1) insert/delete at arbitrary positions where you already hold a reference to the node (e.g., implementing an LRU's internal list yourself); otherwise ArrayList's contiguous memory layout wins on real-world performance despite worse Big-O for middle inserts.

Q: Can a HashMap have a null key? What about a TreeMap? [Medium]

HashMap allows exactly one null key (stored in bucket 0, hash treated as 0) and multiple null values. TreeMap throws NullPointerException on a null key (with the default comparator) because it needs to compare keys to place them in the tree — comparing against null fails.

Q: How would you sort a List<Employee> by department, then by salary descending? [High]

employees.sort(Comparator.comparing(Employee::getDept).thenComparing(Employee::getSalary, Comparator.reverseOrder())); — chain comparators with thenComparing rather than writing a manual multi-field compareTo.

Q: What's the difference between Comparable and Comparator, and can a class use both? [High]

Comparable defines ONE natural ordering inside the class (compareTo); Comparator defines ordering externally and you can have many. Yes — a class can implement Comparable for its default/natural order (e.g., by ID) while callers pass different Comparators for ad-hoc sorts (e.g., by name) without changing the class.

Q: Why is Iterator.remove() safe but list.remove() during a for-each loop is not? [Medium]

Iterator.remove() updates the iterator's internal expectedModCount to match the real modCount after removing, so the next hasNext()/next() call sees consistent state. Calling list.remove() directly changes modCount without informing the iterator, so the next call detects a mismatch and throws CME.

Confidential C

2.10 Cheat Sheet — Collections (last-minute revision)

- ArrayList = array, O(1) get, O(n) middle insert. LinkedList = nodes, O(1) end ops, O(n) get.
- HashMap: index=(n-1)&hash, treeify at 8 nodes (needs table>=64), resize when size > capacity*0.75 (default 16, 0.75).
- HashMap = 1 null key allowed, not thread-safe. Hashtable = no nulls, globally synchronized (legacy). ConcurrentHashMap = bucket-level locking, no nulls.
- LinkedHashMap = insertion/access order (accessOrder=true → LRU building block). TreeMap = sorted, O(log n), Red-Black tree.
- HashSet is backed by HashMap internally (dummy values). TreeSet by TreeMap. LinkedHashSet by LinkedHashMap.
- Comparable = compareTo(), one ordering, inside the class. Comparator = compare(), many orderings, outside the class, chainable via thenComparing().
- Fail-fast (ArrayList/HashMap) throws CME via modCount check. Fail-safe (CopyOnWriteArrayList/ConcurrentHashMap) iterates a snapshot, never throws CME.
- PriorityQueue = binary heap, only head is guaranteed min/max, poll() O(log n).
- List.of()/Map.of() = truly immutable. Collections.unmodifiableList() = mutable view can still change underneath. Arrays.asList() = fixed-size, set() works, add()/remove() don't.

Confidential C

PART 3

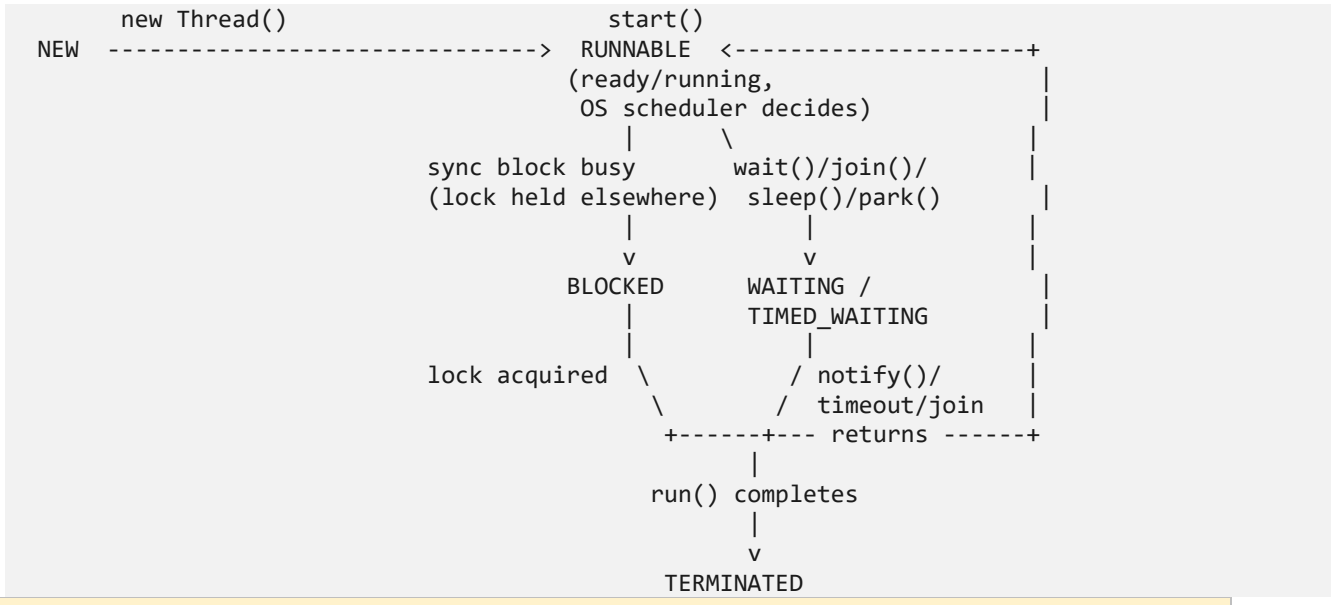
Multithreading & Concurrency Deep Dive

This is usually the topic that separates 3-year-experience candidates from junior ones. Interviewers don't just want definitions — they want to hear you reason about race conditions, visibility, and thread-pool sizing like you've been paged for one.

3.1 Process vs Thread, and the Thread Lifecycle

Process	Thread
Independent memory space, heavyweight, isolated	Shares heap/method-area with sibling threads, lightweight
Inter-process communication needs IPC (sockets, pipes)	Threads communicate directly via shared memory — fast but unsafe without sync
Crash of one process doesn't crash another	An uncaught exception in one thread can leave shared state corrupted for others

Thread Lifecycle



Interview trap

BLOCKED is specifically for a thread waiting to acquire a synchronized monitor lock held by another thread.

WAITING/TIMED_WAITING is different — a thread that called wait(), join(), sleep(), or LockSupport.park() and is voluntarily paused, not contending for a lock.

Mixing these two up in an explanation is a common giveaway that someone hasn't actually debugged a thread dump.

Creating threads: 3 ways

```
// 1) Extend Thread – rarely done in real code (wastes single inheritance)
class MyThread extends Thread { public void run() { ... } }

// 2) Implement Runnable – preferred, decouples the task from threading mechanism
```


Confidential C

```
Runnable task = () -> System.out.println("running");
new Thread(task).start();

// 3) Implement Callable<V> — like Runnable but RETURNS a value and can throw checked
exceptions
Callable<Integer> job = () -> { return 42; };
Future<Integer> future = executorService.submit(job);
```

start() vs run() — asked constantly

thread.start() creates a NEW OS-level thread and eventually calls run() on it — asynchronous.

thread.run() just calls the method like any normal method call on the CURRENT thread — no new thread is created. Calling run() directly is a classic interview gotcha question.

3.2 synchronized, Intrinsic Locks & the Monitor

Every Java object has an intrinsic lock (monitor), tracked in its object header. synchronized acquires that monitor before entering, and releases it on exit — even if an exception is thrown.

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }   // locks 'this'
    public static synchronized void resetAll() { }       // locks Counter.class object
    public void partial() {
        synchronized (this) { count++; }                 // block-level, finer-grained control
    }
}
```

Instance method sync	Static method sync
Locks the object instance (this)	Locks the Class object (Counter.class) — shared across ALL instances

Why count++ isn't atomic

count++ is really 3 steps: read count, add 1, write count back. Two threads can both read the same old value before either writes, and one increment is lost. This is THE canonical race-condition example — always have this ready with a concrete trace.

3.3 volatile & the Java Memory Model

Without synchronization, each CPU core may cache a variable's value locally — one thread's write may never become visible to another thread's cached read. volatile forces every read to go to main memory and every write to flush to main memory immediately.

volatile guarantees	volatile does NOT guarantee
Visibility — writes are immediately visible to other threads	Atomicity — volatile int count; count++ is STILL a race condition
Ordering — prevents instruction reordering around it (happens-before)	Mutual exclusion — two threads can still both read/act on a stale check-then-act sequence

```
private volatile boolean running = true;   // classic correct use: a flag checked by another
thread
public void stop() { running = false; }     // guaranteed visible to the loop below
public void loop() { while (running) { doWork(); } }

// WRONG use of volatile — this is still NOT thread-safe:
private volatile int counter = 0;
```


Confidential C

```
public void increment() { counter++; }   // read-modify-write is not atomic even with volatile
```

happens-before, briefly

- A volatile write happens-before every subsequent volatile read of the same variable — the JMM guarantees everything the writing thread did BEFORE the write is visible to the reading thread AFTER the read.
- Other happens-before sources: a monitor unlock happens-before the next lock on the same monitor; Thread.start() happens-before anything in the new thread; a thread's actions happen-before another thread's return from join() on it.

3.4 Explicit Locks: ReentrantLock, ReadWriteLock, StampedLock

```
private final ReentrantLock lock = new ReentrantLock();
public void update() {
    lock.lock();
    try {
        // critical section
    } finally {
        lock.unlock();   // MUST be in finally — a leaked lock deadlocks everyone else forever
    }
}
```

synchronized	ReentrantLock
JVM-managed, auto-released on exception/return	Manually released — must unlock() in finally
No tryLock() — blocks indefinitely	tryLock(timeout) — can back off instead of blocking forever
No fairness option	new ReentrantLock(true) — fair ordering (FIFO), at a throughput cost
Cannot interrupt a waiting thread	lockInterruptibly() allows interruption while waiting
Single condition (wait/notify on the object)	Multiple Condition objects per lock (lock.newCondition())

- ReadWriteLock (ReentrantReadWriteLock): multiple readers can hold the read lock simultaneously; a writer needs exclusive access — good when reads vastly outnumber writes (e.g., a config cache).
- StampedLock (Java 8+): adds optimistic reading — read without locking at all, then validate the stamp afterward; falls back to a real read lock only if data changed mid-read. Higher throughput, more complex to use correctly, and NOT reentrant.

3.5 Atomic Classes & CAS

java.util.concurrent.atomic classes (AtomicInteger, AtomicLong, AtomicReference) provide lock-free thread safety using Compare-And-Swap (CAS) — a hardware-level atomic instruction.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();     // atomic, no lock needed
counter.compareAndSet(5, 10);  // "if current value is 5, set it to 10" — atomically

// CAS pseudocode inside the JVM/hardware:
// do { current = get(); } while (!compareAndSwap(memoryLocation, current, current + 1));
// If another thread changed the value between get() and swap, retry the loop.
```

Why CAS beats locks under low contention

Confidential C

No thread ever blocks or context-switches — it just retries in a tight loop ("optimistic concurrency"). Under HIGH contention, though, constant retries can burn more CPU than a lock would — this trade-off is worth mentioning if asked "when would you NOT use Atomic classes".

Confidential C

3.6 Executor Framework & Thread Pools

Manually creating a new Thread per task doesn't scale — thread creation is expensive and unbounded threads exhaust memory/OS resources. ExecutorService manages a pool of reusable worker threads.

```
ExecutorService fixed = Executors.newFixedThreadPool(4);   // fixed worker count
ExecutorService cached = Executors.newCachedThreadPool();  // grows unbounded, reuses idle
threads
ExecutorService single = Executors.newSingleThreadExecutor(); // sequential, one worker
ScheduledExecutorService sched = Executors.newScheduledThreadPool(2); // delayed/periodic
tasks

fixed.submit(() -> doWork());
fixed.shutdown();          // graceful — finishes queued tasks, rejects new ones
fixed.shutdownNow();       // aggressive — interrupts running tasks
```

Production warning: avoid Executors.* factory methods

newFixedThreadPool and newCachedThreadPool use an UNBOUNDED queue/thread count internally, which has caused real OutOfMemoryError incidents under load spikes. Production code should construct ThreadPoolExecutor directly with an explicit bounded queue and a rejection policy.

ThreadPoolExecutor — the parameters interviewers actually test

```
new ThreadPoolExecutor(
    corePoolSize,      // threads kept alive even when idle
    maximumPoolSize,   // hard ceiling on threads
    keepAliveTime, TimeUnit.SECONDS,  // how long extra (above core) idle threads survive
    new LinkedBlockingQueue<>(queueCapacity),  // where tasks wait if all core threads are
busy
    new ThreadPoolExecutor.CallerRunsPolicy()  // rejection policy once queue AND max threads
are full
);
```

Execution order of a submitted task

1. If fewer than corePoolSize threads exist, start a new thread for the task (even if other core threads are idle).
2. Else, if the queue has room, enqueue the task.
3. Else, if pool size < maximumPoolSize, spin up a new (non-core) thread.
4. Else, the RejectedExecutionHandler kicks in: AbortPolicy (throws), CallerRunsPolicy (runs it on the submitting thread — natural backpressure), DiscardPolicy (silently drops), DiscardOldestPolicy (evicts the oldest queued task).

Future vs CompletableFuture

```
Future<Integer> f = executor.submit(() -> compute());
int result = f.get();   // BLOCKS the calling thread until done — no callback support

CompletableFuture<Integer> cf = CompletableFuture.supplyAsync(() -> compute())
    .thenApply(r -> r * 2)             // transform result, non-blocking
    .thenCompose(r -> anotherAsyncCall(r))  // chain another async operation
    .exceptionally(ex -> { log.error("failed", ex); return -1; });

CompletableFuture.allOf(cf1, cf2, cf3).join();  // wait for all to finish
```

Interview one-liner

"Future gives me a handle to a result I have to block on with get(). CompletableFuture lets me compose a non-blocking pipeline — thenApply/thenCompose/thenCombine — and attach error handling, which is what you actually want for orchestrating parallel microservice calls."

Confidential C

3.7 ConcurrentHashMap Locking — Revisited in Depth

- Java 7 and earlier: segment-based locking — the map was split into ~16 segments, each with its own lock, so up to 16 threads could write concurrently.
- Java 8+: segments are gone. Locking happens per-bin (per bucket): an empty-bin insert uses a lock-free CAS; inserting into a non-empty bin synchronizes only on that bin's first node. Reads (get) are largely lock-free using volatile reads.
- size() in Java 8+ is an approximation maintained via a striped counter (like LongAdder), not a locked exact count — trade a tiny bit of precision for much less write contention.

3.8 Producer-Consumer Pattern

```
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);

// Producer
new Thread(() -> { while (true) queue.put(produceTask()); }).start();  // blocks if queue is
full

// Consumer
new Thread(() -> { while (true) { Task t = queue.take(); process(t); } }).start(); // blocks
if empty
```

- BlockingQueue handles ALL the waiting/notifying internally — no manual wait()/notify() needed. This is the interview-preferred answer over hand-rolled wait/notify unless they explicitly ask you to implement it from scratch.

3.9 Deadlock

Four necessary conditions (Coffman conditions) — all must hold simultaneously for a deadlock: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

```
// Classic deadlock — two threads lock two resources in OPPOSITE order
// Thread 1:                          Thread 2:
synchronized(lockA) {                 synchronized(lockB) {
  synchronized(lockB) { ... }           synchronized(lockA) { ... }   // <- circular wait
}                                     }
```

- Fix: always acquire locks in a GLOBAL, CONSISTENT order across all threads — breaks the circular-wait condition. Alternatively use tryLock(timeout) to back off and retry instead of blocking forever.
- Diagnosing in production: take a thread dump (jstack <pid>) — the JVM explicitly reports "Found one Java-level deadlock" with the exact lock cycle.

3.10 Thread-Safety Patterns

- Immutability: an object with all-final fields and no setters, whose internals are never exposed mutable, is inherently thread-safe — no synchronization needed at all. Prefer this over synchronizing a mutable object whenever possible.
- ThreadLocal<T>: gives each thread its own independent copy of a variable — no sharing, no synchronization. Common use: per-request state (e.g. the current user, a DB connection, a date formatter) in a web app.

```
private static final ThreadLocal<SimpleDateFormat> formatter =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
// each thread gets its own SimpleDateFormat — SimpleDateFormat itself is NOT thread-safe
otherwise
```

Production warning: ThreadLocal + thread pools = memory leak

Confidential C

Threads in a pool are reused, so a ThreadLocal value set during request A can leak into request B if not cleaned up. ALWAYS call threadLocal.remove() in a finally block (or a servlet filter's finally) once the task completes — this is a real, frequently-asked production gotcha.

Thread-safe Singleton (asked very often)

```
class Singleton {
    private static volatile Singleton instance;   // volatile is REQUIRED here
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {                      // 1st check, no locking (fast path)
            synchronized (Singleton.class) {
                if (instance == null) {                // 2nd check, inside the lock
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
// This is Double-Checked Locking. Without 'volatile', a thread could see a
// PARTIALLY CONSTRUCTED object due to instruction reordering during 'new Singleton()'.
```


Confidential C

3.11 Multithreading Interview Q&A Bank

Q: Why is 'volatile int counter; counter++' still not thread-safe? [Very High]

volatile only guarantees visibility and ordering, not atomicity. counter++ is read-modify-write across 3 bytecode steps; two threads can interleave those steps and lose an update even though each sees the freshest value at read time. Use AtomicInteger or synchronization for compound actions.

Q: Explain the difference between synchronized and ReentrantLock, and when you'd reach for the latter. [Very High]

synchronized is simpler and JVM-managed (auto-unlock), but can't be interrupted while waiting, has no timeout, and offers only one implicit condition. I'd reach for ReentrantLock when I need tryLock() with a timeout to avoid indefinite blocking, fairness ordering, or multiple wait-conditions on the same lock (e.g. a bounded buffer with separate 'not full' / 'not empty' conditions).

Q: What's a deadlock, and how do you prevent it? [Very High]

Two or more threads each hold a lock the other needs, and neither can proceed — circular wait. Prevention: always acquire multiple locks in the same global order across the whole codebase, keep lock scope minimal, prefer tryLock with timeout over blocking indefinitely, and avoid calling out to external/unknown code while holding a lock.

Q: Why does the double-checked locking singleton need the volatile keyword? [High]

Without volatile, the JVM/CPU can reorder the instructions inside 'new Singleton()' — allocating memory, running the constructor, assigning the reference — such that another thread can observe a non-null 'instance' reference pointing to a HALF-CONSTRUCTED object. volatile establishes a happens-before edge so the full construction is visible before the reference becomes visible.

Q: What happens if you call run() instead of start() on a Thread? [High]

run() executes synchronously on the CURRENT thread like a normal method call — no new thread is spawned, defeating the entire purpose. start() is what actually asks the JVM/OS to create a new thread of execution.

Q: How would you size a thread pool for a CPU-bound vs an I/O-bound workload? [Very High]

CPU-bound: roughly number of cores (+1), since more threads than cores just adds context-switch overhead with no parallelism gain. I/O-bound: threads spend most time WAITING (network/disk), so you can size much higher — a common formula is threads = cores * (1 + waitTime/computeTime) — to keep the CPU busy while others wait.

Q: Difference between Future and CompletableFuture? [High]

Future.get() blocks the calling thread and has no way to attach a callback or chain further async work. CompletableFuture supports non-blocking composition (thenApply/thenCompose/thenCombine), exception handling (exceptionally/handle), and combining multiple futures (allOf/anyOf) — essential for orchestrating parallel calls without blocking a thread just to wait.

Q: Explain what happens internally with ConcurrentHashMap when two threads write to the SAME bucket at the same time. [High]

If the bucket is currently empty, insertion is attempted via a lock-free CAS on that bin. If the bucket already has a node (collision), the thread synchronizes on that bin's FIRST node before appending/updating — so only threads targeting the SAME bucket ever contend; threads writing to different buckets proceed fully in parallel.

Q: What are the four necessary conditions for a deadlock (Coffman conditions)? [Medium]

Mutual exclusion (resource held exclusively), hold-and-wait (holding one resource while waiting for another), no preemption (a resource can't be forcibly taken away), and circular wait (a cycle of threads each waiting on the next). Breaking any ONE of these prevents deadlock — in practice, breaking circular wait via consistent lock ordering is the most common fix.

Q: Why should you always call unlock() in a finally block with ReentrantLock? [High]

Unlike synchronized, ReentrantLock does NOT auto-release on an exception. If the critical section throws and unlock() is skipped, the lock stays held forever, and every other thread waiting on it deadlocks permanently. lock(); try { ... } finally { unlock(); } is the only safe pattern.

Confidential C

Q: What's a race condition vs a deadlock — are they the same thing? [Medium]

No. A race condition is when the OUTCOME depends on unpredictable thread interleaving (e.g., lost updates) — the program still finishes, just with a wrong result. A deadlock is when threads get permanently STUCK waiting on each other — the program hangs. Race conditions are about correctness; deadlocks are about liveness.

Confidential C

3.12 Cheat Sheet — Multithreading (last-minute revision)

- Thread states: NEW → RUNNABLE ⇄ BLOCKED (waiting on a lock) / WAITING,TIMED_WAITING (wait/join/sleep) → TERMINATED
- start() = new OS thread; run() = normal method call, no new thread
- synchronized: instance method locks 'this'; static method locks the Class object; auto-releases on exception
- volatile = visibility + ordering only, NOT atomicity. counter++ under volatile is still a race condition.
- ReentrantLock: manual unlock() in finally, tryLock(timeout), lockInterruptibly(), fair ordering option, multiple Conditions
- Atomic classes use CAS (compare-and-swap) — lock-free, great under low contention, can thrash under high contention
- ThreadPoolExecutor: core→queue→max→reject (Abort/CallerRuns/Discard/DiscardOldest). Avoid Executors.newFixedThreadPool in prod — unbounded internals.
- Future.get() blocks; CompletableFuture composes non-blocking chains (thenApply/thenCompose/exceptionally)
- ConcurrentHashMap (Java 8+): per-bin CAS/lock, no global lock, size() is an approximation
- Deadlock = Coffman's 4 conditions, all required. Fix = consistent lock ordering, or tryLock with timeout.
- Double-checked-locking singleton NEEDS volatile — otherwise reference can leak before construction finishes.
- ThreadLocal in a pooled-thread environment: always remove() in finally, or you leak state across requests.

Confidential C

PART 4

Java 8+ Features

Before Java 8, Java code was purely imperative — you told the computer exactly HOW to do something, step by step, usually with loops. Java 8 introduced a functional style: you describe WHAT you want, and the JVM figures out how. This part builds that up from first principles — lambdas first, then streams, then the supporting cast.

4.1 Lambda Expressions — Starting From Why

Before Java 8, if you wanted to pass 'a piece of behavior' as an argument (e.g., 'sort this list using THIS comparison logic'), you had no choice but to write a whole anonymous class just to wrap one method:

```
// Before Java 8 — anonymous class just to pass one comparison rule
Collections.sort(list, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return a.length() - b.length();
    }
});
```

That's a lot of ceremony for one line of actual logic. A lambda expression is simply shorthand for implementing a single-method interface — it lets you write just the logic, and the compiler infers everything else:

```
// Same thing, as a lambda
Collections.sort(list, (a, b) -> a.length() - b.length());

// General lambda syntax:
(parameters) -> expression
(parameters) -> { statements; return value; }

() -> System.out.println("no args")
x -> x * 2                          // single param, parens optional
(int x, int y) -> x + y             // explicit types allowed but rarely needed
```

The key idea to internalize

A lambda is NOT magic — it's just a compact way to implement an interface that has exactly ONE abstract method. The compiler looks at the target type (e.g., Comparator<String>) and matches your lambda's parameters/return to that interface's single method. This is why lambdas only work where a 'functional interface' is expected.

4.2 Functional Interfaces

A functional interface is any interface with exactly one abstract method (it can have any number of default/static methods too). The @FunctionalInterface annotation isn't required, but it makes the compiler enforce the rule and documents intent.

java.util.function ships a small set of generic, reusable functional interfaces so you rarely need to declare your own. Learning these four unlocks almost everything else:

Interface	Signature & meaning
Predicate<T>	boolean test(T t) — a yes/no question about T. Used in filter().
Function<T,R>	R apply(T t) — transforms T into R. Used in map().

Confidential C

Interface	Signature & meaning
Consumer<T>	void accept(T t) — DOES something with T, returns nothing. Used in forEach().
Supplier<T>	T get() — takes nothing, PRODUCES a T. Used for lazy value creation, e.g. Optional.orElseGet().

```
Predicate<Integer> isEven   = n -> n % 2 == 0;
Function<String,Integer> len = s -> s.length();
Consumer<String> printer    = s -> System.out.println(s);
Supplier<LocalDate> today   = () -> LocalDate.now();

// there are 2-argument variants too: BiFunction<T,U,R>, BiConsumer<T,U>, BiPredicate<T,U>
// and primitive-specialized ones to avoid autoboxing: IntPredicate, IntFunction<R>,
ToIntFunction<T>
```

Method References — a shorthand for a shorthand

When your lambda body is literally just calling one existing method, you can reference that method by name instead of wrapping it in a lambda:

```
list.forEach(s -> System.out.println(s));   // lambda
list.forEach(System.out::println);          // method reference — identical behavior

// Four kinds:
Type::staticMethod        // Integer::parseInt
object::instanceMethod    // myList::add
Type::instanceMethod      // String::toUpperCase   (the instance becomes the implicit first
arg)
Type::new                 // ArrayList::new         (constructor reference)
```


Confidential C

4.3 Streams — What Problem They Actually Solve

Imagine you need: 'from this list of employees, get the names of everyone in Engineering earning over 80k, sorted alphabetically.' The old way is a pile of nested loops and temporary lists. Streams let you express this as a declarative PIPELINE of transformations:

```
List<String> result = employees.stream()
    .filter(e -> e.getDept().equals("Engineering"))   // keep some
    .filter(e -> e.getSalary() > 80000)
    .map(Employee::getName)                             // transform each
    .sorted()                                            // reorder
    .collect(Collectors.toList());                       // terminate: produce a result
```

A Stream is NOT a data structure — it doesn't store elements. It's a pipeline describing a computation over a source (a Collection, an array, a file, etc). Nothing runs until you call a terminal operation.

Intermediate vs Terminal Operations, and Laziness

```
                SOURCE                    INTERMEDIATE OPS                  TERMINAL OP
  list.stream()  ----->  filter()  ----->  map()  ----->  sorted()  ----->  collect() / forEach()
  (lazy: nothing runs yet)     (still nothing runs — just builds the pipeline)      (NOW it
runs,
                                                                                    element
by element)
```

Intermediate (lazy, returns a Stream)	Terminal (eager, triggers execution)
filter, map, sorted, distinct, limit, skip, peek	collect, forEach, reduce, count, anyMatch, findFirst, toArray

Why laziness matters — the interview-favorite detail

Because nothing runs until a terminal op is called, the stream processes ONE ELEMENT AT A TIME through the WHOLE pipeline, not one stage across all elements. filter().map().sorted() does NOT filter all elements, then map all elements, then sort all — for most ops it goes element-by-element through filter+map, then sorted needs to see everything (it's a 'stateful' op that breaks the per-element flow). This lets short-circuiting ops like findFirst() or anyMatch() stop early without processing the whole source.

Also: a Stream can only be consumed ONCE. Calling a terminal operation twice on the same stream throws IllegalStateException — you have to create a fresh stream.

Collectors — turning a stream back into something useful

```
List<String> names = stream.collect(Collectors.toList());
Set<String> uniq   = stream.collect(Collectors.toSet());
String joined      = stream.collect(Collectors.joining(", ", "[", "]"));  // "[a, b, c]"

// grouping — extremely common in interviews
Map<String, List<Employee>> byDept = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDept));

Map<String, Long> countByDept = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));

double avgSalary = employees.stream()
        .collect(Collectors.averagingDouble(Employee::getSalary));
```

reduce() — the fundamental building block

map/filter/collect are all convenience wrappers; reduce() is the primitive operation underneath: it combines a stream into a single value by repeatedly applying a combining function.

```
int sum = Stream.of(1, 2, 3, 4).reduce(0, (a, b) -> a + b);   // 0 is the identity/seed
```


Confidential C

```
// step by step: ((((0+1)+2)+3)+4) = 10

Optional<Integer> max = Stream.of(3, 7, 2).reduce(Integer::max);   // no seed -> returns
Optional
```

Parallel Streams — powerful, but not free

- stream.parallel() or list.parallelStream() splits the work across the ForkJoinPool.commonPool() (shared JVM-wide, sized to CPU cores).
- Only helps for CPU-bound work on large datasets with an efficiently splittable source (arrays/ArrayList split well; LinkedList doesn't). For small collections, the coordination overhead usually makes it SLOWER than sequential.

Production warning

Never use a parallel stream for a task that does blocking I/O (DB calls, HTTP calls) inside it — it shares the SAME common ForkJoinPool used everywhere else in the JVM (including other unrelated parallel streams / CompletableFuture.supplyAsync by default). One slow blocking task can starve unrelated parallel work across your whole application.

4.4 Optional<T> — Making 'Might Be Absent' Explicit

Before Optional, a method returning null to mean 'nothing found' was a silent trap — callers had no compiler-enforced reminder to check, leading to NullPointerException. Optional<T> is a wrapper that forces the caller to explicitly acknowledge the possibility of absence.

```
Optional<User> findUser(String id) {
    User u = repository.find(id);
    return Optional.ofNullable(u);   // wraps null safely (Optional.of(u) would NPE if u is
null)
}

// Consuming it — the RIGHT ways
String name = findUser("1").map(User::getName).orElse("Unknown");
findUser("1").ifPresent(u -> log.info("found {}", u));
User u = findUser("1").orElseThrow(() -> new UserNotFoundException("1"));
```

Interview trap: Optional.get() and Optional as a field/parameter

Calling .get() without checking isPresent() first defeats the entire purpose — it throws NoSuchElementException, just a different flavor of the same NPE problem Optional was meant to solve. Always prefer map/orElse/orElseThrow/ifPresent.

Optional is designed as a RETURN TYPE for methods where 'no result' is a normal, expected outcome. It's considered bad practice to use it as a method PARAMETER type, a class FIELD, or in a constructor — for those, just use a nullable reference or the Objects.requireNonNull pattern.

4.5 default and static Methods in Interfaces

Before Java 8, adding a new method to an interface would break every existing implementation (they'd suddenly fail to compile). default methods solve this: an interface can now provide a body, and implementing classes inherit it automatically unless they choose to override it. This is exactly how the JDK added forEach() to Collection and stream() to Collection without breaking millions of existing classes.

```
interface Vehicle {
    void drive();
    default void honk() { System.out.println("Beep!"); }   // has a body — safe to add later
    static Vehicle createDefault() { return () -> System.out.println("driving"); }
    // interfaces can have static factory methods too
}
```


Confidential C

What if two interfaces both give a class the same default method?

The implementing class MUST explicitly override it and resolve the conflict (optionally calling one via InterfaceName.super.method()). Java refuses to silently pick one — this is how Java sidesteps the classic multiple-inheritance diamond problem for behavior.

4.6 java.time — The Modern Date/Time API

The old Date and Calendar classes were mutable, not thread-safe, and had a famously confusing API (months indexed from 0!). java.time (Java 8+) is fully immutable and thread-safe by design.

Class	Represents
LocalDate	Date only, no time/zone — 2026-07-29
LocalTime	Time only, no date/zone — 14:30:00
LocalDateTime	Date + time, no zone
ZonedDateTime	Date + time + timezone — use for anything crossing time zones
Duration / Period	Duration = time-based amount (hours/minutes/seconds); Period = date-based amount (years/months/days)

- Every operation (plusDays(), minusHours()) returns a NEW immutable instance rather than mutating — same immutability principle as String, and thread-safe for the same reason.

Confidential C

4.7 Java 8+ Interview Q&A Bank

Q: What exactly is a functional interface, and why do lambdas need one? [Very High]

An interface with exactly one abstract method. A lambda has no name and no declared type of its own — the compiler needs a single-method 'shape' to match the lambda's parameters and body against, which is exactly what a functional interface provides. That's why you can't use a lambda where an interface with 2+ abstract methods is expected.

Q: Why is a Stream not a data structure? [High]

It holds no elements of its own — it's a lazy description of a computation over a source (a List, array, etc). You can't index into it or reuse it; once a terminal operation consumes it, it's done, and you'd need a fresh stream from the source to run the pipeline again.

Q: What's the difference between map() and flatMap()? [Very High]

map() transforms each element 1-to-1 (T -> R). flatMap() is for when each element itself produces a STREAM of results (T -> Stream<R>), and it flattens all those inner streams into one single stream — e.g., turning a List<List<String>> into one flat Stream<String>.

Q: Explain how Optional prevents NullPointerException — does it eliminate NPEs entirely? [High]

It doesn't eliminate NPEs by magic — it makes 'this might be absent' part of the TYPE, so the compiler and code reviewers can see it, and encourages chaining (map/orElse) instead of raw null checks. You can still misuse it by calling .get() blindly, which throws its own exception — the discipline still matters, Optional just makes the right pattern easier to follow.

Q: When would a parallel stream actually make things slower? [Medium]

Small collections (the thread coordination/splitting overhead exceeds the work saved), sources that don't split efficiently (like LinkedList), or when the operation involves blocking I/O — since parallelStream() uses the shared common ForkJoinPool, a blocking task can also starve unrelated parallel work elsewhere in the JVM.

Q: What's the difference between Predicate, Function, Consumer, and Supplier — how do you remember which is which? [High]

Think of it by shape: Predicate takes something, returns a boolean (a question). Function takes something, returns something else (a transformation). Consumer takes something, returns nothing (an action). Supplier takes nothing, returns something (a factory).

Q: Why were default methods added to interfaces in Java 8? [Medium]

To let the JDK (and any interface author) add new methods to an existing interface without breaking every class that already implements it — critical for retrofitting forEach()/stream() onto Collection across the entire ecosystem without a breaking release.

Q: What happens if you call reduce() on an empty stream with no identity value? [Medium]

The single-argument reduce(BinaryOperator) returns an empty Optional, since there's no way to produce a result with zero elements and no seed value. The two-argument reduce(identity, accumulator) instead just returns the identity value directly.

Q: Is a lambda expression an object? What does it actually compile to? [Medium]

Yes — at runtime it's an instance of the target functional interface. It does NOT compile to an anonymous inner class (a common misconception); javac generates an invokedynamic call site, and the actual implementation is generated lazily at runtime via LambdaMetafactory — generally lighter-weight than an anonymous class (no separate .class file per lambda).

Confidential C

4.8 Cheat Sheet — Java 8+ (last-minute revision)

- Lambda = shorthand implementation of a functional interface's single abstract method
- Predicate<T> = T->boolean | Function<T,R> = T->R | Consumer<T> = T->void | Supplier<T> = ()->T
- Method references: Type::static, obj::instance, Type::instance (instance becomes arg0), Type::new
- Stream = lazy pipeline, not a data structure; single-use; nothing runs until a terminal op
- Intermediate (lazy): filter, map, sorted, distinct, limit. Terminal (eager): collect, forEach, reduce, count, anyMatch
- map() = 1-to-1 transform. flatMap() = flattens nested streams into one
- Collectors.groupingBy() for grouping, Collectors.joining() for string concat, Collectors.counting() for tallies
- Optional: prefer map/orElse/orElseThrow/ifPresent over get(); use only as a return type, not a field or parameter
- default methods = interface methods with a body, added in Java 8 to avoid breaking implementers; conflicting defaults must be explicitly resolved
- java.time (LocalDate/LocalDateTime/ZonedDateTime) is immutable and thread-safe — every mutation returns a new instance

Confidential C

PART 5

JVM Internals

You don't need to be a JVM engineer, but you DO need to be able to explain what happens between typing `java MyApp` and your code executing, and what's actually happening when the app 'pauses' or runs out of memory. This part builds the mental model from the ground up.

5.1 JVM Architecture — the Big Picture

The JVM has three major subsystems working together: the Class Loader (gets your .class files INTO memory), the Runtime Data Areas (WHERE things live in memory once loaded), and the Execution Engine (what actually RUNS the bytecode).



5.2 Class Loading — How .class Files Get Into Memory

Class loading happens lazily, on first use — not all at once at startup. It happens in three phases, always in this order: Loading, Linking, Initialization (often abbreviated LLI).

Phase 1: Loading

- Finds the .class file (from the classpath, a JAR, or the network) and reads its bytecode into the Method Area, creating a Class object in the heap that represents the type at runtime (what Foo.class refers to).

Phase 2: Linking (3 sub-steps)

1. Verify — bytecode verifier checks the class file is structurally valid, well-formed, and doesn't violate JVM security constraints (e.g., no illegal type casts, no stack overflows at the bytecode level). This is why you can trust running downloaded/untrusted bytecode to some extent.
2. Prepare — allocates memory for static fields and sets them to their DEFAULT values (0, null, false) — NOT their actual initializer values yet.
3. Resolve — (can happen lazily) symbolic references in the constant pool (e.g., a reference to another class by name) are resolved to actual direct memory references.

Phase 3: Initialization

Confidential C

- Runs static initializer blocks and assigns static fields their ACTUAL values, top to bottom, in source order. This is when static { ... } blocks execute, and it happens exactly ONCE per classloader, triggered by first active use (instantiation, static method call, static field access — but NOT just referencing a static final constant that the compiler inlined).

Interview trap: static block execution order

```
class A { static { System.out.println("A"); } }
class B extends A { static { System.out.println("B"); } }
new B();  // prints "A" then "B" — parent's static initializers ALWAYS run before the child's,
          // and only on the FIRST time the class is actively used, regardless of how many
          // instances you create afterward.
```

Classloader Hierarchy & Delegation

```
Bootstrap ClassLoader
  (loads core JDK classes: java.lang.*, java.util.* — native code)
        |
Platform ClassLoader
  (loads JDK extension modules; was 'Extension ClassLoader' pre-Java 9)
        |
Application ClassLoader
  (loads YOUR classes from the classpath — 'the' classloader)
```

Parent-first delegation model

When asked to load a class, a classloader FIRST delegates to its parent before trying itself. So the Application loader asks Platform, which asks Bootstrap, and only if none of them can find it does the Application loader actually load it itself.

Why this matters: it prevents you from accidentally (or maliciously) shadowing core classes — if you write your own java.lang.String, the Bootstrap loader will always win and load the REAL String first, so your fake one is simply never used for core lookups.

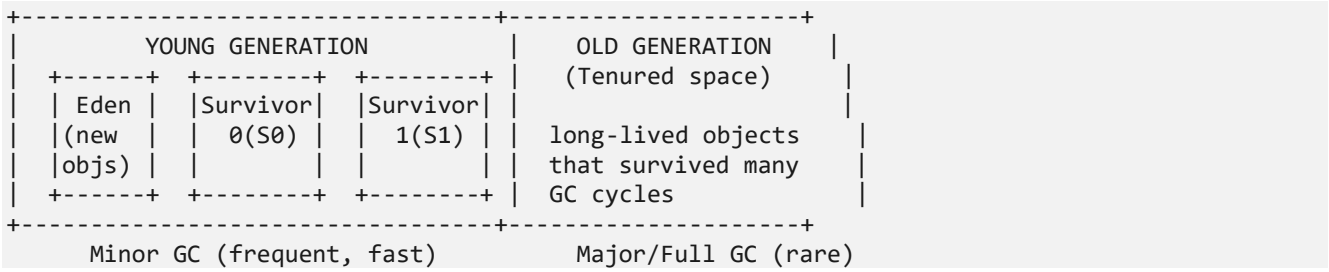
Confidential C

5.3 Runtime Memory Areas

Area	What lives there / key facts
Heap	All objects and arrays. Shared across all threads. Garbage-collected. Divided into generations (below).
Stack	One per thread. Holds stack frames — local variables, method parameters, partial results, and the call chain. Not GC'd — popped automatically on method return.
Method Area / Metaspace	Class metadata, static variables, the runtime constant pool, method bytecode. Called 'Metaspace' since Java 8, and lives in NATIVE (off-heap) memory, not the JVM heap — this is why PermGen's old OutOfMemoryError: PermGen space largely went away after Java 8.
PC Register	One per thread. Tiny — just holds the address of the currently executing bytecode instruction for that thread.
Native Method Stack	Supports native (non-Java, e.g. JNI/C) method calls.

The Heap in Detail — Generational GC

The generational hypothesis observes that most objects die young (a loop variable, a temporary DTO) while a small number live a very long time (caches, singletons). The heap is split accordingly so the GC can clean the 'usually empty, usually garbage' area cheaply and frequently, and only rarely bother with the long-lived area.



Object lifecycle through the heap

1. New objects are allocated in Eden.
2. When Eden fills up, a Minor GC runs: live objects are copied to one Survivor space (S0 or S1); Eden is then wiped entirely. Minor GCs are cheap partly BECAUSE most objects in Eden are already dead by then — there's little to copy.
3. Each object that survives a Minor GC gets its 'age' incremented. Survivors bounce between S0 and S1 on each subsequent Minor GC.
4. Once an object's age crosses a threshold (default 15, tunable), it's PROMOTED to the Old Generation.
5. When the Old Generation fills up, a Major/Full GC runs — this scans the ENTIRE heap (sometimes including a full stop-the-world pause), which is why full GCs are the ones that show up as latency spikes in production.

5.4 Garbage Collection Algorithms

Regardless of algorithm, GC fundamentally works by tracing reachability from a set of GC Roots (local variables on any thread's stack, active static fields, JNI references) — anything NOT reachable from a root is garbage, no matter how many objects reference each other in an isolated island.

Confidential C

Why reference counting alone doesn't work (a common follow-up question)

Two objects that only reference EACH OTHER (a cycle) but are unreachable from any GC Root would never hit a zero reference count, yet they're still garbage. Java's tracing (mark-and-sweep style) approach correctly identifies these as garbage; naive reference counting would leak them forever.

Collector	How it works / when it's used
Serial GC	Single-threaded, stop-the-world for everything. Fine for small heaps / single-core environments, e.g. simple CLI tools.
Parallel GC	Multiple threads do the collection work, but still stop-the-world. Maximizes throughput; was the Java 8 default. Good when pause time doesn't matter but total CPU throughput does.
CMS (Concurrent Mark Sweep)	Deprecated/removed in modern JDKs — did most marking concurrently with app threads to reduce pause times, but suffered from heap fragmentation (no compaction step).
G1 (Garbage First)	Default GC since Java 9. Splits the heap into many small equal-size regions rather than fixed generational blocks, and prioritizes collecting the regions with the MOST garbage first ('garbage first'). Aims for predictable, low pause times you can configure a target for.
ZGC / Shenandoah	Modern low-latency collectors (JDK 11+/12+) — sub-millisecond pause times even on multi-terabyte heaps, using concurrent compaction. Used where latency is critical (trading platforms, real-time systems).

Minor GC vs Major GC vs Full GC — precise definitions

- Minor GC: cleans the Young Generation only. Frequent, fast, usually causes a brief stop-the-world pause but a short one.
- Major GC: cleans the Old Generation.
- Full GC: cleans the ENTIRE heap (Young + Old, plus often Metaspace) — the most expensive, and the one that shows up as a visible latency spike or 'the app froze for 2 seconds' in production incident reports.

5.5 Interpreter vs JIT Compiler — Why Java Isn't 'Slow'

Bytecode isn't native machine code — the JVM's interpreter reads and executes it one instruction at a time, which is flexible (same bytecode runs everywhere) but slower than compiled native code. The JIT (Just-In-Time) compiler fixes this by watching which methods run frequently ('hot' methods) and compiling THOSE specific methods down to native machine code at runtime, then swapping in that fast native version for future calls.

```
First few calls to a method --> Interpreter (slow start, no compile delay)
Method gets called a LOT    --> JIT compiles it to native code in the background
Future calls                --> Jump straight to the fast native version
```

- HotSpot (Oracle/OpenJDK's JVM) uses tiered compilation: C1 (client compiler) kicks in fast with light optimization for a quick warm-up win, and C2 (server compiler) kicks in later with heavier, more aggressive optimizations for methods that stay hot.

Confidential C

- This is exactly why Java benchmarks often show code getting FASTER the longer it runs ("warm-up") — a well-known gotcha when writing microbenchmarks (always warm up before measuring, or use a proper tool like JMH).

JIT optimizations worth name-dropping

Inlining — a small, frequently-called method's body gets pasted directly into the caller, eliminating call overhead.

Escape analysis — if the JIT proves an object never 'escapes' the method (no reference leaves it), it can allocate it on the STACK instead of the heap, or even eliminate the allocation entirely — reducing GC pressure without you doing anything.

Confidential C

5.6 JVM Interview Q&A Bank

Q: Walk me through what happens from typing `java MyApp` to your main() method actually running. [Very High]

The JVM starts, the Bootstrap/Platform/Application classloaders load MyApp.class (delegating up the parent chain first), the class goes through Linking (verify bytecode, prepare static fields with defaults, resolve references), then Initialization runs static blocks and assigns real static values, and finally the JVM invokes the public static void main(String[]) method, starting execution via the interpreter (with JIT kicking in later for hot code).

Q: Why did PermGen get replaced with Metaspace in Java 8? [High]

PermGen had a FIXED max size set at JVM startup (-XX:MaxPermSize), and applications with lots of dynamically generated classes (common with certain frameworks doing heavy reflection/proxying) would frequently hit OutOfMemoryError: PermGen space. Metaspace lives in native memory and grows dynamically (bounded only by available system memory or an optional -XX:MaxMetaspaceSize), which eliminated that whole class of production incidents.

Q: What are GC Roots, and why does the JVM trace from them instead of just counting references? [High]

GC Roots are the fixed starting points guaranteed to be reachable — local variables on thread stacks, active static fields, JNI references. Tracing from roots correctly identifies ISLANDS of objects that only reference each other but aren't reachable from any root as garbage — something naive reference counting would miss entirely, since those objects' reference counts never hit zero.

Q: Explain the difference between a Minor GC and a Full GC, and why Full GCs matter for production performance. [Very High]

Minor GC cleans only the Young Generation, is fast, and happens often. Full GC scans the entire heap (Young + Old + often Metaspace) and is far more expensive — it's usually the direct cause of visible latency spikes/'freeze' complaints in production, which is why tuning heap sizing and generation ratios to minimize promotion into Old Gen (and thus Full GCs) is a real production skill.

Q: Why does the JVM use a generational heap instead of treating all objects the same? [Medium]

Empirically, most objects die young (the 'weak generational hypothesis') — loop variables, temporary objects, request-scoped DTOs. By segregating a small, frequently-collected Young Gen from a rarely-collected Old Gen, the GC can do cheap, fast collections most of the time and only pay the expensive full-heap-scan cost occasionally.

Q: What is JIT compilation, and why doesn't the JVM just compile everything to native code upfront? [Medium]

JIT compiles only 'hot' (frequently executed) methods to native code at runtime, based on actual observed execution — this lets it apply optimizations (like inlining based on ACTUAL call patterns, not guesses) that a purely ahead-of-time compiler couldn't know about. Compiling everything upfront would also slow startup significantly for code paths that barely ever run.

Q: What's the parent-first delegation model in class loading, and what problem does it solve? [Medium]

A classloader always asks its parent to try loading a class first before attempting it itself (Application -> Platform -> Bootstrap). This guarantees core JDK classes are always loaded by the trusted Bootstrap loader first, preventing a user-defined class from shadowing/spoofing a core class like java.lang.String.

Q: If two objects only reference each other but nothing else in the app references either of them, are they garbage collected? [High]

Yes. Java's GC uses reachability from GC Roots, not reference counting — since neither object is reachable from any root, both are correctly identified as garbage and collected together, despite each having a non-zero reference count from the other.

Confidential C

5.7 Cheat Sheet — JVM Internals (last-minute revision)

- Class Loader -> Runtime Data Areas -> Execution Engine (Interpreter + JIT + GC)
- Class loading = Loading -> Linking (Verify, Prepare, Resolve) -> Initialization (LLI), lazy on first active use
- Classloader hierarchy: Bootstrap -> Platform -> Application, parent-first delegation
- Heap = objects, GC'd, shared. Stack = per-thread frames, auto-cleaned. Metaspace = class metadata, off-heap since Java 8. PC register = per-thread instruction pointer.
- Young Gen (Eden + S0/S1) -> Minor GC, frequent/cheap. Old Gen -> Major GC. Full GC = entire heap, expensive, causes latency spikes.
- Objects promoted from Young to Old after surviving enough Minor GCs (age threshold, default 15)
- GC traces reachability from GC Roots (stack locals, static fields, JNI refs) — NOT reference counting, so reference cycles are still collected correctly
- G1 = default GC since Java 9, region-based, collects garbage-heavy regions first. ZGC/Shenandoah = sub-ms pause, huge heaps.
- Interpreter runs bytecode directly (slow start, no compile delay). JIT (C1 fast/light, C2 slow/aggressive) compiles hot methods to native code at runtime — inlining + escape analysis are its key tricks.

Confidential C

PART 6

Spring Framework

Spring's entire reason for existing is to solve ONE problem well: objects in a real application depend on other objects, and if each object is responsible for CREATING its own dependencies, your codebase becomes rigid, hard to test, and hard to change. Everything else in Spring is built on top of the fix for that one problem.

6.1 Inversion of Control & Dependency Injection — From First Principles

Without Spring, if an OrderService needs a PaymentGateway, the naive approach is for OrderService to construct it itself:

```
class OrderService {
    private PaymentGateway gateway = new StripeGateway();  // OrderService CONTROLS its own
dependency
}
```

This is rigid: OrderService is now permanently welded to StripeGateway. Testing OrderService means you're ALSO testing StripeGateway (or fighting to avoid it). Swapping payment providers means editing OrderService's source code.

Inversion of Control means flipping who's in charge: instead of OrderService creating its own dependency, something EXTERNAL creates the dependency and hands ('injects') it to OrderService. OrderService just declares what it needs:

```
class OrderService {
    private final PaymentGateway gateway;
    OrderService(PaymentGateway gateway) { this.gateway = gateway; }  // dependency is
INJECTED
}

// Spring creates the object graph for you, based on configuration/annotations:
@Service
class OrderService {
    private final PaymentGateway gateway;
    @Autowired  // optional on a single constructor since Spring 4.3+
    OrderService(PaymentGateway gateway) { this.gateway = gateway; }
}
```

The one-sentence definition interviewers are listening for

"IoC is the principle — control over object creation is inverted, moved out of the class itself. DI is the PATTERN that implements IoC — dependencies are supplied ('injected') from outside rather than constructed internally. Spring's IoC Container is the concrete tool that reads your configuration/annotations and wires the whole object graph together."

The 3 ways to inject a dependency

Type	How, and when to use it
Constructor injection	Dependencies passed via constructor. PREFERRED — makes dependencies explicit and immutable (final fields), and makes it impossible to construct the object in a broken/half-wired state.
Setter injection	Dependencies set via a setter method. Use for OPTIONAL dependencies that have a sensible default.

Confidential C

Type	How, and when to use it
Field injection (@Autowired on the field)	Shortest to write but WORST practice — hides dependencies (no visible constructor signature), makes the class impossible to instantiate without Spring/reflection, and can't be made final. Common in tutorials, avoided in production code.

6.2 The IoC Container: BeanFactory vs ApplicationContext

The IoC Container is the engine that actually does the wiring — it reads your bean definitions (from XML, Java @Configuration classes, or annotations), instantiates the objects, injects dependencies, and manages their lifecycle.

BeanFactory	ApplicationContext
The base/root interface — basic DI container	Extends BeanFactory — adds event publishing, internationalization, AOP integration, and more
Lazy: beans created only when first requested	Eager by default for singletons — created at startup, so config errors surface immediately instead of at runtime
Rarely used directly today	This is what you actually get in every real Spring app (e.g. AnnotationConfigApplicationContext, or the one Spring Boot wires up for you)

6.3 Declaring Beans: Component Scanning & @Configuration

Annotation	Meaning
@Component	Generic — 'let Spring manage this class as a bean'
@Service	Semantically a service-layer bean — functionally identical to @Component, purely for readability/intent
@Repository	Data-access-layer bean — ALSO enables automatic translation of platform-specific DB exceptions into Spring's unified DataAccessException hierarchy
@Controller / @RestController	Web-layer bean — handles HTTP requests; @RestController = @Controller + @ResponseBody on every method (returns data, not a view name)

```
@Configuration           // this class itself provides bean definitions
public class AppConfig {
    @Bean                // explicit bean definition — useful for 3rd-party classes you can't
annotate
    public RestTemplate restTemplate() { return new RestTemplate(); }
}
```

@Component vs @Bean — a very common point of confusion

@Component goes ON a class you wrote, and component-scanning discovers it automatically.

@Bean goes on a METHOD inside an @Configuration class, and you manually construct/return the object — the only option for classes from a library you can't annotate (e.g. RestTemplate, ObjectMapper).

Confidential C

6.4 Bean Lifecycle

This is one of the most reliably-asked Spring questions at the 3-year-experience level — interviewers want to see you understand there's a defined SEQUENCE, not just 'Spring creates the bean'.

```
1. Instantiate the bean (constructor called, dependencies resolved)
2. Populate properties (setter injection, @Autowired fields applied)
3. Aware interfaces called, if implemented:
     BeanNameAware -> setBeanName()
     BeanFactoryAware -> setBeanFactory()
     ApplicationContextAware -> setApplicationContext()
4. BeanPostProcessor.postProcessBeforeInitialization()
     (runs for EVERY bean, e.g. proxying setup for @Transactional)
5. @PostConstruct method runs
6. InitializingBean.afterPropertiesSet() runs (if implemented)
7. Custom init-method runs (if configured via @Bean(initMethod=...))
8. BeanPostProcessor.postProcessAfterInitialization()
     (this is WHERE AOP proxies actually get created)
9. --- bean is now fully ready and in use ---
10. @PreDestroy method runs (on container shutdown)
11. DisposableBean.destroy() runs (if implemented)
12. Custom destroy-method runs (if configured)
```

Why this order matters in practice

@PostConstruct is the modern, preferred hook for 'run this once all my @Autowired dependencies are set' — e.g., validating config, warming a cache. It's guaranteed to run AFTER dependency injection is complete but BEFORE the bean is handed out to anyone else, which is exactly the guarantee you need.

AOP proxies (for @Transactional, @Cacheable, etc.) are created in postProcessAfterInitialization — meaning the object you originally instantiated isn't necessarily the object other beans end up holding a reference to; they may hold a PROXY wrapping it. This is the root cause of the classic 'self-invocation doesn't trigger @Transactional' bug covered in the AOP section below.

6.5 Bean Scopes

Scope	Meaning
singleton (default)	ONE shared instance per Spring container — this is why beans should generally be stateless; mutable instance fields on a singleton bean are a common concurrency bug
prototype	A NEW instance every time the bean is requested/injected
request	One instance per HTTP request (web apps only)
session	One instance per HTTP session (web apps only)

Interview trap: injecting a prototype bean into a singleton

If Singleton A holds an @Autowired reference to Prototype B, Spring resolves B only ONCE — at the time A is created — and A holds that SAME instance forever. This defeats the purpose of 'a new instance every time'. Fix: inject an ObjectFactory<B>/Provider<B> or use a scoped proxy (proxyMode = ScopedProxyMode.TARGET_CLASS) so a fresh B is fetched on each actual use.

6.6 Circular Dependencies

Confidential C

If bean A needs bean B in its constructor, and bean B needs bean A in its constructor, neither can be fully constructed first — a genuine chicken-and-egg problem.

- With FIELD or SETTER injection, Spring can resolve this: it creates A as a raw, partially-initialized object, registers it early in a cache, creates B (injecting the partial A), then finishes populating A's fields with B. This 'early reference' trick is only possible because setter/field injection doesn't require ALL dependencies at construction time.
- With CONSTRUCTOR injection (the recommended style!), Spring CANNOT resolve this — it throws BeanCurrentlyInCreationException at startup. This is actually considered a FEATURE, not a limitation: a circular dependency between two constructor-injected beans is a real design smell, and constructor injection surfaces it immediately instead of hiding it.

The real fix, not a workaround

A genuine circular dependency almost always means A and B are too tightly coupled and share a responsibility that should be extracted into a third class C, which both A and B depend on instead of on each other. Reaching for @Lazy to paper over it should be a last resort, not the default answer.

Confidential C

6.7 Aspect-Oriented Programming (AOP)

Some concerns cut across many unrelated classes — logging, transaction management, security checks, caching. Sprinkling the same boilerplate into every method that needs it violates separation of concerns. AOP lets you define that behavior ONCE and declare WHERE it should apply, and Spring weaves it in without you touching the original method bodies.

```
@Aspect
@Component
class LoggingAspect {
    @Before("execution(* com.app.service.*.*(..))")     // pointcut: which methods
    public void logBefore(JoinPoint jp) {
        log.info("Calling {}", jp.getSignature());
    }

    @Around("@annotation(Timed)")                         // pointcut: methods annotated @Timed
    public Object measure(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try {
            return pjp.proceed();               // actually invoke the real method
        } finally {
            log.info("{} took {}ns", pjp.getSignature(), System.nanoTime() - start);
        }
    }
}
```

Advice type	When it runs
@Before	Before the target method executes
@After	After the method completes, regardless of outcome (success or exception) — like 'finally'
@AfterReturning	Only after a SUCCESSFUL return, has access to the return value
@AfterThrowing	Only if the method throws an exception
@Around	Wraps the ENTIRE call — must explicitly call proceed(); most powerful, can change arguments/return value or skip the call entirely

How AOP actually works: proxies

Spring AOP doesn't modify your bytecode ahead of time — it works by creating a PROXY object at runtime that wraps your real bean. Every call routes through the proxy first, which runs the relevant advice, then delegates to the real method.

JDK Dynamic Proxy	CGLIB Proxy
Used when the target class implements at least one INTERFACE	Used when the target class has NO interface — CGLIB generates a runtime SUBCLASS of your concrete class instead
Proxy implements the same interface(s)	Proxy extends your actual class — which is why CGLIB can't proxy final classes/methods

The #1 AOP gotcha: self-invocation bypasses the proxy

```
class OrderService {
   public void placeOrder() { this.sendConfirmation(); }   // internal call — BYPASSES the proxy entirely!
```


Confidential C

JDK Dynamic Proxy	CGLIB Proxy

```
    @Transactional
    public void sendConfirmation() { ... }
}
```

When placeOrder() calls this.sendConfirmation() directly, it's calling the method on the RAW object (`this`), not through the Spring-managed proxy that would have started the transaction. @Transactional (and @Cacheable, @Async — anything proxy-based) is SILENTLY skipped. Fix: move the annotated method to a separate bean and call it through that bean's injected reference, so the call actually passes through the proxy.

6.8 Spring MVC Request Flow: DispatcherServlet

Every incoming HTTP request to a Spring web app funnels through a single Front Controller — the DispatcherServlet — which coordinates the rest of the framework rather than each controller handling routing itself.

```
HTTP Request
     |
     v
DispatcherServlet  (the front controller — single entry point)
     |
     v
HandlerMapping        -- figures out WHICH controller method should handle this URL
     |
     v
HandlerAdapter        -- actually invokes that controller method
     |
     v
Controller method runs -- (business logic, returns data or a view name)
     |
     v
[ @RestController: response body serialized directly (e.g. to JSON via Jackson) ]
[ @Controller: ViewResolver maps the returned name to an actual View/template ]
     |
     v
HTTP Response
```


Confidential C

6.9 Spring Framework Interview Q&A Bank

Q: What's the difference between IoC and DI, in your own words? [Very High]

IoC is the general principle of inverting control of object creation away from the class itself. DI is the specific pattern that implements it — dependencies are supplied from outside (via constructor, setter, or field) rather than the class constructing them internally. Spring's container is the tool that actually performs the DI based on IoC as its guiding principle.

Q: Why is constructor injection generally preferred over field injection? [Very High]

Constructor injection makes dependencies explicit in the signature, allows fields to be final (immutable, thread-safe by construction), guarantees the object can never exist in a partially-wired state, and makes the class trivially testable with plain 'new' + mocks — no Spring container or reflection needed in unit tests. Field injection hides all of that and requires reflection even to construct the object for a test.

Q: Walk me through the Spring bean lifecycle. [Very High]

Instantiate -> populate dependencies -> Aware callbacks (BeanNameAware etc.) -> BeanPostProcessor.postProcessBeforeInitialization -> @PostConstruct -> InitializingBean.afterPropertiesSet -> custom init-method -> BeanPostProcessor.postProcessAfterInitialization (this is where AOP proxies get created) -> bean is ready -> on shutdown: @PreDestroy -> DisposableBean.destroy -> custom destroy-method.

Q: Why can constructor-injected circular dependencies NOT be resolved, while setter-injected ones can? [High]

Constructor injection requires ALL dependencies to exist before the object itself can be constructed — there's no way to hand bean A a not-yet-finished bean B before B needs a not-yet-finished A, so it's a genuine deadlock and Spring throws BeanCurrentlyInCreationException. Setter/field injection can construct a bean with NO dependencies yet, register that partial instance early, and fill in dependencies afterward — breaking the cycle at the cost of briefly exposing a semi-constructed object.

Q: Explain how Spring AOP actually implements cross-cutting behavior like @Transactional. [Very High]

Spring creates a runtime PROXY around your bean — a JDK dynamic proxy if your class implements an interface, or a CGLIB subclass proxy if it doesn't. Every external call goes through the proxy first, which runs the relevant advice (e.g., starts a transaction) before delegating to your actual method, then commits/rolls back afterward based on the outcome.

Q: Why does calling an @Transactional method from another method IN THE SAME CLASS not trigger the transaction? [Very High]

Because that call happens via 'this', which is the raw, unproxied object — it never passes through the Spring-managed proxy that would start the transaction. Any proxy-based annotation (@Transactional, @Cacheable, @Async) is silently skipped on self-invocation. The fix is to move the annotated method into a separate bean and call it via an injected reference to THAT bean, forcing the call through the proxy.

Q: What's the difference between @Component, @Service, and @Repository if they're functionally almost identical? [Medium]

All three make Spring register the class as a bean via component scanning. @Service and @Component are functionally identical — the distinction is purely semantic/documentation. @Repository additionally enables Spring's exception translation, converting database-specific exceptions (e.g. a JDBC SQLException) into Spring's unified, unchecked DataAccessException hierarchy, so calling code doesn't need to depend on a specific persistence technology's exception types.

Q: What problem does bean scope 'prototype' solve, and what's the classic mistake when using it? [Medium]

It gives you a fresh instance on every request instead of one shared singleton — useful for stateful, non-thread-safe objects. The classic mistake is injecting a prototype bean directly into a singleton's field: Spring resolves it only ONCE at the singleton's creation time, so you end up with what's effectively a permanent single instance anyway — defeating the purpose. Fix with an ObjectFactory/Provider or a scoped proxy so a new instance is fetched on each actual use.

Q: What's the difference between @Around advice and the other advice types? [Medium]

Confidential C

@Around is the only advice type that wraps the ENTIRE invocation and has full control — it must explicitly call proceed() to let the real method run, and can inspect/modify arguments, change the return value, suppress the call entirely, or catch exceptions before they propagate. @Before/@After/@AfterReturning/@AfterThrowing are all more limited, fixed hook points around a call that always happens.

Confidential C

6.10 Cheat Sheet — Spring Framework (last-minute revision)

- IoC = principle (control inverted). DI = pattern (dependencies supplied externally). Container = the tool that does it.
- Constructor injection preferred: explicit, immutable (final), testable without Spring, fails fast on missing deps.
- BeanFactory = lazy, base interface. ApplicationContext = eager singletons by default, adds events/i18n/AOP — what you actually use.
- @Component/@Service/@Repository/@Controller = auto-detected via scanning. @Bean = manual, inside @Configuration, for classes you can't annotate.
- Bean lifecycle: instantiate -> populate -> Aware callbacks -> BPP before -> @PostConstruct -> afterPropertiesSet -> init-method -> BPP after (AOP proxy created here) -> ready -> @PreDestroy -> destroy()
- Scopes: singleton (default, one per container) | prototype (new each time) | request | session
- Constructor-injected circular deps = startup failure (feature, surfaces design smell). Setter/field-injected = resolved via early references.
- AOP = proxy-based. JDK dynamic proxy (interface exists) vs CGLIB (subclass, no interface needed).
- Self-invocation (this.method()) bypasses the proxy — @Transactional/@Cacheable/@Async silently no-op.
- DispatcherServlet (front controller) -> HandlerMapping -> HandlerAdapter -> Controller -> ViewResolver (or direct JSON for @RestController)

Confidential C

PART 7

Spring Boot

Plain Spring gives you a powerful IoC container, but you still had to hand-configure a DataSource, a JSP/Thymeleaf ViewResolver, a Tomcat deployment descriptor, and dozens of beans just to get 'Hello World' running. Spring Boot's entire value proposition is: sensible defaults for everything, configurable when you need to override them, and zero XML/boilerplate to start.

7.1 What Spring Boot Actually Adds Over Plain Spring

Plain Spring	Spring Boot
You manually declare every bean (DataSource, ObjectMapper, ViewResolver...)	Auto-configuration inspects your classpath and wires sensible beans automatically
You manually assemble compatible dependency versions	Starters bundle a tested, compatible set of dependencies under one artifact
You deploy a WAR to an external Tomcat/JBoss server	An embedded server (Tomcat/Jetty/Undertow) is packaged INSIDE your jar — java -jar app.jar just runs
Configuration typically via XML or scattered @Configuration classes	Centralized application.properties/yml + type-safe @ConfigurationProperties
No built-in production monitoring	Actuator gives you health/metrics/env endpoints out of the box

7.2 @SpringBootApplication — What It Actually Is

This single annotation is a shorthand that bundles THREE separate annotations — understanding what each one does individually is what separates 'I copy-pasted this' from actually understanding your own app's entry point.

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}

// is EXACTLY equivalent to:
@Configuration           // this class can itself declare @Bean methods
@EnableAutoConfiguration // turn on Boot's classpath-scanning auto-configuration machinery
@ComponentScan           // scan this package (and sub-packages) for @Component/@Service/etc.
public class MyApp { ... }
```

Why package placement matters

@ComponentScan defaults to scanning the package of the annotated class AND everything below it. This is why the convention is to put your main application class in the ROOT package (e.g. com.company.app) — if it were nested inside com.company.app.controllers, sibling packages like com.company.app.services would never get scanned.

7.3 Auto-Configuration Internals

Confidential C

This is the actual 'magic' — and it isn't magic at all, it's just conditional bean registration. When your app starts, Boot loads a long list of candidate auto-configuration classes and lets each one decide FOR ITSELF, using conditional annotations, whether it should actually contribute any beans.

```
@Configuration
@ConditionalOnClass(DataSource.class)  // only if this class is on the classpath
public class DataSourceAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean  // only if user hasn't defined their own DataSource
    public DataSource dataSource(DataSourceProperties props) {
        return props.initializeDataSourceBuilder().build();
    }
}
```

Conditional annotation	Meaning
@ConditionalOnClass	Activate only if a given class is present on the classpath (e.g., only configure a DataSource if a JDBC driver is present)
@ConditionalOnMissingBean	Activate only if the user HASN'T already defined their own bean of that type — this is exactly how you override any auto-configured bean: just define your own
@ConditionalOnProperty	Activate only if a specific application.properties key is set (and optionally matches a specific value)
@ConditionalOnWebApplication	Activate only in a web (servlet/reactive) context, not a plain CLI app

How Boot finds these classes: AutoConfiguration.imports

Each auto-configuration jar on your classpath (spring-boot-autoconfigure, or a 3rd-party starter) contains a file at META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports listing its candidate configuration classes (this replaced the older spring.factories mechanism in Spring Boot 2.7+). Boot reads ALL of these across every jar, then evaluates each one's conditions.

The one-sentence answer interviewers want

"Auto-configuration isn't magic — it's a big list of @Configuration classes, each guarded by @Conditional annotations, that Boot evaluates at startup. If the relevant class is on the classpath AND you haven't already defined a competing bean yourself, Boot's default gets registered; otherwise it silently backs off."

7.4 Starters

A 'starter' is just a Maven/Gradle dependency with NO code of its own — it exists purely to pull in a curated, version-compatible bundle of other dependencies.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- pulls in: spring-webmvc, an embedded Tomcat, Jackson (JSON), validation, and spring-boot-starter -->
```

- spring-boot-starter-web: MVC + embedded Tomcat + Jackson
- spring-boot-starter-data-jpa: Spring Data JPA + Hibernate + a connection pool (HikariCP)
- spring-boot-starter-security: Spring Security auto-configured with a default login form/basic auth until you customize it

Confidential C

- spring-boot-starter-test: JUnit, Mockito, AssertJ, Spring Test — all in one

7.5 Embedded Servers

Traditional Java web apps were packaged as a WAR file and deployed INTO an already-running external server (Tomcat, JBoss). Spring Boot flips this: the server itself is a dependency bundled INSIDE your executable jar, started programmatically as part of your own main() method.

```
// This ordinary-looking line is what actually starts the embedded Tomcat, builds the whole
// Spring ApplicationContext, and wires everything together:
SpringApplication.run(MyApp.class, args);
```

Why this mattered for the industry (worth mentioning in an interview)

This 'fat jar' model is exactly what made Java applications fit naturally into Docker containers and cloud-native deployment (Kubernetes, etc.) — you ship one self-contained artifact and run it the same way everywhere, instead of managing a separate application-server installation per environment.

7.6 Configuration: application.properties/yml, @Value, @ConfigurationProperties

```
# application.yml
server:
  port: 8081
app:
  payment:
    api-key: ${PAYMENT_API_KEY}     # pulled from an environment variable
    timeout-ms: 3000
```

@Value	@ConfigurationProperties
Injects ONE property at a time into a field: @Value("${app.payment.timeout-ms}")	Binds an ENTIRE prefixed group of properties into a strongly-typed POJO in one shot
No compile-time validation of the key name — a typo just silently fails/defaults	Fails fast and clearly if types don't match; supports IDE autocomplete via metadata
Fine for one-off simple values	Preferred for anything with more than 1-2 related properties

```
@ConfigurationProperties(prefix = "app.payment")
@Component
public class PaymentProperties {
    private String apiKey;
    private int timeoutMs;
    // getters and setters
}
```

7.7 Profiles

Profiles let you swap configuration/beans per environment (dev, test, prod) without changing code — you activate one at startup and Spring only applies the matching configuration.

```
@Configuration
@Profile("prod")
public class ProdConfig {
    @Bean public DataSource dataSource() { return realProductionDb(); }
}

// application-dev.yml / application-prod.yml – profile-specific override files
// activate via: -Dspring.profiles.active=prod   OR   SPRING_PROFILES_ACTIVE=prod env var
```


Confidential C

7.8 Actuator

Actuator exposes production-readiness endpoints out of the box — critical for anything running behind a load balancer or in Kubernetes, which need to know if your app is actually healthy.

Endpoint	Purpose
/actuator/health	Overall app health — Kubernetes liveness/readiness probes hit this
/actuator/metrics	JVM memory, GC pause times, HTTP request counts/latencies, and more
/actuator/info	Arbitrary build/version metadata you configure
/actuator/env	All active configuration properties (sensitive values are masked by default)

Production example: a custom health check

```
@Component
public class PaymentGatewayHealthIndicator implements HealthIndicator {
   public Health health() {
      return isGatewayReachable() ? Health.up().build()
                        : Health.down().withDetail("reason", "timeout").build();
   }
}
```

Now /actuator/health automatically includes this check alongside the built-in DB/disk-space ones — if the payment gateway is down, your overall health check reports DOWN too, so Kubernetes stops routing traffic to this instance.

7.9 Centralized Exception Handling: @ControllerAdvice

Instead of a try-catch in every single controller method, @ControllerAdvice lets you define exception-handling logic ONCE, globally, for every controller in the app.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ErrorResponse(ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)   // thrown by @Valid failures
    public ResponseEntity<?> handleValidation(MethodArgumentNotValidException ex) {
        return ResponseEntity.badRequest().body(ex.getBindingResult().getFieldErrors());
    }
}
```

7.10 Bean Validation: @Valid and @Validated

```
class CreateUserRequest {
    @NotBlank private String name;
    @Email    private String email;
    @Min(18)  private int age;
}
```


Confidential C

```
@PostMapping("/users")
public User create(@Valid @RequestBody CreateUserRequest req) { ... }
// A failing constraint throws MethodArgumentNotValidException BEFORE the method body even
runs —
// caught by the @ControllerAdvice handler above.
```

- @Valid triggers standard JSR-380 (Bean Validation) rules. @Validated (Spring's own annotation) additionally supports validation GROUPS and is required to enable method-level validation on @Service classes.

7.11 What SpringApplication.run() Actually Does

1. Determine the application type (Servlet web, Reactive web, or plain non-web) based on what's on the classpath.
2. Load and prepare the Environment (application.properties/yml, env vars, command-line args, active profiles — all merged with a defined precedence order).
3. Create the appropriate ApplicationContext (e.g. AnnotationConfigServletWebServerApplicationContext for a web app).
4. Run all registered ApplicationContextInitializers.
5. 'Refresh' the context — this is where the ACTUAL bean lifecycle from Part 6 happens: bean definitions are read, auto-configuration classes are evaluated, beans are instantiated/wired, and (for a web app) the embedded server actually starts listening.
6. Run any CommandLineRunner / ApplicationRunner beans — a common hook for 'do this once at startup' logic (seed data, warm a cache, verify external connectivity).

CommandLineRunner vs ApplicationRunner

Functionally almost identical 'run this after startup' hooks. CommandLineRunner.run(String... args) gets raw command-line args; ApplicationRunner.run(ApplicationArguments args) gets a parsed, structured wrapper (easy access to named --options). Multiple runners execute in the order defined by @Order.

7.12 Testing Slices

Loading the ENTIRE application context for every test is slow. Boot provides 'test slice' annotations that spin up only the beans relevant to one layer.

Annotation	What it loads
@SpringBootTest	The FULL application context — use for true end-to-end/integration tests
@WebMvcTest(MyController.class)	Only the web layer (controllers, filters, JSON serialization) — services/repos are mocked via @MockBean
@DataJpaTest	Only JPA repositories + an in-memory test database — fast repository-layer tests

Confidential C

7.13 Spring Boot Interview Q&A Bank

Q: What does @SpringBootApplication actually consist of, and why does it matter where you place the main class? [Very High]

It's @Configuration + @EnableAutoConfiguration + @ComponentScan combined. @ComponentScan defaults to scanning the annotated class's OWN package and everything beneath it, so the main class needs to sit at the root of your package structure — otherwise sibling packages never get scanned and their beans silently never get registered.

Q: Explain how Spring Boot auto-configuration actually decides which beans to create. [Very High]

Boot loads a list of candidate auto-configuration classes (declared in each jar's META-INF/spring/...AutoConfiguration.imports file), then evaluates each against its @Conditional annotations — most commonly @ConditionalOnClass (is the relevant library on the classpath) and @ConditionalOnMissingBean (has the user already defined their own bean of this type). Only configurations whose conditions all pass actually contribute beans.

Q: If Boot auto-configures a DataSource for you, how do you override it with your own custom configuration? [High]

Just define your own @Bean of type DataSource. Almost every Boot auto-configuration is guarded with @ConditionalOnMissingBean, so as soon as YOUR bean of that type exists, Boot's default backs off automatically — no need to explicitly disable or exclude anything in most cases.

Q: What's the actual difference between a 'starter' and a normal dependency? [Medium]

A starter has no functional code of its own — it's a pure aggregator POM whose sole purpose is to pull in a tested, mutually-compatible set of actual dependencies (e.g. spring-boot-starter-web pulls in spring-webmvc, an embedded Tomcat, and Jackson) so you don't have to hunt down compatible versions of each yourself.

Q: Why does Spring Boot embed the server instead of deploying a WAR to an external one? [Medium]

It makes the application a single, self-contained, runnable artifact (java -jar app.jar) with no separate server installation/configuration step per environment — which is exactly the model that fits containerized (Docker/Kubernetes) deployment, where you want one immutable artifact that runs identically everywhere.

Q: When would you use @ConfigurationProperties instead of @Value? [Medium]

Whenever you have more than a couple of RELATED properties — @ConfigurationProperties binds a whole prefixed group into one type-safe POJO in a single step, fails clearly at startup on type mismatches, and supports IDE autocompletion. @Value pulling in each property individually gets unwieldy and gives no compile-time safety on typo'd keys.

Q: What's the practical difference between @SpringBootTest and @WebMvcTest? [Medium]

@SpringBootTest boots the ENTIRE application context — realistic but slow, appropriate for true integration tests. @WebMvcTest loads only the web layer (the controller under test, filters, JSON conversion) and expects you to @MockBean out the service/repository layers — much faster, appropriate for testing controller behavior/routing/validation in isolation.

Q: Why does /actuator/health matter for a service running in Kubernetes? [Medium]

Kubernetes uses liveness and readiness probes to decide whether to keep routing traffic to a pod or restart it — /actuator/health (optionally extended with custom HealthIndicators for downstream dependencies) is the standard endpoint those probes hit. If your app reports DOWN, Kubernetes can stop sending traffic to that instance before customers ever see errors from it.

Confidential C

7.14 Cheat Sheet — Spring Boot (last-minute revision)

- @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan; main class must sit at package root
- Auto-configuration = @Conditional-guarded @Configuration classes, listed in AutoConfiguration.imports, evaluated at startup
- @ConditionalOnClass (classpath check), @ConditionalOnMissingBean (override by defining your own), @ConditionalOnProperty
- Starters = pure dependency aggregators, no code of their own, bundle compatible versions
- Embedded server = self-contained fat jar, no external Tomcat needed — fits Docker/Kubernetes deployment naturally
- @Value = single property, no type safety. @ConfigurationProperties = whole group, type-safe POJO — prefer for anything non-trivial
- Profiles: @Profile + application-{profile}.yml + spring.profiles.active to swap config per environment
- Actuator: /health (K8s probes), /metrics, /info, /env — extend with custom HealthIndicator beans
- @RestControllerAdvice + @ExceptionHandler = centralized error handling, one place instead of try-catch everywhere
- @Valid (standard Bean Validation) vs @Validated (Spring's, needed for validation groups + method-level validation)
- SpringApplication.run(): determine app type -> prepare Environment -> create ApplicationContext -> refresh (bean lifecycle happens here, server starts) -> run CommandLineRunner/ApplicationRunner beans
- @SpringBootTest = full context (slow, integration). @WebMvcTest = web layer only. @DataJpaTest = repository layer only — both faster, more focused

Confidential C

PART 8

Hibernate & JPA

JDBC forces you to manually map every ResultSet row to an object, manage connections, and hand-write SQL for every query. JPA/Hibernate exist to let you work with plain Java objects and have the framework translate that into SQL — but that translation layer hides real behavior (lazy loading, caching, dirty checking) that you MUST understand, or it will bite you in production.

8.1 JPA vs Hibernate — Spec vs Implementation

JPA	Hibernate
A SPECIFICATION (a set of interfaces/annotations) — javax.persistence / jakarta.persistence	An IMPLEMENTATION of that specification (plus extra, Hibernate-only features)
Defines EntityManager, @Entity, @Id, JPQL, etc.	Provides the actual Session (extends EntityManager), the SQL generation engine, caching, etc.
You code against JPA interfaces to stay portable across providers	Other JPA providers exist too (EclipseLink) — Hibernate is just the dominant one

The one-liner interviewers want

"JPA is the interface, Hibernate is the most common implementation of it — like how JDBC is a spec and each database vendor ships its own driver implementing it."

8.2 Entity Mapping Basics

```
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String name;
}
```

@GeneratedValue strategy	Behavior
IDENTITY	DB auto-increment column generates the ID — simple, but disables Hibernate's JDBC batch inserts (ID must be known immediately after each insert)
SEQUENCE	Uses a DB sequence object — allows batching, generally the recommended choice for Postgres/Oracle
AUTO	Hibernate picks a strategy based on the underlying database dialect
TABLE	Simulates a sequence using a regular table — portable but slowest, rarely used

Confidential C

8.3 Entity Relationships

The trickiest part of relationships isn't the annotation — it's understanding OWNERSHIP: which side actually holds the foreign key column, and therefore which side's changes actually get persisted to the database.

```
@Entity
class Employee {
    @ManyToOne
    @JoinColumn(name = "department_id")   // Employee is the OWNING side — it has the FK
column
    private Department department;
}

@Entity
class Department {
    @OneToMany(mappedBy = "department")   // mappedBy = 'I don't own this FK, Employee does'
    private List<Employee> employees;
}
```

Interview trap: mappedBy and the 'inverse side'

The side WITHOUT mappedBy owns the foreign key and is the only side whose changes are actually written to the DB by default. If you only add an Employee to department.getEmployees() (the inverse/mappedBy side) and never set employee.setDepartment(department) (the owning side), Hibernate writes NOTHING to the database — a very common 'why isn't my relationship saving' bug.

Relationship	Typical real-world example
@OneToOne	User <-> UserProfile (one profile per user)
@OneToMany / @ManyToOne	Department <-> Employee (many employees, one department) — by far the most common
@ManyToMany	Student <-> Course (needs a join table); in practice often modeled as TWO @OneToMany relationships to an explicit join entity instead, so you can add extra columns (e.g. enrollment date) to the join table

Fetch Types: LAZY vs EAGER

LAZY	EAGER
Related entity is loaded ONLY when you actually access it (a proxy is loaded initially)	Related entity is loaded IMMEDIATELY, in the same query (or an extra one) as the parent
Default for @OneToMany and @ManyToMany	Default for @ManyToOne and @OneToOne — often a hidden performance trap

Production default worth stating out loud in an interview

Most experienced teams explicitly override @ManyToOne/@OneToOne to FetchType.LAZY everywhere, and load related data deliberately (via JOIN FETCH or an entity graph) only where actually needed — relying on EAGER defaults is a common source of unpredictable extra queries and the N+1 problem below.

Cascade Types

- CascadeType.PERSIST/MERGE/REMOVE/ALL: control whether an operation on the PARENT entity automatically propagates to related child entities. E.g., CascadeType.ALL on Order -> OrderItems means deleting an Order automatically deletes its OrderItems too.

Confidential C

- orphanRemoval = true (only meaningful on the parent side of a One-to-Many): if a child is REMOVED FROM the parent's collection (not just deleted directly), Hibernate deletes it from the DB too — needed for a true 'this child only exists as part of its parent' relationship.

8.4 The Persistence Context — Hibernate's First-Level Cache

The EntityManager (Hibernate's Session) maintains a Persistence Context — essentially a per-transaction cache of every entity it has loaded or saved. This is what powers two behaviors people often don't realize are connected:

- Identity guarantee: calling entityManager.find(Employee.class, 1L) twice in the SAME transaction returns the exact SAME Java object reference both times — Hibernate doesn't hit the DB again, it returns the cached instance.
- Dirty checking: Hibernate compares an entity's current field values against a snapshot taken when it was loaded. At transaction commit (flush time), ANY changed field gets an automatic UPDATE statement generated — you never call an explicit 'save' after just mutating a loaded entity's setter.

```
@Transactional
public void giveRaise(Long employeeId, double amount) {
    Employee e = entityManager.find(Employee.class, employeeId);
    e.setSalary(e.getSalary() + amount);
    // NO explicit save()/update() call needed — dirty checking generates the
    // UPDATE automatically when the transaction commits and the context flushes.
}
```

Entity Lifecycle States

State	Meaning
Transient	A plain 'new Employee()' — not yet associated with any persistence context, no row in the DB
Persistent (managed)	Attached to an active persistence context — any changes are tracked and auto-flushed to the DB
Detached	Was persistent, but the persistence context has since closed (e.g. transaction ended) — changes are no longer tracked
Removed	Marked for deletion via entityManager.remove() — the DELETE happens at flush/commit time

The classic detached-entity bug: LazyInitializationException

If you load an entity with a LAZY collection, close the persistence context (e.g. the @Transactional method returns), and THEN try to access that lazy collection (e.g. in your web layer while building a JSON response), Hibernate throws LazyInitializationException — there's no active session left to run the extra query.

Fixes: fetch what you need (JOIN FETCH / entity graph) while the session is still open, restructure the query to fetch eagerly for that specific use case, or map to a DTO inside the transactional service method before returning.

Confidential C

8.5 The N+1 Select Problem — the Single Most-Asked Hibernate Topic

Imagine fetching 50 departments, then looping over each to print its employees (a LAZY @OneToMany):

```
List<Department> depts = departmentRepo.findAll();   // 1 query: SELECT * FROM department
for (Department d : depts) {
    System.out.println(d.getEmployees().size());     // triggers 1 SEPARATE query PER
department!
}
// Total: 1 + 50 = 51 queries for what should have been 1 or 2.
```

Why this is so dangerous in practice

It's invisible in development with a small/seeded dataset (a handful of extra queries, unnoticed), and only becomes a real, visible performance problem in production once the table has thousands of rows — a classic 'worked fine in testing, fell over in prod' story, which is exactly why interviewers love asking about it.

The fixes

```
// Fix 1: JOIN FETCH in JPQL — pulls departments AND employees in ONE query
@Query("SELECT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees();

// Fix 2: @EntityGraph — declarative, doesn't require writing custom JPQL
@EntityGraph(attributePaths = "employees")
List<Department> findAll();

// Fix 3: batch fetching — still N+1-shaped, but fetches employees in BATCHES
// instead of one-row-at-a-time, e.g. via @BatchSize(size = 20) on the collection
```

8.6 JPQL vs Native Queries vs Criteria API

Approach	When to use
JPQL (@Query("SELECT e FROM Employee e WHERE..."))	Queries against ENTITIES/fields (not table/column names) — portable across databases, the default choice
Native SQL (@Query(nativeQuery = true))	When you need a DB-specific feature JPQL can't express (window functions, vendor-specific hints, complex CTEs)
Criteria API	Type-safe, PROGRAMMATICALLY built queries — useful for dynamic filters (e.g. an admin search screen with many optional filter fields), verbose but avoids string-concatenation query building
Spring Data derived queries	findByLastNameAndAgeGreaterThan(...) — Spring generates the JPQL from the METHOD NAME itself, no query written at all for simple cases

8.7 @Transactional — Propagation & Isolation

@Transactional is itself just AOP (recall Part 6) — it wraps your method in a proxy that starts a transaction before the method and commits/rolls back after, based on whether an exception was thrown.

Confidential C

Propagation	Meaning
REQUIRED (default)	Join the existing transaction if one is active; otherwise start a new one
REQUIRES_NEW	ALWAYS start a brand-new transaction, suspending any existing one — e.g. so an audit-log write commits even if the outer business transaction later rolls back
NESTED	Starts a true nested transaction with a savepoint — a rollback here only undoes work since that savepoint, not the whole outer transaction
SUPPORTS / NOT_SUPPORTED / MANDATORY / NEVER	Less common — control whether the method requires, forbids, or is indifferent to an existing transaction
Isolation level	**What it prevents**
READ_UNCOMMITTED	Nothing — allows dirty reads (seeing another transaction's uncommitted changes)
READ_COMMITTED (most DBs' default)	Prevents dirty reads; non-repeatable reads and phantom reads still possible
REPEATABLE_READ (MySQL/InnoDB default)	Prevents dirty + non-repeatable reads; phantom reads still possible (though InnoDB's implementation largely avoids these too)
SERIALIZABLE	Prevents all three — transactions behave as if run one at a time, at the cost of the most locking/lowest concurrency

Interview trap: @Transactional on a private or self-invoked method

Exactly the same proxy limitation from Part 6's AOP section applies here — @Transactional is silently ignored on private methods (no proxy can intercept them) and on public methods called via 'this' from within the same class.

8.8 Optimistic vs Pessimistic Locking

Optimistic Locking	Pessimistic Locking
Assumes conflicts are RARE — no DB lock taken while reading/editing	Assumes conflicts are LIKELY — takes an actual DB row lock immediately (SELECT ... FOR UPDATE)
Implemented via a @Version column; UPDATE fails (OptimisticLockException) if the version changed since it was read	Other transactions trying to touch that row simply BLOCK until the lock is released
Better throughput/concurrency; requires handling the conflict (retry, show user an error)	Guarantees no conflict, but risks blocking/deadlocks and hurts throughput under contention

```
@Entity
class Account {
    @Id private Long id;
    private BigDecimal balance;
    @Version
    private int version;    // Hibernate auto-increments this on every UPDATE,
                             // and adds "WHERE version = ?" to the SQL —
```


Confidential C

```
                                // if another transaction updated it first, 0 rows match and it
throws.
}
```

8.9 The equals()/hashCode() Trap in Entities

Recall from Part 1 that equals/hashCode must stay consistent — but for JPA entities, the 'natural' choice of using the auto-generated @Id creates its own trap: before the entity is saved, its ID is null.

Why this breaks HashSet/HashMap

If you add a transient (unsaved) entity to a HashSet using its ID for hashCode(), then save it (Hibernate assigns the ID), the entity's hashCode CHANGES after insertion into the Set — it's now sitting in the wrong bucket, and set.contains(thatSameEntity) can return false.

Common fix: base equals()/hashCode() on a business/natural key that never changes (e.g. an email or an externally-assigned unique code) instead of the DB-generated ID — or use a constant hashCode() (e.g. return getClass().hashCode()) if no natural key exists, sacrificing bucket distribution for correctness.

Confidential C

8.10 Hibernate/JPA Interview Q&A Bank

Q: What is the N+1 select problem, and how do you actually detect and fix it? [Very High]

Fetching a list of N parent entities, then lazily accessing a related collection/entity on each one triggers one additional query PER parent — N+1 total queries instead of 1 or 2. Detect it by turning on SQL logging (or a tool like p6spy/datasource-proxy) and counting queries in a loop; fix with JOIN FETCH, @EntityGraph, or batch fetching depending on the use case.

Q: Explain the difference between FetchType.LAZY and FetchType.EAGER, and which you'd choose by default. [Very High]

LAZY loads the related entity/collection only when actually accessed (a proxy stands in until then); EAGER loads it immediately alongside the parent. I default everything to LAZY (overriding @ManyToOne/@OneToOne's EAGER default) and fetch eagerly only per-query where actually needed — EAGER-by-default is a common cause of unpredictable extra queries and can quietly cause N+1 or over-fetching across a whole codebase.

Q: What causes LazyInitializationException, and what are the real fixes (not just 'make it EAGER')? [Very High]

It happens when you try to access a lazy association after the persistence context/session that loaded the entity has already closed — there's no active session to run the extra query. Real fixes: fetch what you need up front (JOIN FETCH/EntityGraph) while the session is open, or convert to a DTO inside the still-open transactional method before returning it up the stack. Blanket-switching to EAGER just trades an exception for a silent, often much larger, performance problem.

Q: What is dirty checking, and how does Hibernate know which fields changed? [High]

When an entity is loaded, Hibernate keeps a snapshot of its field values. At flush/commit time, it compares the entity's CURRENT state against that snapshot; any field that differs gets included in an automatically-generated UPDATE statement — you never need to call an explicit save() after mutating a managed entity via its setters within a transaction.

Q: What does mappedBy actually mean, and what happens if you set the relationship only on the mappedBy side? [High]

mappedBy marks the INVERSE side of a bidirectional relationship — the side WITHOUT mappedBy owns the foreign key column and is what Hibernate actually persists. If you only update the mappedBy (inverse) side's collection and never set the reference on the owning side, Hibernate writes nothing to the database for that relationship — a very common 'my relationship silently isn't saving' bug.

Q: Explain optimistic locking with @Version — what actually happens on a conflicting concurrent update? [High]

Hibernate adds a version column to the entity and includes 'WHERE version = ?' in every UPDATE. If a concurrent transaction already bumped the version between your read and write, zero rows match the WHERE clause, and Hibernate throws OptimisticLockException — the application then decides how to handle it (retry, surface a conflict error to the user), rather than silently overwriting someone else's change.

Q: Why is it risky to use an entity's auto-generated @Id in its equals()/hashCode()? [Medium]

Before the entity is persisted, its ID is null, so its hashCode differs before vs. after being saved. If the entity was added to a HashSet/HashMap while still transient, it ends up in the wrong bucket once its hashCode changes post-save, breaking contains()/get() for that entity. A stable business key (or a constant hashCode) avoids this.

Q: What's the practical difference between JPQL and native SQL queries? [Medium]

JPQL queries against entity names and fields (not table/column names) and stays portable across database vendors — Hibernate translates it into the appropriate dialect's SQL. Native queries are needed when you require a vendor-specific feature JPQL simply can't express, at the cost of database portability.

Q: What's the difference between @Transactional propagation REQUIRED and REQUIRES_NEW? [Medium]

REQUIRED joins whatever transaction is already active, or starts one if none exists — a rollback anywhere rolls back everything. REQUIRES_NEW always suspends any existing transaction and starts a completely independent one, so its

Confidential C

outcome (commit or rollback) is isolated from the outer transaction — useful for something like an audit log write you want to persist even if the surrounding business transaction later fails.

Confidential C

8.11 Cheat Sheet — Hibernate & JPA (last-minute revision)

- JPA = specification (interfaces/annotations). Hibernate = the dominant implementation of it.
- @GeneratedValue: IDENTITY (simple, disables batching) vs SEQUENCE (allows batching, preferred)
- mappedBy side = inverse, no FK column. Non-mappedBy side = owning, holds the FK — must set the owning side for changes to persist.
- LAZY vs EAGER: default @ManyToOne/@OneToOne to LAZY everywhere; fetch eagerly per-query only where needed
- Persistence context = 1st-level cache, per transaction: guarantees identity + powers dirty checking (auto UPDATE on flush, no explicit save needed)
- Entity states: Transient (new, unmanaged) -> Persistent/managed (tracked) -> Detached (context closed) / Removed (marked for delete)
- LazyInitializationException = accessing a lazy association after the session closed. Fix with JOIN FETCH/EntityGraph or DTO conversion inside the transaction — not blanket EAGER.
- N+1 problem: 1 parent query + N child queries in a loop. Fix: JOIN FETCH, @EntityGraph, or batch fetching.
- @Transactional propagation: REQUIRED (join/create) | REQUIRES_NEW (always isolated) | NESTED (savepoint)
- Isolation: READ_UNCOMMITTED < READ_COMMITTED < REPEATABLE_READ < SERIALIZABLE (more prevention, less concurrency)
- Optimistic locking (@Version, fails on conflict, higher throughput) vs Pessimistic locking (SELECT FOR UPDATE, blocks, guarantees no conflict)
- Never base entity equals()/hashCode() on an auto-generated ID that's null before save — use a stable business key instead

Confidential C

PART 9

Spring Security + JWT

Security in a web app boils down to two separate questions asked on every request: WHO are you (authentication), and are you ALLOWED to do this specific thing (authorization). Spring Security's entire architecture is a pipeline of filters built to answer those two questions before your controller code ever runs.

9.1 Authentication vs Authorization

Authentication	Authorization
Proving WHO you are (login, verifying credentials)	Deciding WHAT you're allowed to do, given who you are
Result: a 401 Unauthorized if it fails (misleadingly named — it really means 'not authenticated')	Result: a 403 Forbidden if it fails (you ARE known, but not permitted)
Example: checking a username/password, or validating a JWT signature	Example: checking if this authenticated user has the ADMIN role

9.2 The Security Filter Chain

Before a request ever reaches your @RestController, it passes through a chain of Servlet Filters that Spring Security registers. Each filter has one job; together they establish who the caller is and whether they're allowed through.

```
HTTP Request
     |
     v
SecurityContextPersistenceFilter -- loads existing SecurityContext (e.g. from session)
     |
     v
UsernamePasswordAuthenticationFilter -- handles form-login POST /login
(or your custom JwtAuthenticationFilter, for token-based auth)
     |
     v
ExceptionTranslationFilter -- catches AccessDeniedException/AuthenticationException
     |
     v
AuthorizationFilter -- makes the final allow/deny decision
     |
     v
Your DispatcherServlet / @RestController
```

Key idea to internalize

Security in Spring is NOT something wrapped around your controller like AOP — it happens entirely in the SERVLET FILTER layer, BEFORE Spring MVC's DispatcherServlet is even reached. This is why a security rejection never touches your controller code at all.

9.3 Configuring Security (Modern Lambda DSL)

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```


Confidential C

```
        return http
            .csrf(csrf -> csrf.disable())      // typically disabled for stateless JWT APIs
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

- This XML-free, lambda-based DSL (SecurityFilterChain bean) is the current standard since Spring Security 5.7+ — the older WebSecurityConfigurerAdapter subclassing style is deprecated.

9.4 How a Login Request Actually Flows Through Spring Security

```
1. UsernamePasswordAuthenticationFilter intercepts POST /login, builds an
   UNauthenticated Authentication object (username + raw password)
2. Passes it to the AuthenticationManager
3. AuthenticationManager delegates to a matching AuthenticationProvider
4. AuthenticationProvider calls UserDetailsService.loadUserByUsername()
   to fetch the real user record (from DB, typically)
5. PasswordEncoder.matches(raw, encoded) compares the submitted password
   against the stored (hashed) one
6. On success: a fully AUTHENTICATED Authentication object is built and
   stored in the SecurityContext (held in a ThreadLocal for that request)
7. On failure: AuthenticationException is thrown, translated into a 401
```

Why passwords are hashed with BCrypt, not encrypted

Encryption is reversible (given the key); hashing (BCrypt, Argon2) is NOT — you can never recover the original password from the hash, only check if a given input produces the SAME hash. BCrypt specifically also includes a built-in random SALT and a configurable 'work factor' (cost), deliberately making it slow to compute — which makes brute-force/rainbow-table attacks impractical even if your password database leaks.

Confidential C

9.5 JWT (JSON Web Tokens) — Stateless Authentication

Traditional session-based auth stores login state on the SERVER (a session store), and the browser just holds a session ID cookie. That doesn't scale cleanly across multiple server instances without a shared session store. JWT flips this: the token ITSELF carries the user's identity/claims, cryptographically signed, so ANY server instance can verify it independently with no shared state at all.

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huIiwicm9sZSI6IkFETUlOIn0.5mZ2xhc3N3aW5n...
|------ HEADER ------| |----------- PAYLOAD -----------| |--- SIGNATURE ---|
{"alg":"HS256"}        {"sub":"john","role":"ADMIN","exp":..}  HMACSHA256(
algorithm used           the actual CLAIMS — readable          base64(header)+"."
                         by ANYONE (base64, NOT encrypted!)     +base64(payload), key)
```

Critical misconception to avoid in an interview

A JWT's payload is base64-ENCODED, not encrypted — anyone can decode it and read the claims (try it at jwt.io). The SIGNATURE is what makes it trustworthy: it proves the token was issued by someone holding the secret key and hasn't been tampered with since. NEVER put sensitive data (passwords, secrets) directly in a JWT payload — it is fully readable by the client and anyone who intercepts it.

Verification — what happens on every incoming request

1. Extract the token from the Authorization: Bearer <token> header.
2. Recompute the signature using the server's secret key over the received header+payload.
3. Compare it to the signature embedded in the token — a mismatch means the token was tampered with or wasn't issued by this server, and is rejected.
4. Check the exp (expiry) claim — an expired token is rejected even with a valid signature.
5. If valid, build an Authentication object from the token's claims and set it in the SecurityContext for this request — no database lookup required at all for a pure JWT flow.

Implementing the filter

```
public class JwtAuthFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws IOException, ServletException {
        String token = extractTokenFromHeader(req);
        if (token != null && jwtUtil.isValid(token)) {
            var auth = new UsernamePasswordAuthenticationToken(
                jwtUtil.getUsername(token), null, jwtUtil.getAuthorities(token));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        chain.doFilter(req, res);  // ALWAYS continue — let downstream authorization
    }                               // rules make the final allow/deny call
}
```

- OncePerRequestFilter guarantees the filter runs exactly ONCE per request, even with internal request forwarding — a naive Filter could otherwise run twice and double-process the token.

Access Tokens vs Refresh Tokens

Access Token	Refresh Token
Short-lived (minutes), sent on EVERY API request, proves identity for that request	Long-lived (days/weeks), used ONLY to obtain a new access token once the old one expires
If stolen, damage window is small (it expires quickly)	Stored more securely (often httpOnly cookie), and can be individually revoked server-side (unlike a pure stateless access token)

Confidential C

Access Token	Refresh Token

Why you can't just make access tokens long-lived instead

A pure JWT can't be individually 'logged out' or revoked server-side before its expiry (there's no server-side state to delete) — so keeping access tokens SHORT limits the damage of a stolen token, while the refresh token (which CAN be revoked, since issuing a new access token is a server-side operation you control) handles staying logged in.

9.6 Method-Level Security

```
@EnableMethodSecurity   // must be enabled on a @Configuration class

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }

@PreAuthorize("#userId == authentication.principal.id")  // caller can only see their own data
public User getUser(Long userId) { ... }
```

- @PreAuthorize runs BEFORE the method (can reference method arguments, as shown above). @PostAuthorize runs AFTER, and can inspect the RETURN VALUE (e.g. 'only return this if returnObject.owner == principal').
- This is Spring AOP again under the hood — the same proxy mechanism from Part 6, meaning the same self-invocation limitation applies here too.

9.7 CORS and CSRF — Frequently Confused

CORS (Cross-Origin Resource Sharing)	CSRF (Cross-Site Request Forgery)
A BROWSER SECURITY RELAXATION mechanism — the server explicitly tells the browser which other origins are allowed to call it via JS (fetch/XHR)	An ATTACK where a malicious site tricks a victim's browser into submitting a request to a DIFFERENT site where the victim happens to already be logged in (relying on auto-sent cookies)
Configured via allowed origins/methods/headers	Defended against via a CSRF token the attacker's page can't know/replay

Why CSRF protection is usually disabled for stateless JWT APIs

CSRF attacks exploit the browser AUTOMATICALLY attaching cookies to a request, including ones initiated by a malicious third-party page. A JWT sent via a manually-set Authorization header is NOT automatically attached by the browser the way a cookie is — an attacker's page has no way to make the victim's browser send that header — so the entire CSRF attack vector doesn't apply the same way, and Spring Security's CSRF protection (designed for cookie/session-based auth) is typically disabled for pure token-based APIs.

If you DO store the JWT in a cookie (rather than sending it via header) for XSS-mitigation reasons, you need CSRF protection back, since now it IS auto-attached by the browser.

9.8 Common Vulnerabilities — Interview Awareness Checklist

Vulnerability	Defense
SQL Injection	Use parameterized queries/PreparedStatement (JPA/Hibernate does this by default) — never string-concatenate user input into SQL

Confidential C

Vulnerability	Defense
XSS (Cross-Site Scripting)	Escape/encode any user-supplied content rendered back into HTML; frameworks like Thymeleaf auto-escape by default
CSRF	CSRF tokens for cookie/session-based apps (see above); largely N/A for header-based JWT APIs
Broken Authentication	Strong password hashing (BCrypt), account lockout after repeated failures, short-lived tokens
Sensitive Data Exposure	HTTPS everywhere, never log tokens/passwords/PII, encrypt sensitive fields at rest

Confidential C

9.9 Spring Security + JWT Interview Q&A Bank

Q: Where does Spring Security actually run — inside Spring MVC, or somewhere else? [Very High]

Entirely in the Servlet FILTER layer, before DispatcherServlet is ever reached. A chain of filters (session/context loading, authentication, exception translation, authorization) decides whether the request is even allowed to proceed — a rejected request never touches your controller code.

Q: Is a JWT encrypted? Can a client read what's inside it? [Very High]

No — the header and payload are only base64-ENCODED, fully readable by anyone who has the token (try decoding one at jwt.io). What makes it trustworthy is the SIGNATURE, computed with a secret key only the server (or servers) knows — it proves the token wasn't tampered with, not that its contents are hidden. Sensitive data should never be placed directly in the payload.

Q: Walk me through what happens when a request with a JWT hits your API. [Very High]

A custom filter (typically extending OncePerRequestFilter) extracts the token from the Authorization header, recomputes the signature using the server's secret and compares it to the token's embedded signature, checks the expiry claim, and — if everything checks out — builds an Authentication object from the token's claims and sets it in the SecurityContext for that request. No database call is needed for a pure JWT flow, which is what makes it 'stateless'.

Q: Why use short-lived access tokens plus a separate refresh token instead of one long-lived token? [High]

A stateless JWT can't be individually revoked server-side before it naturally expires. Keeping the access token short-lived limits the damage window if it's ever stolen; the refresh token (used only to mint new access tokens, and which CAN be revoked/tracked server-side) lets the user stay logged in without needing a long-lived, un-revocable access token floating around.

Q: Why is CSRF protection typically disabled for a stateless JWT-based REST API? [High]

CSRF attacks rely on the browser AUTOMATICALLY attaching credentials (cookies) to a forged cross-site request. A JWT sent via a manually-set Authorization header isn't something a malicious page can force the victim's browser to attach — so the attack vector CSRF protection defends against doesn't apply the same way. If the JWT is instead stored in a cookie, CSRF protection becomes necessary again.

Q: What's the difference between @PreAuthorize and @PostAuthorize? [Medium]

@PreAuthorize evaluates BEFORE the method runs and can reference the method's arguments (e.g., checking a passed-in userId against the caller's own ID). @PostAuthorize evaluates AFTER the method returns and can inspect the actual RETURN VALUE (e.g., only allow the result through if returnObject.ownerId matches the caller) — useful when the authorization decision depends on data you don't have until after fetching it.

Q: Why use BCrypt instead of a fast hash like SHA-256 for passwords? [Medium]

SHA-256 is designed to be FAST, which is exactly the wrong property for password storage — it makes brute-force and rainbow-table attacks cheap if the hash database ever leaks. BCrypt is deliberately slow (configurable work factor) and includes a built-in random salt per password, making large-scale brute-force attacks computationally impractical.

Q: What's the actual difference between a 401 and a 403 response, and when does each occur? [Medium]

401 Unauthorized means authentication itself failed or wasn't provided — the server doesn't know who you are (despite the confusing name). 403 Forbidden means you ARE authenticated/known, but you don't have permission for this specific action — e.g., a logged-in non-admin user hitting an admin-only endpoint.

Confidential C

9.10 Cheat Sheet — Spring Security + JWT (last-minute revision)

- Authentication = who you are (401 on failure). Authorization = what you're allowed to do (403 on failure).
- Security runs in the SERVLET FILTER chain, before DispatcherServlet — rejections never reach your controller
- Modern config: SecurityFilterChain @Bean with lambda DSL, not WebSecurityConfigurerAdapter (deprecated)
- Login flow: UsernamePasswordAuthenticationFilter -> AuthenticationManager -> AuthenticationProvider -> UserDetailsService + PasswordEncoder
- BCrypt: slow on purpose + built-in salt — defends against brute-force even if the DB leaks
- JWT = header.payload.signature — payload is base64-encoded (readable), NOT encrypted. Signature proves authenticity.
- JWT verification: extract token -> recompute signature -> compare -> check exp -> set SecurityContext. No DB call needed.
- Short-lived access token + longer-lived, revocable refresh token — limits damage from a stolen access token
- @PreAuthorize (before, can check args) vs @PostAuthorize (after, can check return value) — both are AOP-proxy based, same self-invocation limitation as @Transactional
- CORS = browser-enforced cross-origin permission (server opts in). CSRF = an attack exploiting auto-sent cookies — largely N/A for header-based JWT, still relevant if JWT lives in a cookie
- SQLi defense = parameterized queries (JPA does this by default). XSS defense = output escaping. Broken auth defense = BCrypt + short token lifetimes + lockouts

Confidential C

PART 10

Microservices

A monolith fails as ONE unit and scales as ONE unit — simple, but eventually every team is blocked on the same deploy pipeline and the same codebase. Microservices split the system into independently deployable services, but that trades simplicity for a whole new category of problems: how do services FIND each other, what happens when one is slow or down, and how do you keep data consistent without one big database transaction. This part is about those new problems and their standard solutions.

10.1 Monolith vs Microservices — the Actual Trade-off

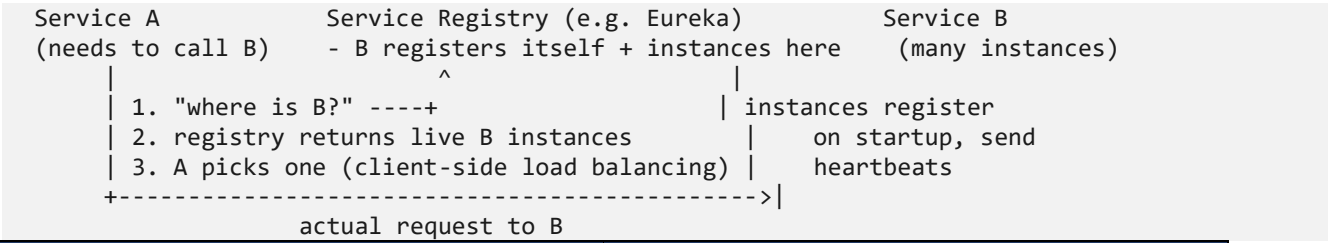
Monolith	Microservices
One codebase, one deployment, one database (usually)	Many independently deployable services, each with its OWN database
Simple to develop/test/deploy initially; a single transaction across modules is trivial	Each service can be built, deployed, and scaled independently, by independent teams
Scales as a whole — can't scale just the busy module	Can scale JUST the hot service (e.g. only the search service, not the whole app)
One bug can bring down the entire application	A failure can be isolated to one service — if done right, the rest keeps working
Gets harder to maintain/deploy as it grows ('big ball of mud')	Adds real operational complexity: network calls, distributed data, service discovery, tracing

The honest interview answer about when to use which

"Microservices solve organizational scaling problems (many teams working independently) more than they solve technical ones — they trade simplicity for independent deployability. Most systems should START as a well-modularized monolith and split out services only once a specific module has a genuinely different scaling/team/deployment need — splitting too early adds massive complexity for no real benefit."

10.2 Service Discovery

In a monolith, one module calls another via a plain Java method call. In microservices, Service A needs to call Service B over the network — but B might have multiple running instances, each with a different, possibly-changing IP address (especially in a container/Kubernetes environment where instances come and go). Hardcoding IPs doesn't work.



Client-side discovery	Server-side discovery
The CALLING service queries the registry directly and picks an instance itself (e.g. Netflix Eureka + a client-side load balancer)	A separate load balancer/router queries the registry and routes the request — the caller doesn't need discovery logic at all (e.g. AWS ALB, Kubernetes Service)

Confidential C

10.3 API Gateway

Without a gateway, every client (mobile app, web frontend, 3rd party) needs to know the addresses of EVERY individual microservice, and every service needs to duplicate cross-cutting logic like auth checks and rate limiting. An API Gateway (Spring Cloud Gateway is the modern choice) sits in front of everything as a single entry point.

- Routing: forwards /orders/** to the Order Service, /users/** to the User Service, etc. — clients only ever talk to the gateway.
- Cross-cutting concerns handled ONCE, centrally: authentication/JWT validation, rate limiting, request logging, CORS, SSL termination.
- Can aggregate multiple downstream service calls into one response for the client, reducing round trips over a slow mobile network.

10.4 Inter-Service Communication: Sync vs Async

Synchronous (REST / gRPC)	Asynchronous (message queue: Kafka / RabbitMQ)
Caller BLOCKS waiting for a direct response — simple mental model, request/response	Caller publishes an event/message and moves on — a consumer processes it independently, whenever it can
Tighter coupling: if the callee is down/slow, the caller is directly affected	Looser coupling: the queue buffers messages, so a temporarily-down consumer doesn't block the producer
Good fit: needs an immediate answer to proceed (e.g. 'is this payment valid right now')	Good fit: fire-and-forget notifications, events other services react to eventually (e.g. 'order placed' triggering email + inventory update + analytics)

```
// Declarative REST client (Feign) — call another service almost like a local method
@FeignClient(name = "inventory-service")
public interface InventoryClient {
    @GetMapping("/stock/{sku}")
    StockResponse getStock(@PathVariable String sku);
}
```

Why cascading failures are the real danger of synchronous chains

If Service A calls B, which calls C, and C becomes slow, A's threads pile up waiting on B, which is waiting on C — a slowdown in ONE deep service can exhaust thread pools and take down services that have nothing directly wrong with them. This is exactly the failure mode circuit breakers exist to prevent.

Confidential C

10.5 Circuit Breaker Pattern (Resilience4j)

Modeled directly on an electrical circuit breaker: if calls to a downstream service keep failing, STOP calling it for a while instead of letting every new request pile up waiting on a service that's clearly unhealthy.

```
  CLOSED (normal) -- failure rate exceeds threshold -->  OPEN (fail fast, no calls made)
     ^                                                     |
     |                                                     | after a wait duration
     +---- success rate looks healthy again ---- HALF_OPEN <----+
                                                  (lets a FEW test
                                                   calls through)
```

State	Behavior
CLOSED	Normal operation — requests pass through and are monitored
OPEN	Circuit has tripped — requests fail IMMEDIATELY (or return a fallback) without even attempting the real call, protecting both the caller's threads and the already-struggling downstream service
HALF_OPEN	After a cooldown period, a LIMITED number of test requests are let through to see if the downstream service has recovered

```
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallbackStock")
public StockResponse getStock(String sku) {
    return inventoryClient.getStock(sku);
}
public StockResponse fallbackStock(String sku, Throwable t) {
    return StockResponse.unknown();  // degrade gracefully instead of cascading failure
}
```

- Often paired with a Bulkhead (isolating thread pools per downstream dependency, so one slow dependency can't exhaust threads needed for calls to a different, healthy dependency) and a Retry (with exponential backoff, for transient failures).

10.6 Centralized Configuration (Spring Cloud Config)

With dozens of services, each with dev/staging/prod configs, keeping properties files in sync across every service's repo doesn't scale. A Config Server centralizes all configuration (often backed by a Git repo) and every service fetches its config from it at startup.

- Enables changing config WITHOUT rebuilding/redeploying a service (combined with Spring Cloud Bus, config changes can even be pushed to running instances live).
- Sensitive values are typically layered with a secrets manager (Vault, AWS Secrets Manager) rather than stored in plain text, even in a private Git repo.

10.7 Distributed Transactions: the Saga Pattern

In a monolith, placing an order (reserve inventory + charge payment + create shipment) can be ONE database transaction — all-or-nothing. Across separate microservices with separate databases, there's no single transaction that spans all of them. Two-Phase Commit (2PC) technically exists but is rarely used in practice — it requires all participants to be locked and available simultaneously, which kills availability and doesn't fit how independent services are meant to operate.

The Saga pattern instead breaks the operation into a SEQUENCE of local transactions, each in its own service, with a defined COMPENSATING action to undo each step if a later step fails.

```
  Reserve Inventory --> Charge Payment --> Create Shipment
```


Confidential C

```
       |                    |                    |
       | (if payment fails) | (if shipment fails) |
       v                    v                    v
  Release Inventory <-- Refund Payment <-----------+
  (compensating action) (compensating action)
```

Choreography	Orchestration
Each service publishes an event when it finishes; the NEXT service listens and reacts — no central coordinator	A central Saga Orchestrator explicitly tells each service what to do next and handles failure/compensation logic itself
Simpler for a FEW steps; no single point of failure; but the overall flow is implicit, spread across services, harder to trace	The overall flow is explicit and easy to see in ONE place; but the orchestrator becomes a critical, more complex component

The trade-off you're actually giving up: strong consistency

A Saga gives you EVENTUAL consistency, not the immediate, all-or-nothing guarantee of a single ACID transaction — for a brief window mid-saga, inventory might show as reserved while payment hasn't been confirmed yet. This is a genuine, deliberate trade-off, not a bug, and it's exactly why idempotency (below) and careful state modeling matter so much in a microservices architecture.

10.8 Idempotency & Distributed Tracing

Network calls fail in ambiguous ways — a timeout doesn't tell you if the downstream service actually processed the request or not. Safe retries REQUIRE the operation to be idempotent: calling it multiple times has the same effect as calling it once.

```
// Client sends a unique idempotency key with the request
POST /payments
Idempotency-Key: 8f14e45f-ea-4c...

// Server: if this key was already processed, return the ORIGINAL result
// instead of charging the card a second time on a retried request
```

- Distributed tracing (Spring Cloud Sleuth / Micrometer Tracing + Zipkin/Jaeger) attaches a single correlation/trace ID to a request when it first enters the system, and propagates it through every downstream call — letting you reconstruct the full path of one request across a dozen services when debugging a production issue.

10.9 CAP Theorem (a Standard Follow-Up Question)

In the presence of a network Partition (P) — which WILL happen eventually in any distributed system — you must choose between Consistency (every read sees the latest write) and Availability (every request gets a response, even if it might be stale).

CP (Consistency + Partition tolerance)	AP (Availability + Partition tolerance)
During a partition, some nodes refuse requests rather than risk returning stale/conflicting data (e.g. a strongly-consistent config store like ZooKeeper/etcd)	During a partition, every node keeps responding, possibly with stale data, and reconciles later (e.g. many NoSQL stores tuned for availability, like Cassandra in its default mode)

A precise nuance worth stating

You always have Partition tolerance in a real distributed system (network partitions aren't optional to design around) — CAP is really about the choice between C and A specifically DURING a partition, not a free 3-way pick where you get to keep all three normally.

Confidential C

10.10 API Versioning

Strategy	Example
URI versioning	/api/v1/orders vs /api/v2/orders — simple, visible, but clutters the URL space
Header versioning	Accept: application/vnd.company.v2+json — keeps URLs clean, less visible/discoverable
Query parameter	/api/orders?version=2 — simple but easy to forget/omit accidentally

Confidential C

10.11 Microservices Interview Q&A Bank

Q: When would you actually recommend splitting a monolith into microservices, and when would you NOT? [High]

When distinct modules have genuinely different scaling needs, are owned by separate teams that need independent deploy cadences, or need different technology stacks. NOT when the system is small/early-stage, a single team owns everything, or the split is purely aspirational — premature splitting adds massive distributed-systems complexity (network calls, data consistency, service discovery) without a corresponding real benefit.

Q: What problem does a Circuit Breaker actually solve, and walk me through its three states. [Very High]

It prevents a struggling downstream service from cascading its failure upstream by exhausting the CALLER's resources (threads/connections) waiting on calls that are likely to fail anyway. CLOSED = normal operation, monitoring failure rate. OPEN = failures exceeded a threshold, so calls fail fast (or return a fallback) without even attempting the real call. HALF_OPEN = after a cooldown, a limited number of test requests are allowed through to check if the dependency has recovered before fully closing again.

Q: Why can't you just use a normal ACID transaction across multiple microservices? [Very High]

Each service typically owns its own separate database, and there's no single database transaction that can span independent services/databases. Two-Phase Commit exists in theory but requires all participants locked and available simultaneously, which badly hurts availability and doesn't fit how independently-deployed, independently-scaled services are meant to operate — which is why the Saga pattern (a sequence of local transactions plus compensating actions) is used instead, at the cost of only eventual consistency.

Q: What's the difference between choreography-based and orchestration-based Sagas? [High]

Choreography: each service publishes an event on completion, and the next service reacts to it — no central coordinator, simple for a few steps but the overall flow is implicit and harder to trace as complexity grows. Orchestration: a central orchestrator explicitly tells each service what to do and manages compensating actions on failure — the flow is explicit and easier to reason about, at the cost of the orchestrator itself becoming a critical, more complex component.

Q: Why does a retried network request need to be idempotent? [High]

A timeout or network failure doesn't tell you whether the downstream service actually processed the original request before failing to respond — so a naive retry risks double-processing (e.g. charging a payment twice). An idempotency key lets the server recognize 'I've already handled this exact request' and return the original result instead of repeating the side effect.

Q: Explain the CAP theorem, and clear up the common misconception about what it actually claims. [Medium]

The common misconception is that you 'pick 2 of 3' freely at all times. In reality, partition tolerance isn't optional in a real distributed system — network partitions will happen — so CAP is really about the choice between Consistency and Availability SPECIFICALLY DURING a partition: return possibly-stale data to stay available, or refuse requests to guarantee correctness.

Q: What's the practical difference between client-side and server-side service discovery? [Medium]

Client-side (e.g. Netflix Eureka + a client load balancer): the calling service itself queries the registry and picks an instance, so discovery/load-balancing logic lives in every client. Server-side (e.g. a Kubernetes Service or a cloud load balancer): a separate component does the discovery and routing, so calling services don't need any discovery-aware code at all — simpler clients, at the cost of an extra network hop through the router.

Q: When would you choose synchronous REST calls between services vs an asynchronous message queue? [Medium]

Synchronous when the caller genuinely needs an immediate answer to proceed (e.g. validating a payment before confirming an order). Asynchronous when the operation is fire-and-forget or multiple independent consumers need to react to the same event eventually (e.g. 'order placed' triggering email, inventory update, and analytics) — async also naturally decouples services so a temporarily-down consumer doesn't block the producer.

Confidential C

10.12 Cheat Sheet — Microservices (last-minute revision)

- Microservices trade simplicity for independent deployability/scalability — split for genuine team/scaling needs, not by default
- Service discovery: client-side (caller queries registry + picks instance, e.g. Eureka) vs server-side (a router does it, e.g. K8s Service)
- API Gateway = single entry point, centralizes routing + auth + rate limiting + logging, so clients don't need to know every service's address
- Sync (REST/Feign) = tight coupling, immediate answer needed. Async (Kafka/RabbitMQ) = loose coupling, fire-and-forget/event-driven
- Circuit breaker: CLOSED (normal) -> OPEN (fail fast, protects both sides) -> HALF_OPEN (test recovery) -> CLOSED
- Saga pattern replaces distributed ACID transactions: sequence of local transactions + compensating actions, eventual consistency only
- Choreography (event-driven, no coordinator, simple but implicit flow) vs Orchestration (central coordinator, explicit flow, single more-complex component)
- Idempotency keys make retries safe when you can't tell if the original request already succeeded
- Distributed tracing (Sleuth/Zipkin) = one correlation ID follows a request across every service it touches
- CAP theorem: partition tolerance is mandatory in practice; the real choice is Consistency vs Availability DURING a partition
- API versioning: URI (/v1/, /v2/ — simple, visible) vs header (clean URLs, less discoverable) vs query param

Confidential C

PART 11

SQL

Every backend role tests SQL because it's the layer where a subtly wrong query silently returns wrong data, or a missing index silently turns a 10ms query into a 10-second one under production load. This part covers the concepts, then the actual query patterns interviewers ask you to write live.

11.1 Joins

Every join answers the same question — 'combine rows from two tables based on a matching condition' — they differ only in what happens to ROWS THAT DON'T MATCH.

```
employees                    departments
id | name  | dept_id         id | name
1  | Asha   | 10               10 | Engineering
2  | Ravi   | 10               20 | Sales
3  | Meera  | NULL             30 | HR   (no employees)
```

Join type	Result on the data above
INNER JOIN	Asha-Engineering, Ravi-Engineering — ONLY rows with a match on both sides. Meera (no dept) and HR (no employees) are excluded entirely.
LEFT JOIN (LEFT OUTER)	ALL employees, matched dept where possible: Asha-Engineering, Ravi-Engineering, Meera-NULL. HR is excluded (it's not in the left/employees table).
RIGHT JOIN (RIGHT OUTER)	ALL departments, matched employees where possible: Engineering-Asha, Engineering-Ravi, Sales-NULL, HR-NULL.
FULL OUTER JOIN	EVERYTHING from both sides — every employee AND every department, NULL-filled wherever there's no match on the other side.
CROSS JOIN	Every row from table A paired with every row from table B (Cartesian product) — no matching condition at all; rarely intentional in production queries
SELF JOIN	A table joined to itself — e.g. finding each employee's MANAGER, who is also a row in the same employees table (via a manager_id column referencing employees.id)

Interview trap: WHERE vs ON with an OUTER JOIN

Putting a filter on the RIGHT-side table in the WHERE clause after a LEFT JOIN silently converts it back into an INNER JOIN — because unmatched rows have NULL in that column, and 'NULL = something' is never true, so WHERE filters them out. If you need to filter the joined table's rows WITHOUT losing the outer-join behavior, put that condition in the ON clause instead of WHERE.

11.2 Normalization

Confidential C

Normalization is the process of structuring tables to eliminate redundant data and the update anomalies that come with it (change a customer's address in one row and forget to update the other 5 rows that also had it copy-pasted in).

Normal Form	Rule (in plain terms)
1NF	Every column holds a single, atomic value — no comma-separated lists crammed into one cell, no repeating groups of columns (phone1, phone2, phone3)
2NF	Must already be 1NF, AND every non-key column depends on the WHOLE primary key, not just part of it (relevant only for composite/multi-column primary keys)
3NF	Must already be 2NF, AND no non-key column depends on ANOTHER non-key column (no 'transitive' dependencies) — e.g. storing both zip_code and city in an orders table, where city is really determined by zip_code, not by the order itself

Why you deliberately DEnormalize sometimes (a real follow-up question)

Normalization optimizes for data integrity and minimizes redundancy, but every extra normal form typically means an extra JOIN at query time. For read-heavy analytics/reporting workloads where data integrity risk is low and query speed matters most, deliberately denormalizing (duplicating some data to avoid a join) is a legitimate, common trade-off — this is exactly the reasoning behind a lot of data-warehouse/reporting-table design.

11.3 Primary Key, Foreign Key, Unique Key

Key type	Meaning
Primary Key	Uniquely identifies each row; implicitly NOT NULL + unique; a table can have only ONE
Unique Key	Also enforces uniqueness, but CAN allow a NULL (behavior varies slightly by DB), and a table can have MULTIPLE unique keys
Foreign Key	A column referencing another table's primary key — enforces REFERENTIAL INTEGRITY (you can't insert an order for a customer_id that doesn't exist, and by default can't delete a customer who still has orders)

11.4 ACID Properties

Property	Guarantee
Atomicity	A transaction is all-or-nothing — if any part fails, the ENTIRE transaction rolls back, leaving no partial changes
Consistency	A transaction moves the database from one VALID state to another — all constraints (FKs, checks, uniqueness) must hold before and after

Confidential C

Property	Guarantee
Isolation	Concurrent transactions don't see each other's uncommitted, in-progress changes (the exact DEGREE of isolation is tunable — see isolation levels in Part 8)
Durability	Once committed, a transaction's changes survive even a crash immediately afterward — typically guaranteed via a write-ahead log flushed to disk

Confidential C

11.5 Indexes — the Single Most Practically Important SQL Topic

Without an index, finding a row means scanning every row in the table (a 'full table scan') — fine for 100 rows, disastrous for 100 million. An index is a separate, sorted data structure (almost always a B-Tree) that lets the database jump straight to matching rows instead of scanning everything.

```
                    [ M ]
                   /     \
              [ D, H ]     [ R, W ]
             /   |   \     /   |   \
        [A,B] [E,F] [J,K] [N,P] [S,T] [X,Y,Z]

B-Tree: shallow, wide, always balanced. Finding any value takes only a FEW hops
(O(log n)) regardless of table size — this is why indexed lookups stay fast
even as a table grows into the millions of rows.
```

Clustered Index	Non-Clustered (Secondary) Index
Determines the ACTUAL physical storage order of the table's rows — there can be only ONE per table (usually the primary key)	A SEPARATE structure that stores the indexed column(s) plus a pointer back to the actual row — a table can have MANY
Range scans on the clustered column are extremely fast (rows are physically adjacent)	Looking up via a non-clustered index, then needing OTHER columns not in the index, requires an extra lookup back to the actual row ('bookmark lookup')

Why indexes aren't free — the trade-off interviewers want you to know

Every INSERT/UPDATE/DELETE must also update every index on that table, not just the underlying data — so a table with 10 indexes has meaningfully SLOWER writes than one with 1 or 2. This is why you don't just 'index everything' — you index columns that are actually queried/filtered/joined on frequently, and stay conscious of the write-heavy vs read-heavy trade-off for that specific table.

- A composite index on (last_name, first_name) helps queries filtering on last_name ALONE, or on BOTH columns in that order — but does NOT help a query filtering ONLY on first_name, because of how the B-Tree is sorted (left-to-right prefix rule).

11.6 GROUP BY, HAVING vs WHERE, Aggregate Functions

WHERE	HAVING
Filters INDIVIDUAL rows, BEFORE grouping happens	Filters GROUPS, AFTER aggregation (GROUP BY) has already happened
Cannot reference an aggregate function like COUNT(*) or AVG(salary)	CAN reference aggregate functions — that's specifically what it's for

```
SELECT department, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM employees
WHERE status = 'ACTIVE'          -- filters ROWS first (only active employees considered)
GROUP BY department
HAVING COUNT(*) > 5              -- filters GROUPS after aggregation (only large departments)
ORDER BY avg_salary DESC;
```

11.7 Window Functions

A GROUP BY collapses multiple rows into one summary row per group. A window function does the opposite: it computes an aggregate/ranking value ACROSS a group of rows, but keeps every original row intact (just adds a new computed column) — the 'window' (via PARTITION BY) defines which rows are grouped together for the calculation.

```
SELECT
```


Confidential C

```
    name, department, salary,
    RANK()      OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank,
    AVG(salary) OVER (PARTITION BY department)                      AS dept_avg
FROM employees;
-- Every individual employee row is still returned, each annotated with
-- its rank WITHIN its own department, and that department's average salary.
```

Function	Behavior
ROW_NUMBER()	1, 2, 3, 4... always unique, no gaps or ties — even IDENTICAL values get different sequential numbers
RANK()	1, 2, 2, 4 — TIES share the same rank, but the next rank SKIPS (gap after a tie)
DENSE_RANK()	1, 2, 2, 3 — ties share the same rank, but the NEXT rank does NOT skip
LAG() / LEAD()	Access a PREVIOUS or NEXT row's value within the window — e.g. comparing this month's revenue to last month's, in one query, without a self-join

11.8 Subqueries vs CTEs

```
-- CTE (Common Table Expression) — named, readable, can be referenced multiple times
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 100000
)
SELECT department, COUNT(*) FROM high_earners GROUP BY department;

-- Recursive CTE — walking a hierarchy (e.g. an org chart / manager chain)
WITH RECURSIVE org_chain AS (
    SELECT id, name, manager_id FROM employees WHERE id = 1     -- anchor
    UNION ALL
    SELECT e.id, e.name, e.manager_id FROM employees e
    JOIN org_chain o ON e.manager_id = o.id                     -- recursive step
)
SELECT * FROM org_chain;
```

- A CTE is generally more READABLE than an equivalent nested subquery and (in most modern DBs) performs the same — pick a CTE whenever the logic is complex enough that naming the intermediate result helps a human read the query.

11.9 Reading an EXPLAIN Plan (the Practical Skill)

EXPLAIN (or EXPLAIN ANALYZE, which actually runs the query and shows real timings) shows you HOW the database plans to execute a query — this is how you actually confirm whether your index is being used, instead of just assuming it.

Term you'll see in a plan	What it means
Seq Scan / Full Table Scan	No index used — every row was read. Red flag on a large table.
Index Scan	The index was used to find matching rows directly — what you want to see
Index Only Scan	Even better — every column needed was IN the index itself, no need to even touch the actual table row

Confidential C

Term you'll see in a plan	What it means
Nested Loop / Hash Join / Merge Join	Different strategies the optimizer picked for a JOIN, based on table sizes and available indexes

Confidential C

11.10 Classic SQL Coding Questions

Find the Nth highest salary

```
-- Using DENSE_RANK (handles ties correctly — 2nd highest means 2nd DISTINCT salary)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 2;

-- Classic portable alternative without window functions:
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Find duplicate rows

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Department-wise top earner (a very common one)

```
SELECT name, department, salary FROM (
    SELECT name, department, salary,
           RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 1;
```

Delete duplicate rows, keeping only one copy

```
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id) FROM users GROUP BY email  -- keep the lowest id per duplicate email
);
```


Confidential C

11.11 SQL Interview Q&A Bank

Q: What's the difference between WHERE and HAVING, and why can't you use an aggregate in WHERE? [Very High]

WHERE filters individual rows BEFORE any grouping/aggregation happens, so an aggregate value like COUNT(*) doesn't exist yet at that stage of query processing. HAVING filters groups AFTER GROUP BY has produced its aggregated rows, which is why it's the only place you can filter on an aggregate result like HAVING COUNT(*) > 5.

Q: Explain the difference between a clustered and a non-clustered index. [Very High]

A clustered index determines the actual physical storage order of the table's rows on disk — there can only be one per table, typically the primary key. A non-clustered index is a separate structure holding the indexed column(s) plus a pointer back to the real row; a table can have many, but looking up other columns not in the index requires an extra lookup back to the base table.

Q: Why would adding MORE indexes to a table ever be a bad idea? [High]

Every index must be updated on every INSERT/UPDATE/DELETE to that table, not just the base data — so more indexes mean measurably slower writes, plus extra storage. Indexes should be added deliberately for columns actually used in WHERE/JOIN/ORDER BY frequently, weighed against how write-heavy that specific table is.

Q: What's the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()? [Very High]

ROW_NUMBER() always assigns unique, sequential numbers even to tied values. RANK() gives tied rows the same rank but then SKIPS the next rank (1,2,2,4). DENSE_RANK() also gives tied rows the same rank but does NOT skip afterward (1,2,2,3). Which one's 'correct' depends on whether you want gaps after ties (RANK) or not (DENSE_RANK).

Q: Why does putting a condition on the right-hand table in the WHERE clause silently break a LEFT JOIN? [High]

Unmatched rows from a LEFT JOIN have NULL in every column from the right-hand table. A WHERE condition like rightTable.status = 'X' evaluates to UNKNOWN (not true) for those NULLs, so WHERE filters them out — silently turning your LEFT JOIN back into the equivalent of an INNER JOIN. The condition needs to go in the ON clause instead if you want to preserve the outer-join behavior.

Q: Walk me through 1NF, 2NF, and 3NF with a concrete example of a violation. [Medium]

1NF requires atomic values — a 'phone_numbers' column storing '555-1111, 555-2222' violates it. 2NF (relevant with a composite key) requires every non-key column to depend on the WHOLE key, not part of it. 3NF forbids transitive dependencies — e.g. an orders table storing both zip_code and city, where city is really determined by zip_code rather than by the order itself, should have city moved to a separate zip_codes lookup table.

Q: When would you deliberately denormalize a database design? [Medium]

For read-heavy reporting/analytics workloads where query speed matters more than storage efficiency or update-anomaly risk — duplicating some columns to avoid an expensive join at read time is a common, deliberate trade-off in data warehouse and reporting-table design, even though it goes against strict normalization principles.

Q: What does EXPLAIN actually tell you, and what's the biggest red flag to look for in its output? [Medium]

It shows the database's chosen EXECUTION PLAN for a query — which indexes (if any) it plans to use, the join strategy for each join, and estimated/actual row counts and costs. The biggest red flag on a large table is a 'Seq Scan' / full table scan where you expected an index to be used — it usually means the query isn't sargable (index-usable) as written, or the needed index doesn't exist.

Q: What's the difference between a CTE and a subquery, and are they functionally equivalent? [Medium]

A CTE (WITH clause) names an intermediate result so it can be referenced clearly, potentially multiple times, in the main query — mostly a readability/maintainability improvement. In most modern database engines they produce equivalent execution plans to an equivalent nested subquery, though some databases historically materialized CTEs as a separate optimization barrier — worth checking your specific engine's behavior if performance is on the line.

Confidential C

11.12 Cheat Sheet — SQL (last-minute revision)

- INNER JOIN = matches only. LEFT/RIGHT JOIN = all of one side + matches. FULL OUTER = all of both sides.
- Filtering the right-side table in WHERE after a LEFT JOIN silently turns it into an INNER JOIN — filter in ON instead
- 1NF = atomic values. 2NF = depends on the WHOLE key. 3NF = no transitive dependency between non-key columns.
- Primary key = unique + NOT NULL, one per table. Unique key = unique, can allow NULL, many per table. Foreign key = enforces referential integrity.
- ACID: Atomicity (all-or-nothing) | Consistency (valid state to valid state) | Isolation (concurrent txns don't see uncommitted changes) | Durability (survives a crash post-commit)
- Index = B-Tree, O(log n) lookup. Clustered = physical row order, one per table. Non-clustered = separate structure + pointer, many per table.
- Every index speeds up reads but slows down writes (must update on every INSERT/UPDATE/DELETE) — don't index everything
- WHERE filters rows before grouping (no aggregates allowed). HAVING filters groups after aggregation (aggregates allowed).
- ROW_NUMBER (unique, no gaps) vs RANK (ties share, gap after) vs DENSE_RANK (ties share, no gap)
- Window functions (OVER/PARTITION BY) keep every row + add a computed column; GROUP BY collapses rows into one per group
- EXPLAIN: watch for Seq Scan (bad, no index used) vs Index Scan / Index Only Scan (good)

Confidential C

PART 12

Design Patterns

A design pattern is a NAMED, reusable solution to a recurring design problem — the value isn't the code (which you could write ad-hoc), it's the shared VOCABULARY. Saying 'I used a Strategy pattern here' tells another engineer the shape of your design in three words instead of a paragraph. Interviewers care less about you reciting UML and more about whether you recognize WHEN a pattern actually fits — and, just as importantly, when it's overkill.

12.1 SOLID Principles — the Foundation Patterns Are Built On

Most design patterns exist to help you follow these five principles in practice. Knowing SOLID cold is arguably asked about MORE often than any individual pattern.

Principle	Meaning
S — Single Responsibility	A class should have ONE reason to change. A class that both validates orders AND sends emails has two reasons to change, and changes to one risk breaking the other.
O — Open/Closed	Open for EXTENSION, closed for MODIFICATION. Add new behavior by adding new code (a new subclass/implementation), not by editing existing, already-tested code.
L — Liskov Substitution	A subclass must be usable anywhere its parent type is expected, without breaking correctness. The classic violation: Square extends Rectangle, but setting Square's width also changes its height, breaking code that expects Rectangle's independent width/height behavior.
I — Interface Segregation	Prefer several small, focused interfaces over one large, bloated one — a class shouldn't be forced to implement methods it doesn't need.
D — Dependency Inversion	Depend on ABSTRACTIONS (interfaces), not concrete implementations — this is literally the principle behind Spring's entire DI approach from Part 6.

Interview trap: Liskov violation example, concretely

```
class Rectangle { void setWidth(int w){...} void setHeight(int h){...} }
class Square extends Rectangle {
   void setWidth(int w) { super.setWidth(w); super.setHeight(w); }  // forces height to match!
}
```

Any code that does rect.setWidth(5); rect.setHeight(10); expecting a 5x10 rectangle silently gets a 10x10 square instead if 'rect' is actually a Square — the subclass breaks the parent's contract, even though it 'is-a' Rectangle conceptually.

12.2 Creational Patterns — Controlling Object Creation

Singleton

Confidential C

Ensures a class has exactly ONE instance, globally accessible. Covered in depth in Part 3 (thread-safe double-checked locking) — the enum-based singleton is worth remembering as the simplest, safest modern implementation:

```
enum ConfigManager {
    INSTANCE;
    private final Map<String,String> config = new HashMap<>();
}
// Enum instances are created ONCE by the JVM, inherently thread-safe, and immune
// to reflection/serialization attacks that can break a hand-rolled singleton.
```

Factory Method

Instead of the caller directly calling 'new ConcreteClass()' (coupling it to a specific implementation), a factory METHOD decides which concrete class to instantiate, based on input.

```
interface Notification { void send(String msg); }
class EmailNotification implements Notification { public void send(String m) {...} }
class SmsNotification implements Notification { public void send(String m) {...} }

class NotificationFactory {
    static Notification create(String type) {
        return switch (type) {
            case "EMAIL" -> new EmailNotification();
            case "SMS"   -> new SmsNotification();
            default -> throw new IllegalArgumentException(type);
        };
    }
}
```

Builder

Solves the 'telescoping constructor' problem — a class with many optional fields would otherwise need a huge combination of overloaded constructors. A Builder lets you set only the fields you care about, fluently, and validates the whole thing at the end.

```
Pizza pizza = Pizza.builder()
        .size("Large")
        .addTopping("Cheese")
        .addTopping("Mushroom")
        .build();
// Modern Java: a 'record' often replaces a simple builder for pure immutable data;
// Lombok's @Builder generates this boilerplate automatically in real codebases.
```

Abstract Factory

A 'factory of factories' — produces a FAMILY of related objects without specifying their concrete classes. E.g. a UIFactory that returns a matched Button + Checkbox + Scrollbar for either a 'Windows' or 'Mac' look-and-feel, guaranteeing you never accidentally mix a Mac button with a Windows checkbox.

Prototype

Create new objects by CLONING an existing 'prototype' instance rather than building from scratch — useful when object construction is expensive (e.g., involves a DB read) and you need many similar-but-slightly-different copies.

Confidential C

12.3 Structural Patterns — Composing Objects

Adapter

Converts one interface into another that the client actually expects — the classic real-world analogy is a physical power plug adapter. Used constantly when integrating a 3rd-party library whose interface doesn't match what your code needs.

```
// 3rd-party library has an incompatible interface:
class LegacyPaymentGateway { void makePayment(String amt) { ... } }

// Your app expects this interface everywhere:
interface PaymentProcessor { void pay(BigDecimal amount); }

class LegacyPaymentAdapter implements PaymentProcessor {
    private final LegacyPaymentGateway legacy;
    public void pay(BigDecimal amount) { legacy.makePayment(amount.toString()); }
}
```

Decorator

Adds new behavior to an object DYNAMICALLY by wrapping it, without touching its class or affecting other instances of that class — the textbook example is Java's own I/O streams.

```
InputStream in = new BufferedInputStream(new GZIPInputStream(new FileInputStream("f.gz")));
// Each wrapper ADDS a capability (buffering, decompression) around the base stream —
// this IS the Decorator pattern, already living in java.io.
```

Facade

Provides a SIMPLE, unified interface hiding a complex subsystem of many interacting classes — e.g. a single OrderFacade.placeOrder() that internally coordinates inventory, payment, and shipping services, so calling code doesn't need to know about (or correctly sequence) all three.

Proxy

A stand-in object that controls access to a REAL object — you already saw this in depth in Part 6: Spring's AOP proxies (for @Transactional, security checks) are a direct real-world application of this pattern.

Composite

Lets you treat a single object and a GROUP of objects through the same interface — classic use case: a file system, where a File and a Folder (which can itself contain Files AND Folders) both implement the same FileSystemNode interface, so client code can recurse without checking which type it has.

Confidential C

12.4 Behavioral Patterns — Communication Between Objects

Strategy

Defines a FAMILY of interchangeable algorithms, each behind a common interface, and lets the client pick/swap which one to use at runtime. This is the single most commonly used pattern in real Spring codebases, usually without anyone naming it explicitly.

```
interface DiscountStrategy { BigDecimal apply(BigDecimal price); }
class BlackFridayDiscount implements DiscountStrategy { ... }
class LoyaltyDiscount implements DiscountStrategy { ... }

class Checkout {
    private final DiscountStrategy strategy;   // injected — could be ANY implementation
    Checkout(DiscountStrategy strategy) { this.strategy = strategy; }
    BigDecimal total(BigDecimal price) { return strategy.apply(price); }
}
```

Recognize this from Part 6?

This is EXACTLY constructor-based dependency injection. Spring's whole DI model is Strategy pattern applied everywhere — every @Autowired interface dependency is a swappable strategy the container wires in for you.

Observer

Defines a one-to-many dependency: when one object (the SUBJECT) changes state, all its registered dependents (OBSERVERS) are notified automatically. Spring's ApplicationEvent/@EventListener mechanism, and message queue pub/sub, are both real-world Observer implementations.

```
@Component
class OrderPlacedListener {
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        emailService.sendConfirmation(event.getOrder());
    }
}
// publisher.publishEvent(new OrderPlacedEvent(order)) — Spring notifies ALL registered
// listeners; the publisher doesn't know or care who's listening.
```

Template Method

Defines the SKELETON of an algorithm in a base class, with certain steps deferred to subclasses — the overall sequence is fixed, only specific steps vary.

```
abstract class DataExporter {
    final void export() {          // template — the FIXED overall sequence
        fetchData();
        transform();
        writeOutput();              // varies per format
    }
    private void fetchData() { ... }
    private void transform() { ... }
    abstract void writeOutput();    // subclasses fill in JUST this step (CSV, JSON, XML...)
}
```

Chain of Responsibility

Passes a request along a CHAIN of handlers, each deciding either to handle it or pass it to the next — the requester doesn't need to know which handler will actually process it. Servlet Filters (from Part 9's Spring Security chain) and Spring's HandlerInterceptor chain are real-world examples.

Command

Confidential C

Encapsulates a REQUEST itself as an object, so it can be queued, logged, undone, or passed around like any other value — the basis for undo/redo functionality and for job queues where 'what to do' is decoupled from 'when it actually runs'.

Confidential C

12.5 Design Patterns Interview Q&A Bank

Q: Explain the Open/Closed Principle with a concrete example of violating it. [Very High]

A class should be extendable without modifying its existing, already-tested source. A violation: a PaymentProcessor with an if/else chain checking payment type ('if CREDIT_CARD... else if PAYPAL...') — adding a new payment method means EDITING that existing class. The fix is a Strategy pattern: a PaymentMethod interface with one implementation per type, so adding a new payment method means adding a NEW class, not touching the existing ones.

Q: What's the practical difference between Strategy and Template Method — they both let you vary behavior, so how do you decide? [High]

Strategy varies the ENTIRE algorithm via composition — you inject a completely different implementation object at runtime (e.g., swap the whole discount calculation). Template Method varies only SPECIFIC STEPS of an otherwise-fixed algorithm via inheritance — the overall sequence stays the same, subclasses just fill in certain steps. Rule of thumb: Strategy for swapping a whole behavior at runtime; Template Method for a fixed process with a few customizable steps.

Q: How does Spring's dependency injection relate to the Strategy and Dependency Inversion principle? [High]

Every constructor-injected interface dependency IS the Strategy pattern in practice — the class depends on an abstraction (the interface) and Spring supplies whichever concrete implementation is wired up, swappable without changing the depending class's code. This is also a direct, textbook application of the 'D' in SOLID: depend on abstractions, not concretions.

Q: Give a real-world example of the Decorator pattern already present in the JDK. [Medium]

java.io streams: new BufferedInputStream(new GZIPInputStream(new FileInputStream(f))) — each wrapper adds one capability (buffering, decompression) around the base stream, and any of them could be composed or omitted independently, without needing a combinatorial explosion of specialized stream subclasses for every combination.

Q: When would you use a Facade instead of just letting client code call the individual subsystem classes directly? [Medium]

When a complex subsystem has multiple interacting classes that must be coordinated in a specific way (e.g., an order needing inventory reservation, payment, and shipping in the correct sequence) — a Facade hides that coordination logic behind one simple method, so callers can't accidentally call the steps out of order or forget one, and changes to the internal coordination don't ripple out to every caller.

Q: Explain the Liskov Substitution Principle and why the classic Square-extends-Rectangle example violates it. [Medium]

LSP requires that a subclass be usable anywhere its parent is expected without breaking correctness. If Square extends Rectangle and overrides setWidth to also change height (to keep it square), any code that sets width and height independently, expecting standard Rectangle behavior, gets silently incorrect results when handed a Square instead — the subclass breaks an implicit behavioral contract of the parent, even though geometrically a square 'is-a' rectangle.

Q: What's the difference between Factory Method and Abstract Factory? [Medium]

Factory Method is a single method that creates ONE type of object, with the concrete class decided by a parameter or overridden by a subclass. Abstract Factory creates an entire FAMILY of related objects (e.g. a matched button + checkbox + scrollbar for one UI theme) through a single factory interface, guaranteeing the family stays consistent — you never end up mixing components from different families.

Confidential C

12.6 Cheat Sheet — Design Patterns & SOLID (last-minute revision)

- SOLID: Single responsibility | Open/closed | Liskov substitution | Interface segregation | Dependency inversion
- Singleton: one instance, globally accessible — prefer enum-based for thread-safety + serialization safety
- Factory Method: one method picks the concrete class. Abstract Factory: a whole FAMILY of related objects, kept consistent.
- Builder: fluent, optional-field object construction — avoids telescoping constructors
- Adapter: converts an incompatible interface into the one you need (3rd-party integration)
- Decorator: wraps an object to ADD behavior dynamically, without touching its class (java.io streams)
- Facade: one simple method hides a complex, multi-class subsystem's coordination logic
- Proxy: a stand-in controlling access to a real object (Spring AOP proxies from Part 6)
- Strategy: swap the WHOLE algorithm via composition/injection — this is literally how Spring DI works
- Observer: one-to-many notify-on-change (Spring @EventListener, pub/sub message queues)
- Template Method: fixed overall sequence, subclasses fill in specific steps
- Chain of Responsibility: a chain of handlers, each can handle or pass along (Servlet Filters, HandlerInterceptor)

Confidential C

PART 13

System Design Basics

At the 3-year-experience level, system design interviews aren't asking you to redesign Google — they're checking that you understand the standard building blocks (caching, load balancing, DB scaling) and can reason about trade-offs out loud. This part covers those building blocks, then walks through one complete example end-to-end.

13.1 Vertical vs Horizontal Scaling

Vertical Scaling (scale UP)	Horizontal Scaling (scale OUT)
Add more CPU/RAM to ONE existing server	Add MORE servers, and distribute load across them
Simple — no code changes, no distributed-systems complexity	Requires load balancing, and often means the app must be STATELESS (no server-local session data)
Hard ceiling — there's a maximum size of machine you can buy	Effectively unlimited — keep adding servers as load grows
Single point of failure remains	Failure of one instance doesn't take the whole system down

Why statelessness matters so much for horizontal scaling

If a server stores session data locally (in memory), a user's SECOND request might land on a DIFFERENT server that has no idea who they are. This is why horizontally-scaled systems either externalize session state (e.g. into Redis, shared across all instances) or use stateless tokens (recall JWT from Part 9) that any instance can verify independently.

13.2 Load Balancing

Algorithm	How it distributes requests
Round Robin	Cycles through servers in order — simple, works well when requests are roughly equal cost
Least Connections	Sends to whichever server currently has the FEWEST active connections — better when request cost varies a lot
IP Hash	Same client IP always routes to the SAME server — useful for simple session affinity ("sticky sessions") without externalizing state
Weighted Round Robin	Like round robin, but bigger/more powerful servers get proportionally more requests

- Load balancers also do health checks — automatically routing around an instance that stops responding, which is exactly why the Actuator /health endpoint from Part 7 matters operationally.

Confidential C

13.3 Caching Strategies

A cache trades a small amount of staleness risk for a large reduction in load on your database/backend — the interview skill is knowing WHICH caching strategy fits which read/write pattern.

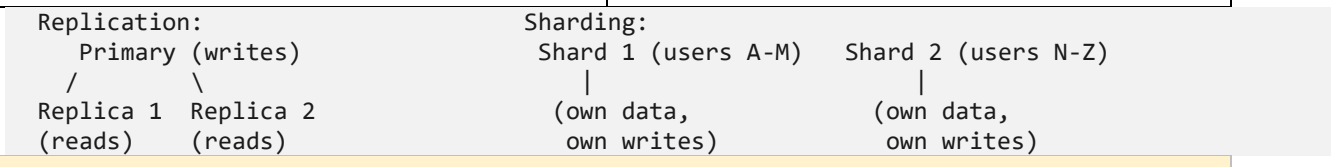
Strategy	How it works / trade-off
Cache-Aside (Lazy Loading)	App checks cache first; on a MISS, reads from DB and populates the cache. Simple and the most common default — but the first request after any miss is always slow, and stale data can linger until it naturally expires (TTL).
Write-Through	Every WRITE goes to the cache AND the DB together, synchronously. Cache is always consistent with the DB, but writes are slower (two systems to update before the write 'completes').
Write-Behind (Write-Back)	Writes go to the cache immediately and are asynchronously flushed to the DB later. Very fast writes, but risks data loss if the cache fails before the flush happens.
Read-Through	The cache ITSELF is responsible for loading from the DB on a miss (rather than the application code doing it) — application code only ever talks to the cache.

Cache Invalidation — 'the two hard problems in CS'

- TTL (Time To Live): simplest approach — every cached entry auto-expires after a fixed duration, accepting some staleness in exchange for simplicity.
- Explicit invalidation: when the underlying data changes, the write path explicitly deletes/updates the corresponding cache entry — more precise, but easy to miss a code path and leave stale data behind.
- Redis is the standard choice for a shared, external cache — an in-memory key-value store, supporting TTLs natively, usable both as a simple cache and (via its data structures) for rate limiting, leaderboards, and pub/sub.

13.4 Database Scaling: Replication & Sharding

Replication	Sharding (Partitioning)
COPIES of the same data across multiple DB servers — typically one WRITE-able primary/master + several READ-only replicas	SPLITS the data itself across multiple DB servers — each shard holds a different SUBSET of rows (e.g. by user_id range or hash)
Solves READ scaling (route reads to replicas) and provides failover	Solves both READ and WRITE scaling — no single server holds all the data or all the write load
Replication lag: a replica can briefly serve slightly stale data right after a write to the primary	Cross-shard queries/joins/transactions become genuinely hard — this is a major complexity cost, not a free win



Interview trap: sharding key choice matters enormously

Confidential C

A poorly chosen shard key (e.g. sharding by signup_date when most active users signed up recently) creates a 'hot shard' — one shard gets disproportionate load while others sit idle, defeating the whole purpose. A well-distributed key (often a hash of user ID) spreads load evenly, at the cost of making range queries ('all users who signed up in March') span multiple shards.

13.5 CDN (Content Delivery Network)

A CDN caches static content (images, CSS, JS, video) on servers physically distributed close to end users geographically, so a user in Mumbai gets served from a nearby edge server instead of a data center in Virginia — cutting latency and offloading traffic from your origin servers entirely for that content.

13.6 Rate Limiting Algorithms

Algorithm	How it works
Token Bucket	A bucket holds up to N tokens, refilled at a steady rate; each request consumes one token, rejected if the bucket is empty. Allows brief BURSTS up to the bucket size, then settles to the steady refill rate.
Leaky Bucket	Requests queue up and are processed at a strictly constant, steady rate, regardless of burstiness — smooths out traffic but adds latency for bursts instead of allowing them.
Fixed Window Counter	Simple counter reset every fixed time window (e.g. per minute) — simple, but allows a burst of 2x the limit right at a window boundary (max requests at the end of one window plus max at the start of the next).
Sliding Window Log/Counter	Tracks requests within a continuously MOVING time window rather than fixed boundaries — more accurate, avoids the boundary-burst problem, at the cost of more memory/computation to track.

13.7 Consistent Hashing (Brief)

A naive 'hash(key) % numServers' approach means adding or removing ONE server reshuffles almost EVERY key's assigned server — catastrophic for a cache (near-total cache miss storm). Consistent hashing places both servers AND keys on a conceptual ring (via hashing); each key belongs to the next server clockwise from it. Adding/removing one server only affects the keys in ITS small slice of the ring, not the whole keyspace — this is what makes systems like distributed caches (and DynamoDB-style databases) able to scale nodes up/down without a massive reshuffle.

Confidential C

13.8 Worked Example: Design a URL Shortener

This is the most commonly asked beginner/mid-level system design question — walking through it end-to-end shows you the STRUCTURE interviewers want from ANY system design answer: requirements first, then API, then data model, then architecture, then the interesting trade-offs.

Step 1: Clarify Requirements

- Functional: given a long URL, generate a short one; given a short URL, redirect to the original.
- Non-functional: high READ:WRITE ratio (many more redirects than URLs created), redirect must be LOW latency, short URLs should be reasonably short and hard to guess sequentially.

Step 2: API Design

```
POST /api/urls  { "longUrl": "https://..." }  ->  { "shortUrl": "https://sho.rt/aZ3kP9" }
GET  /aZ3kP9                                    ->  302 Redirect to the original long URL
```

Step 3: Data Model

```
urls
  short_code   VARCHAR PRIMARY KEY   -- e.g. "aZ3kP9"
  long_url     TEXT NOT NULL
  created_at   TIMESTAMP
  expires_at   TIMESTAMP NULL
```

Step 4: The Interesting Part — Generating the Short Code

Approach	Trade-off
Hash the long URL (e.g. MD5) and truncate	Simple, but different requests for the SAME URL always give the same code (may or may not be desired), and truncated hashes risk collisions requiring a retry/check
Base62-encode an auto-incrementing DB ID	No collisions possible (ID is unique by definition); but a single centralized ID generator is a bottleneck at very high write throughput, and codes are SEQUENTIAL/predictable unless further obfuscated
Pre-generated unique key pool	A background worker pre-generates and stores a pool of valid random unique codes; the create-URL request just claims one from the pool — very fast writes, no runtime collision handling needed

Step 5: High-Level Architecture

```
  Client -> Load Balancer -> [App Server instances] -> Cache (Redis) -> DB
                                    |
                                    +--> on cache MISS for a redirect,
                                         fall through to the DB, then
                                         populate the cache for next time
```

Why caching matters so much specifically here

Given the stated high READ:WRITE ratio, a small set of popular short URLs will account for a hugely disproportionate share of redirect traffic (a power-law/Zipfian distribution, in practice) — caching just those hot entries in Redis can absorb the vast majority of read traffic before it ever touches the database, which is exactly the kind of trade-off reasoning interviewers want to hear you articulate, not just 'add a cache'.

Step 6: Scaling Further

Confidential C

- Read replicas for the DB (high read volume) — recall section 13.4 above.
- Shard the urls table by short_code hash once a single DB can no longer handle write volume — recall the hot-shard trap from earlier in this part.
- A CDN or edge-cache layer for the redirect responses themselves, if latency to a global user base matters.

Confidential C

13.9 System Design Interview Q&A Bank

Q: What's the difference between vertical and horizontal scaling, and why is statelessness important for the latter? [Very High]

Vertical scaling adds resources to one existing machine (simple, but hits a hard ceiling and remains a single point of failure); horizontal scaling adds more machines and distributes load (effectively unlimited, more resilient, but requires coordination). Statelessness matters because if session/user state lives only in one server's memory, a load-balanced request can land on a DIFFERENT server that has no idea who the user is — state needs to be externalized (Redis) or carried in the request itself (a JWT) for horizontal scaling to actually work correctly.

Q: Walk me through cache-aside vs write-through caching, and when you'd pick each. [Very High]

Cache-aside: the app checks the cache first, and on a miss reads from the DB and populates the cache itself — simple, the most common default, but the very first request after a miss is always slow and can serve stale data until TTL expiry. Write-through: every write goes to cache AND DB together synchronously, keeping them always consistent, at the cost of slower writes. I'd pick write-through when read-after-write consistency matters a lot; cache-aside for the common general-purpose case.

Q: What problem does consistent hashing solve that a naive hash(key) % numServers doesn't? [High]

With naive modulo hashing, adding or removing even ONE server changes the divisor and reshuffles almost every key's assigned server — catastrophic for a cache (mass cache-miss storm). Consistent hashing places servers and keys on a ring so that adding/removing a server only remaps the small slice of keys nearest to that server, leaving the rest of the keyspace's assignments untouched.

Q: Explain the difference between database replication and sharding, and what new problem sharding introduces. [Very High]

Replication copies the SAME full data across multiple servers (solves read scaling + failover, at the cost of replication lag). Sharding splits the data itself across servers, each holding a different subset (solves both read AND write scaling), but makes queries/transactions/joins that need to span multiple shards genuinely difficult — and a poorly chosen shard key can create an unevenly-loaded 'hot shard' that defeats the purpose.

Q: Why would you choose Token Bucket over Fixed Window Counter for rate limiting? [Medium]

Fixed Window Counter has a boundary flaw: a client can send the max allowed requests right at the END of one window, and immediately max out again at the START of the next, effectively getting 2x the intended rate in a short burst around the boundary. Token Bucket refills at a steady rate and allows CONTROLLED bursts up to the bucket size, without that specific boundary exploit, while still enforcing a steady-state rate over time.

Q: For a URL shortener, why might Base62-encoding an auto-incrementing ID be preferable to hashing the URL? [Medium]

Hashing (e.g. truncated MD5) can produce COLLISIONS between different URLs, requiring retry/collision-handling logic, and hashing the same URL twice gives the same code (which may or may not be desired behavior). Encoding a unique auto-incrementing ID guarantees no collisions by construction — the trade-off is that a single centralized ID generator can become a bottleneck at very high write throughput, and naive sequential IDs are predictable unless further obfuscated.

Q: When designing a read-heavy system, what's your first instinct for where to add a cache, and why there specifically? [Medium]

Identify where the read traffic is most SKEWED — in most real systems, a small number of 'hot' items account for a disproportionate share of reads (a power-law distribution), so caching just those hot items captures most of the benefit for a fraction of the cache size a full-dataset cache would need. Blindly caching everything wastes memory on rarely-read data that would rarely hit anyway.

Confidential C

13.10 Cheat Sheet — System Design Basics (last-minute revision)

- Vertical scaling = bigger machine, simple, hard ceiling. Horizontal scaling = more machines, needs statelessness + load balancing.
- Load balancing: Round Robin (simple) | Least Connections (variable cost) | IP Hash (sticky sessions)
- Cache-aside (lazy, most common) | Write-through (consistent, slower writes) | Write-behind (fast, riskier) | Read-through (cache handles the miss itself)
- Cache invalidation: TTL (simple, some staleness) vs explicit invalidation (precise, easy to miss a code path)
- Replication = copies data, solves READS + failover. Sharding = splits data, solves READS + WRITES, but cross-shard queries get hard and a bad shard key creates hot shards.
- CDN = caches static content geographically close to users, offloads origin servers
- Rate limiting: Token Bucket (allows controlled bursts) | Leaky Bucket (strictly steady rate) | Fixed Window (simple, boundary-burst flaw) | Sliding Window (accurate, more overhead)
- Consistent hashing: servers+keys on a ring, adding/removing a server only reshuffles its small slice of keys, not the whole keyspace
- System design answer structure: Requirements -> API -> Data model -> High-level architecture -> Scaling/trade-off discussion
- URL shortener key insight: high read:write ratio + power-law hot-key distribution -> caching the hot short codes captures most of the benefit

Confidential C

PART 14

100+ Coding Questions (DSA)

Coding rounds test PATTERN RECOGNITION more than raw cleverness — the same dozen techniques (two pointers, sliding window, BFS/DFS, DP) solve the vast majority of questions you'll actually be asked. This part teaches each pattern with 1-2 fully worked solutions, then gives you a full practice checklist organized the same way, so you drill by pattern instead of randomly.

14.1 Two Pointers & Sliding Window

Use TWO POINTERS when you need to compare/combine elements from different positions in a SORTED (or sortable) structure without nested loops. Use a SLIDING WINDOW when you need the best/valid CONTIGUOUS subarray/substring — grow the window, and shrink it from the left only when a constraint is violated.

Two Sum (sorted array) — Two Pointers

```
int[] twoSumSorted(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;   // need a bigger sum -> move left pointer up
        else right--;               // need a smaller sum -> move right pointer down
    }
    return new int[]{-1, -1};
}
// Time: O(n)  Space: O(1) — beats the O(n^2) brute force by exploiting sorted order
```

Longest Substring Without Repeating Characters — Sliding Window

```
int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (window.contains(s.charAt(right))) {   // constraint violated: shrink from left
            window.remove(s.charAt(left));
            left++;
        }
        window.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
// Time: O(n) — each character is added and removed from the window at most once
```

14.2 Binary Search

Applies any time you can eliminate HALF the remaining search space based on one comparison — not just plain sorted arrays. A strong signal: the problem says 'sorted' or asks for a minimum/maximum value satisfying some monotonic condition.

```
int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;    // avoids int overflow vs (lo+hi)/2
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid + 1;
```


Confidential C

```
        else hi = mid - 1;
    }
    return -1;
}
// Time: O(log n)
```

Interview trap: integer overflow in mid calculation

(lo + hi) / 2 can overflow if lo and hi are both near Integer.MAX_VALUE. lo + (hi - lo) / 2 is the safe form — worth writing this way by default, even if it rarely matters for typical interview input sizes, because interviewers notice.

14.3 Linked Lists

Reverse a Linked List (iterative)

```
ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;   // save the next node before overwriting the link
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;   // prev is now the new head
}
// Time: O(n)  Space: O(1)
```

Detect a Cycle — Floyd's Tortoise and Hare

```
boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;   // fast pointer 'laps' slow inside a cycle
    }
    return false;
}
// Time: O(n)  Space: O(1) — beats an O(n) space HashSet-of-visited-nodes approach
```


Confidential C

14.4 Stacks & Queues

Valid Parentheses

```
boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character,Character> pairs = Map.of(')','(', ']','[', '}','{');
    for (char ch : s.toCharArray()) {
        if (pairs.containsValue(ch)) stack.push(ch);              // opening bracket
        else if (stack.isEmpty() || stack.pop() != pairs.get(ch)) return false;
    }
    return stack.isEmpty();
}
// Time: O(n)  Space: O(n) — ArrayDeque preferred over the legacy Stack class (Part 2)
```

14.5 Trees & BSTs

Almost every tree problem is either a traversal variant, or 'compute something bottom-up via recursion, using the RETURN VALUE of the recursive call on children'. Recognizing which of the two you're doing determines your whole approach.

```
// The 3 depth-first traversal orders — same code shape, order of 3 lines differs
void preorder(TreeNode n)  { if (n==null) return; visit(n); preorder(n.left);
preorder(n.right); }
void inorder(TreeNode n)   { if (n==null) return; inorder(n.left); visit(n); inorder(n.right);
}
void postorder(TreeNode n) { if (n==null) return; postorder(n.left); postorder(n.right);
visit(n); }
// Inorder traversal of a BST specifically visits nodes in SORTED order — a very
// common fact to exploit (e.g. 'validate BST' or 'kth smallest element in a BST')
```

Maximum Depth of a Binary Tree

```
int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
// The classic 'bottom-up' shape: trust the recursive calls to correctly answer
// for the subtrees, then combine their results for the current node.
```

Level Order Traversal (BFS)

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();          // snapshot: how many nodes at THIS level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.add(node.left);
            if (node.right != null) queue.add(node.right);
        }
        result.add(level);
    }
    return result;
}
```


Confidential C

14.6 Graphs: BFS & DFS

BFS	DFS
Explores level-by-level via a Queue — finds the SHORTEST path in an unweighted graph	Explores as deep as possible via a Stack (or recursion) before backtracking — natural fit for exploring ALL paths, cycle detection, topological sort

```
// Number of Islands — a classic grid DFS
int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int col = 0; col < grid[0].length; col++)
            if (grid[r][col] == '1') { count++; sink(grid, r, col); }
    return count;
}
void sink(char[][] grid, int r, int c) {
    if (r<0 || c<0 || r>=grid.length || c>=grid[0].length || grid[r][c]!='1') return;
    grid[r][c] = '0';                 // mark visited by 'sinking' the land
    sink(grid, r+1, c); sink(grid, r-1, c); sink(grid, r, c+1); sink(grid, r, c-1);
}
// Time: O(rows*cols) — each cell visited once
```

14.7 Recursion & Backtracking

Backtracking = try a choice, recurse, then UNDO the choice and try the next one — used whenever you need to explore all combinations/permutations/subsets and prune invalid branches early.

```
List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}
void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current));      // every partial state IS a valid subset
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);                    // choose
        backtrack(nums, i + 1, current, result);  // explore
        current.remove(current.size() - 1);       // un-choose (the actual 'backtrack')
    }
}
// Time: O(2^n) — inherent to generating all subsets
```

14.8 Dynamic Programming

DP applies when a problem has OVERLAPPING SUBPROBLEMS (the same smaller computation gets repeated many times in a naive recursive solution) and OPTIMAL SUBSTRUCTURE (the optimal answer can be built from optimal answers to subproblems). The practical approach: write the brute-force recursion first, identify what repeats, then cache it (memoization) or build it bottom-up (tabulation).

```
// Fibonacci — the canonical example showing WHY memoization matters
// Naive recursion: fib(5) recomputes fib(3) and fib(2) MANY times — O(2^n)
int fibMemo(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];        // already computed — return cached result
    return memo[n] = fibMemo(n-1, memo) + fibMemo(n-2, memo);
}
// Time: O(n) with memoization vs O(2^n) without — this IS the entire point of DP
```

Coin Change (minimum coins to make an amount)

Confidential C

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);        // 'infinity' sentinel — amount+1 coins is never
optimal
    dp[0] = 0;
    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) dp[a] = Math.min(dp[a], dp[a - coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
// Time: O(amount * coins.length)  — bottom-up tabulation, builds smaller amounts first
```

14.9 Greedy Algorithms

A greedy algorithm makes the locally-optimal choice at each step, trusting it leads to a globally-optimal result — this ONLY works for problems that actually have that property (unlike DP, which handles cases where the greedy choice can be wrong). Interviewers often want you to at least CONSIDER greedy first (it's simpler), then explain why it fails if it does.

```
// Maximum number of non-overlapping intervals (classic greedy)
int maxNonOverlapping(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);   // sort by END time — the key insight
    int count = 0, lastEnd = Integer.MIN_VALUE;
    for (int[] interval : intervals) {
        if (interval[0] >= lastEnd) {   // starts after the last chosen interval ended
            count++;
            lastEnd = interval[1];
        }
    }
    return count;
}
// Sorting by END time (not start) is the crux: it always leaves maximum room for future picks
```


Confidential C

14.10 Big-O Cheat Sheet

Structure/Operation	Time Complexity
Array access by index	O(1)
Array search (unsorted)	O(n)
Binary search (sorted)	O(log n)
HashMap get/put (average)	O(1)
Sorting (comparison-based)	O(n log n) — the theoretical best for comparison sorts
Balanced BST operations	O(log n)
BFS/DFS traversal	O(V + E) — vertices plus edges
Recursive Fibonacci without memoization	O(2^n) — the canonical warning example

Confidential C

14.11 Full Practice Checklist — 100+ Problems by Pattern

Use this as a drilling checklist. Problems marked [Very High] are asked constantly across both product and service-based companies; work through those first if time is short.

Arrays & Strings

- Two Sum [Very High] | Best Time to Buy/Sell Stock [Very High] | Move Zeroes | Product of Array Except Self [High] | Maximum Subarray (Kadane's) [Very High] | Merge Intervals [Very High] | Rotate Array | Trapping Rain Water [High] | Container With Most Water [High]
- Longest Substring Without Repeating Characters [Very High] | Minimum Window Substring [High] | Group Anagrams [High] | Valid Anagram | Longest Palindromic Substring [High] | String to Integer (atoi) | Reverse Words in a String | Zigzag Conversion

Searching & Sorting

- Binary Search [Very High] | Search in Rotated Sorted Array [Very High] | Find First and Last Position in Sorted Array | Find Peak Element | Kth Largest Element in an Array [High] | Merge Sort implementation | Quick Sort implementation | Sort Colors (Dutch National Flag)

Linked Lists

- Reverse a Linked List [Very High] | Detect Cycle in a Linked List [Very High] | Merge Two Sorted Lists [High] | Remove Nth Node From End of List | Add Two Numbers (as lists) [High] | Copy List with Random Pointer | Palindrome Linked List | LRU Cache implementation [Very High]

Stacks & Queues

- Valid Parentheses [Very High] | Min Stack | Implement Queue using Stacks | Next Greater Element | Daily Temperatures | Largest Rectangle in Histogram [High] | Evaluate Reverse Polish Notation | Sliding Window Maximum [High]

Trees & BSTs

- Maximum Depth of Binary Tree [Very High] | Validate Binary Search Tree [Very High] | Level Order Traversal [High] | Symmetric Tree | Lowest Common Ancestor [Very High] | Diameter of Binary Tree | Serialize and Deserialize Binary Tree [High] | Kth Smallest Element in a BST | Path Sum | Convert Sorted Array to BST | Binary Tree Zigzag Level Order Traversal

Graphs

- Number of Islands [Very High] | Clone Graph | Course Schedule (topological sort / cycle detection) [Very High] | Word Ladder | Rotting Oranges | Pacific Atlantic Water Flow | Graph Valid Tree | Network Delay Time (Dijkstra)

Recursion & Backtracking

- Subsets [High] | Permutations [High] | Combination Sum | N-Queens | Word Search | Generate Parentheses [High] | Letter Combinations of a Phone Number | Palindrome Partitioning | Sudoku Solver

Dynamic Programming

- Climbing Stairs [Very High] | House Robber [High] | Coin Change [Very High] | Longest Increasing Subsequence [High] | Longest Common Subsequence [High] | Edit Distance [High] | 0/1 Knapsack [Very High] | Word Break | Unique Paths | Maximum Product Subarray | Partition Equal Subset Sum | Decode Ways

Greedy

- Jump Game [High] | Gas Station | Non-overlapping Intervals [High] | Task Scheduler | Meeting Rooms II

Heaps / Priority Queues

Confidential C

- Kth Largest Element in a Stream | Top K Frequent Elements [High] | Merge K Sorted Lists [Very High] | Find Median from Data Stream [High]

Hashing / Design

- Design a Hash Map | Design Twitter | LFU Cache | Insert Delete GetRandom O(1) | Implement Trie (Prefix Tree) [High]

Matrix

- Rotate Image | Spiral Matrix | Set Matrix Zeroes | Word Search II

Confidential C

14.12 DSA Interview Approach Q&A Bank

Q: When you get a new coding problem and don't immediately see the solution, what's your process? [Very High]

Restate the problem in my own words and confirm constraints/edge cases with the interviewer (empty input, duplicates, negative numbers). Work a small example by hand. Start with the brute-force approach out loud, state its complexity, then look for what's being recomputed or what structure (sorted data, a specific pattern like two pointers/sliding window/DP) could improve it — thinking out loud is itself part of what's being evaluated, not just the final code.

Q: How do you decide between recursion with memoization (top-down DP) and an iterative table (bottom-up DP)? [Medium]

Top-down often mirrors the natural recursive brute-force solution more directly and only computes subproblems that are ACTUALLY needed, which can save work if many states are unreachable. Bottom-up avoids recursion call-stack overhead/depth limits and is often easier to further optimize for space (e.g., only keeping the last 2 rows of a DP table). I usually start top-down to get a correct solution fastest, then convert to bottom-up if performance/stack depth matters.

Q: What's the practical difference between BFS and DFS for a graph problem, and how do you decide which to use? [High]

BFS explores level-by-level and is the right choice whenever you need the SHORTEST path/fewest steps in an unweighted graph. DFS explores as deep as possible before backtracking, and fits problems needing to explore ALL paths, detect cycles, or do a topological sort. If the problem says 'minimum/fewest steps', that's a strong BFS signal; if it says 'all possible paths/combinations' or involves ordering with dependencies, that's a DFS signal.

Q: Why is sorting by END time the key insight in interval-scheduling greedy problems? [Medium]

Sorting by end time means at every step you're committing to whichever valid option frees up the earliest possible time for future choices — this greedy choice can be proven to never be worse than any other valid choice at that step (an exchange argument), which is why it provably yields the maximum count of non-overlapping intervals, unlike sorting by start time, which doesn't have that property.

Q: How do you recognize that a problem needs Dynamic Programming rather than plain recursion or greedy? [High]

Look for overlapping subproblems (the same smaller input recurs many times in the brute-force recursion tree) combined with optimal substructure (the best overall answer can be built from best answers to subproblems). If a greedy locally-optimal choice can be proven wrong by a counterexample, but breaking the problem into smaller versions of itself works, that's the DP signal — write the brute-force recursion first, then look at what state actually repeats.

Confidential C

14.13 Cheat Sheet — DSA Patterns (last-minute revision)

- Two pointers: sorted data, compare from both ends, O(n) instead of O(n^2)
- Sliding window: contiguous subarray/substring, grow right, shrink left only when constraint breaks
- Binary search: eliminate half the space each step, O(log n) — use lo + (hi-lo)/2 to avoid overflow
- Fast/slow pointers (Floyd's): cycle detection in O(1) space
- BFS = shortest path / level-by-level, uses a Queue. DFS = explore all paths / cycles / topological sort, uses a Stack or recursion
- Backtracking = choose, recurse, un-choose — for all combinations/permutations/subsets with pruning
- DP = overlapping subproblems + optimal substructure. Write brute force recursion first, then memoize or tabulate.
- Greedy = locally optimal choice, only valid when it provably leads to a globally optimal result — sorting by the right key (often end time) is usually the trick
- Big-O to know cold: array O(1) access, HashMap O(1) avg, binary search O(log n), sorting O(n log n), BFS/DFS O(V+E)

Confidential C

PART 15

One-Day Revision Guide

If you have one day before the interview, don't re-read every part in full — every part already ends with its own detailed cheat sheet. This final part is a SECOND, more compressed pass: the single most important idea per topic, arranged for a fast final skim the night before or the morning of.

15.1 Suggested One-Day Schedule

Time block	Focus
Morning (2-3 hrs)	Re-read every per-part Cheat Sheet section (the last section of Parts 1-14) top to bottom — don't re-read full explanations, just the bullets
Midday (1-2 hrs)	Drill the DSA practice checklist (Part 14.11) — focus on [Very High] tagged problems you haven't solved recently
Afternoon (1 hr)	Re-read this Part 15 once, out loud if possible — explaining concepts out loud surfaces gaps faster than silent reading
Evening	Stop studying at least 1-2 hours before sleep. Review your OWN resume/projects — be ready to explain every line of it in depth (see 15.5)

15.2 The 60-Second Version of Every Part

Core Java

- Overload = compile-time (reference type). Override = runtime (actual object type). Fields are NEVER polymorphic.
- Override equals() -> ALWAYS override hashCode() too, or HashMap/HashSet silently break.
- Checked exceptions = recoverable, compiler-enforced. Unchecked = programmer bugs.

Collections

- HashMap: index=(n-1)&hash, resize at 0.75 load factor, treeify at 8 collisions.
- Fail-fast (ArrayList/HashMap, throws CME) vs fail-safe (CopyOnWriteArrayList/ConcurrentHashMap, snapshot-based).
- Comparable = one natural order, inside the class. Comparator = many orders, outside, chainable.

Multithreading

- volatile = visibility + ordering ONLY, not atomicity. count++ under volatile is still a race.
- Deadlock = 4 Coffman conditions; fix = consistent lock ordering.
- ThreadPoolExecutor flow: core -> queue -> max -> reject. Avoid Executors.newFixedThreadPool in prod (unbounded internals).

Java 8+

- Stream = lazy pipeline, not a data structure, single-use. Nothing runs until a terminal op.

Confidential C

- Predicate=bool | Function=transform | Consumer=action | Supplier=factory.
- Optional: prefer map/orElse/orElseThrow over get(). Return type only, not a field/param.

JVM Internals

- Class loading: Loading -> Linking (Verify/Prepare/Resolve) -> Initialization. Parent-first delegation.
- Young Gen (Eden+S0/S1, Minor GC) -> Old Gen (Major/Full GC, expensive, causes latency spikes).
- GC traces reachability from GC Roots, NOT reference counting — cycles are still collected.

Spring Framework

- Constructor injection preferred: explicit, immutable, testable, fails fast.
- Bean lifecycle: instantiate -> populate -> Aware -> BPP before -> @PostConstruct -> BPP after (AOP proxy created here) -> ready -> @PreDestroy.
- AOP = proxy-based. Self-invocation (this.method()) bypasses the proxy — @Transactional silently no-ops.

Spring Boot

- @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan.
- Auto-config = @Conditional-guarded beans; override by defining your own (@ConditionalOnMissingBean backs off).
- Embedded server = self-contained fat jar, fits Docker/K8s. /actuator/health for liveness/readiness probes.

Hibernate & JPA

- N+1 problem: 1 parent query + N child queries in a loop. Fix with JOIN FETCH / @EntityGraph / batch fetching.
- Default @ManyToOne/@OneToOne to LAZY everywhere; fetch eagerly per-query only when needed.
- Persistence context = 1st-level cache -> powers dirty checking (auto UPDATE on flush, no explicit save()).

Spring Security + JWT

- Security runs in the SERVLET FILTER chain, before DispatcherServlet — rejections never reach your controller.
- JWT payload is base64-ENCODED (readable), NOT encrypted. The SIGNATURE proves authenticity, not secrecy.
- Short-lived access token + longer-lived, revocable refresh token limits damage from a stolen token.

Confidential C

Microservices

- Circuit breaker: CLOSED (normal) -> OPEN (fail fast) -> HALF_OPEN (test recovery).
- Saga pattern replaces distributed ACID transactions: local transactions + compensating actions, EVENTUAL consistency only.
- CAP theorem: partition tolerance is mandatory in practice — the real choice is C vs A DURING a partition.

SQL

- Filtering the right-side table in WHERE after a LEFT JOIN silently turns it into an INNER JOIN — filter in ON instead.
- Index = B-Tree, O(log n). Every index speeds reads but slows writes (updated on every INSERT/UPDATE/DELETE).
- WHERE filters rows before grouping (no aggregates). HAVING filters groups after (aggregates allowed).

Design Patterns & SOLID

- SOLID: Single responsibility | Open/closed | Liskov substitution | Interface segregation | Dependency inversion.
- Strategy pattern = literally how Spring DI works (inject an abstraction, swap the implementation).
- Decorator (java.io streams), Proxy (Spring AOP), Observer (@EventListener) — all patterns already live in code you use daily.

System Design Basics

- Horizontal scaling requires statelessness — externalize session state (Redis) or use stateless tokens (JWT).
- Cache-aside is the default; write-through when read-after-write consistency matters.
- Replication solves reads + failover. Sharding solves reads + writes, but makes cross-shard queries hard and risks hot shards.

DSA

- Sorted data + compare from ends = two pointers. Contiguous subarray/substring = sliding window.
- BFS = shortest path. DFS = all paths / cycles / topological sort.
- DP = overlapping subproblems + optimal substructure. Write brute force first, then memoize/tabulate.

15.3 What to Expect: Typical Round Structure

Round	What's actually being evaluated
Online Assessment / DSA Screen	Pattern recognition and correctness under time pressure — favor a correct O(n log n) solution you finish over an unfinished attempt at optimal
Technical Round(s)	Depth on your OWN stated experience (Java/Spring/SQL) + live coding + system design for more senior/product-company rounds — this is where the 'why', not just 'what', from this handbook pays off
System Design (if applicable at your level)	Structured thinking: requirements -> API -> data model -> architecture -> trade-offs (see Part 13.8's worked example as the template)
Hiring Manager / Behavioral	Ownership, how you handle disagreement/failure, communication clarity — prepare 3-4 concrete STAR-format stories from your actual project experience

Confidential C

15.4 Resume & Project Deep-Dive Prep

For 3 years of experience, interviewers will spend real time on YOUR actual projects — this handbook's job was to make sure the concepts underneath your projects are solid; make sure you can also independently explain:

- For each major project on your resume: what specific PROBLEM it solved, your OWN specific contribution (not the team's), and one hard technical decision you made and why.
- At least one production issue you debugged — what the symptom was, how you diagnosed it, and the actual fix. (Notice how many callouts in this handbook are framed exactly this way — that's not an accident.)
- Be ready to justify technology choices in your projects ('why Spring Boot', 'why this DB') even if the honest answer is 'it's what the team already used' — that's a legitimate answer, said confidently.

15.5 Day-Of Checklist

- Test your camera/mic/IDE screen-share setup the night before if the interview is remote, not 10 minutes before.
- Have a blank text editor or paper ready for coding rounds — know in advance whether they want a shared online editor or your own IDE.
- Prepare 2-3 genuine questions to ask the interviewer at the end — about the team's actual tech stack, on-call practices, or what a typical sprint looks like signals real engagement.
- If you get stuck on a coding question, narrate your thinking rather than going silent — interviewers consistently say a candidate who talks through a partial solution scores better than one who goes quiet.
- It's fine to say "I'm not fully sure, but here's my best reasoning" instead of guessing confidently and being wrong — interviewers notice the difference, and honest uncertainty reads better than false confidence.

You've now covered Core Java through JVM internals, the full Spring/Hibernate/Security stack, microservices, SQL, design patterns, system design, and a full DSA pattern library — with the reasoning behind each answer, not just the answer itself. That's the actual goal: not memorizing this book, but being able to reconstruct any of it on a whiteboard, in your own words, under pressure. Good luck.

Confidential C